THE ETCHED MUTATION

TEM 코드 읽기 교재

쓰지 않고, 읽어서 검사하는 사람이 되기 위한 책

이 책의 모든 예시는 **지금 이 프로젝트 저장소에 실제로 들어 있는 코드**에서 잘라 왔습니다. 남의 예제로 배우고 나서 내 코드에 적용하는 두 단계를, 처음부터 내 코드로 한 단계에 끝내기 위해서입니다.

목표는 하나입니다. AI가 **"고쳤어요"** 라고 말했을 때, 파일을 열어 **진짜인지 눈으로 확인**하는 것. 코드를 쓰는 법은 이 책에 없습니다. 그건 앞으로도 AI가 합니다.

대상 · 박도한 (TEM 제작자)
기준 저장소 · The Etched Mutation, 2026-09-25 시점 (커밋 b894eb1 + 작업 중인 변경분)
구성 · 본문 24장(1~18장 코드 읽기 + 19~24장 엔진 편) + 부록 5편 · 장마다 미니 시험과 필기 칸
엔진 편 · 2026-10-19 추가. 앞 18장은 코드의 **경계**를, 엔진 편은 별이엔진과 오염 추적기의 **속**을 본다. 문체가 다르다(옆에서 말하는 투).
조판 · 오른쪽 여백은 일부러 넓게 비웠습니다. 손으로 쓰세요.
전신 · docs/코드읽기_교재_읽기전용_10교시-260708.md (10교시 전부 이 책 3~10장·15장·18장에 흡수)

차 례

목차

0	이 책을 읽는 법	4
I	**지형 익히기**	
1	지도: 이 프로젝트는 어떻게 생겼나	7
2	웹페이지의 세 겹: html · css · 자바스크립트	11
II	**코드의 세 가지 모양 (7월 10교시 1~5)**	
3	상자 · 목록 · 메모 · 글자	16
4	갈림길: if	20
5	함수: 기능 한 덩어리	24
6	들여쓰기: 목차처럼 훑기	28
7	반복: for · forEach · map · filter	32
III	**이름과 연결 (7월 10교시 6~8)**	
8	이름 읽기: 점 · 낙타 · 밑줄 · let / const / var	35
9	만들기와 부르기: import / export	40
10	데이터 묶음: 객체와 JSON	44
IV	**움직이는 것들**	
11	기다림: async · await · setTimeout	48
12	모듈과 전역: 진입점 세 곳과 캐시버스터	52
13	화면과 사건: DOM과 이벤트	57
V	**검사하는 눈**	
14	오류 읽기: 콘솔 · 네트워크 · 숫자	62
15	비교표(diff) 읽기	66
16	데이터베이스: 표 · SQL · 권한 · jsonb	71

17 3D의 겉모양: Three.js 76

18 큰 파일 지도 + "됐어요" 검사 순서 81

VI 엔진 읽기 (10월 엔진 편 · 속을 본다)

19 엔진이 내놓는 네 가지 85

20 코사인: 두 화살표 사이 각도 90

21 별이엔진 190줄, 한 덩어리씩 95

22 왜 이 식인가: 이론에서 온 것과 직접 정한 것 106

23 오염 추적기: 새 값 10%, 옛 값 90% 112

24 다섯 문제: 돌리기 전에 맞히기 119

부록

A 기호 사전 125

B 코드 영어 단어장 127

C 자주 보는 모양 카드 131

D AI에게 시킬 문구 모음 134

E 미니 시험 정답 135

0장

이 책을 읽는 법

> **목표** 이 책이 무엇을 가르치고 무엇을 가르치지 않는지, 그리고 어떻게 읽어야 남는지를 먼저 정한다.

무엇을 배우는가

딱 하나입니다. **읽기.** 코드를 열어서 "이 파일은 무슨 일을 하는 덩어리들로 되어 있나", "AI가 고쳤다는 자리가 어디고, 정말 거기만 바뀌었나", "빨간 오류 글자가 무슨 뜻인가"를 눈으로 알아보는 것. 이걸 할 수 있으면 지금 프로젝트에서 가장 큰 틈, 즉 "AI가 됐다고 말한 것"과 "진짜 된 것" 사이가 닫힙니다.

무엇을 배우지 않는가

- **코드 쓰기.** 이 책을 다 읽어도 코드는 못 씁니다. 그래도 됩니다. 쓰는 건 AI가 합니다.
- **문법 세부.** 세미콜론을 어디 찍나, 화살표 함수와 일반 함수의 차이가 뭔가 같은 건 안 다룹니다. 읽을 때 걸림돌이 되는 만큼만 짚습니다.
- **알고리즘, 성능, 라이브러리 내부.** Three.js가 어떻게 삼각형을 그리는지는 안 배웁니다. "이 줄이 3D 물체 하나를 만든다"까지만 봅니다.

규칙 세 가지

1. **모든 예시는 실제 TEM 코드다.** 파일 이름과 줄 번호가 상자 위에 적혀 있습니다. 책을 읽다가 실제 파일을 열어 같은 자리를 찾아보면 두 배로 남습니다. 줄 번호는 2026-09-25 기준이라 나중에 조금씩 밀릴 수 있습니다. 그럴 땐 `Ctrl`+`F`로 함수 이름을 찾으세요.
2. **계산의 속은 몰라도 된다.** 이 책에서 "속은 지금 몰라도 됨"이라고 쓴 자리는 정말 몰라도 됩니다. 목표는 경계를 보는 것이지 수식을 푸는 게 아닙니다.
3. **미니 시험은 안 가르친 코드로 낸다.** 각 장 끝의 시험은 그 장에서 안 보여준 진짜 코드입니다. 답은 부록 E에 모아 두었습니다. 먼저 풀고 나서 넘기세요.

책의 모양

표시	뜻
회색 코드 상자	실제 코드. 왼쪽 작은 숫자는 상자 안 줄 번호(파일의 줄 번호가 아님). 상자 위 회색 꼬리표가 파일 경로와 파일 안 줄 번호.
노란 줄	지금 보라는 줄. 그 줄만 보면 됩니다.
연두 상자 "읽는 요령"	그 장의 기술을 한 줄로 줄인 것. 나중에 이것만 다시 봐도 됩니다.
주황 상자 "함정"	이 프로젝트에서 실제로 사람을 속였던 모양.
보라 상자 "실제로 있었던 일"	이 저장소의 기록에 남아 있는 사고. 왜 이 장을 배우는지의 증거.
점선 상자 "✎ 미니 시험"	처음 보는 실제 코드로 내는 문제. 빈칸에 직접 쓰세요.
줄 그어진 칸 "내 손으로 정리"	장 끝. 그 장을 내 말로 세 줄 쓰는 자리.

코드가 하는 일은 결국 세 모양뿐

미리 말해 둡니다. 5만 줄이든 50줄이든, 코드는 아래 세 가지의 조합입니다. 낯선 코드를 만나면 "이건 셋 중 뭐지?"만 물으면 됩니다.

모양	생김새	하는 일
① 저장	`const 이름 = 값`, `[ ]`, `{ }`	값을 이름 붙여 담아 둔다
② 갈림길	`if (조건) { }`	조건에 따라 다른 길로 간다
③ 반복	`for (...) { }`, `.forEach(...)`	같은 일을 목록 개수만큼 한다

여기에 "기능 한 덩어리"(`function`)가 이 셋을 포장하고, "기다림"(`await`)이 시간을 다룹니다. 이 책의 2부와 3부, 4부가 정확히 그 순서입니다.

7월 교재와의 관계

7월 8일에 만든 "코드 읽기 전용 10교시"는 1~5교시까지 진행됐습니다. 그 열 교시는 이 책의 3장부터 10장, 15장, 18장에 전부 들어 있고, 예시와 설명을 늘렸습니다. 그때 본 코드가 다시 나오면 복습이라 생각하고 빨리 넘기세요.

10월에 붙인 엔진 편 (19~24장)

위의 규칙 2 "계산의 속은 몰라도 된다"는 1~18장에만 해당합니다. 10월 19일에 "내 엔진을 내가 설명할 수 있어야 한다"는 요구가 생겨서, 19장부터는 **속을 봅니다**. 별이엔진 190줄을 한 덩어리씩, 오염 추적기의 핵심 한 줄, 그리고 식의 어느 부분이 이론에서 왔고 어느 숫자가 직접 정한 값인지. 문체도 다릅니다. 교재가 아니라 옆에 앉아서 말하는 투로 썼습니다. 1~18장을 다 읽지 않았어도 3장(묶음) · 4장(갈림길) · 7장(반복) · 10장(객체)만 봤으면 19장으로 바로 가도 됩니다.

내 손으로 정리 · 이 책에서 얻고 싶은 것 한 줄

지도: 이 프로젝트는 어떻게 생겼나

> **목표** AI가 "어느 파일을 고쳤다"고 말하면, 그 파일 이름만 듣고 **어느 층의 일인지** 짐작할 수 있게 된다. 코드를 읽기 전에 지도를 먼저 갖는다.

폴더의 큰 그림

저장소 맨 위에는 파일이 아주 많지만(스크린샷 수십 장 포함), 실제로 작품을 이루는 건 아래 뼈대뿐입니다. 이것만 외우면 됩니다.

저장소 뼈대 (2026-09-25)

```
The Etched Mutation/
├─ index.html          ← 진입점 ①  오프닝 · 메뉴 · RECORD · 아카이브
├─ play-test.html      ← 진입점 ②  상영 (1인칭 지형 · 유령 대화 · 봉인)   ※ 30만 바이트, 한 파일
├─ demo-sequence.html  ← 진입점 ③  데모 (js/demo/demoFlow.js 가 두뇌)
├─ admin.html          ← 작가용 편집기 (관객은 못 봄)
├─ css/                ← 옷 (index.css · main-menu.css · tem_cursor.css …)
├─ js/                 ← 신경 (110개 파일, 약 5만 3천 줄)
│   ├─ index.js        ←   index.html 의 두뇌 (866줄)
│   ├─ safety.js       ←   안전 필터
│   ├─ app/            ←   화면 흐름 (opening · archive · confession · recordPlay …)
│   ├─ core/           ←   엔진 (ByeoriEngine · ContaminationTracker · SceneNavigator …)
│   ├─ shared/         ←   공용 계산·지형 (math · api · tem_variant_strata · tem_af_strata_terrain …)
│   ├─ ui/             ←   연출 (lumen_* · strataView · notify …)
│   ├─ lib/            ←   설정·연결 (config · supabaseClient · i18n · repo)
│   ├─ services/       ←   서버 심부름 (NetworkService · AIService …)
│   └─ demo/           ←   데모 전용
├─ supabase/
│   ├─ migrations/     ← 데이터베이스 표를 만들고 고친 기록 (SQL, 날짜순 39개)
│   └─ functions/      ← 서버에서 도는 함수 (record-erosion · claude-scene … 19개)
├─ docs/               ← 정본 문서 189개 (설계 · 결정 · 실험 기록)
├─ test/  tests/       ← 실험 화면 · 자동 검사
├─ tools/              ← 도구 (serve.cjs 로컬 서버 · qa.cjs 순찰 · 저작 자동화 …)
├─ data/  sounds/  assets/  ← 씨앗 데이터 · 소리 · 모형
└─ CLAUDE.md           ← AI 작업 규칙 (311줄)
```

읽는 법

- **html 파일 = 문.** 관객이 브라우저로 여는 건 세 개의 html 뿐입니다. 그 뒤로 css와 js가 끌려 들어옵니다.
- **js 폴더 안은 "층"으로 나뉩니다.** `app`은 화면 흐름, `core`는 판단 엔진, `shared`는 여러 곳이 같이 쓰는 계산, `ui`는 눈에 보이는 연출, `lib`은 바깥 세상(데이터베이스, 언어)과의 연결. AI가 "`js/core/ByeoriEngine.js` 고쳤어요" 하면 판단 엔진을 건드렸다는 뜻이고, "`js/ui/lumen_footprints.js`" 면 보이는 것만 바뀌었다는 뜻입니다.
- **supabase 폴더는 서버 쪽.** 브라우저에서 도는 게 아니라 Supabase 회사 서버에서 돕니다. `migrations`는 표 설계도의 변경 이력이고, `functions`는 관객 브라우저가 직접 못 하는 일(권한 필요한 저장)을 대신 하는 함수들입니다.
- **docs 는 정본.** 코드보다 문서가 먼저입니다. 결정은 여기 남습니다.

진입점 세 곳

이 프로젝트에서 가장 자주 나오는 말이 "진입점 세 곳"입니다. 관객이 들어오는 문이 셋이라, 어떤 기능을 붙일 때 **세 문 모두에 붙였는지** 확인해야 합니다. 한 문에만 붙이고 "됐어요" 하는 게 이 저장소의 단골 사고였습니다.

문	두뇌 파일	무엇을 하나
`index.html`	`js/index.js` (+ `js/app/*`)	오프닝 파동, 메뉴, RECORD(기억 채록), 아카이브, 로그인
`play-test.html`	파일 자기 자신 (안에 script 가 다 들어 있음)	상영. 1인칭으로 지형을 걷고 유령과 대화하고 봉인
`demo-sequence.html`	`js/demo/demoFlow.js`	데모 시연 흐름

> **실제로 있었던 일**
>
> 2026-09-09, RECORD로 바로 가는 단축키(Ctrl+Shift+B)를 붙일 때 기록: "진입점 3곳(index / play-test / demoFlow) 동일 키". 세 문에 같은 키를 붙였다는 걸 코드 주석에 명시했습니다. 이 확인이 없으면 한 문에서만 열리는 단축키가 됩니다. 이 책 18장에서 세 문을 한꺼번에 검사하는 법을 배웁니다.

켜는 법: 바탕화면의 "TEM 켜기"가 하는 일

더블클릭하면 검은 창이 뜨고 브라우저가 열립니다. 그 사이에 일어나는 일은 딱 두 줄입니다.

TEM 켜기.bat

```
@echo off
chcp 65001 > nul
cd /d "%~dp0"
node tools\serve.cjs index.html
```

tools/serve.cjs : 1~3, 19~21

```
// TEM 로컬 서버 — 더블클릭 런처(.bat)에서 호출. 의존성 0 (node 내장 모듈만).
// 사용: node tools/serve.cjs [열_페이지]   (기본 index.html)
// 끄기: 실행된 콘솔 창을 닫으면 됨.

const root = path.join(__dirname, '..'); // tools/ 의 상위 = 프로젝트 루트
const PORT = 8777;
const page = process.argv[2] || 'index.html';
```

읽는 법

- **node**는 자바스크립트를 브라우저 밖에서 돌려주는 프로그램입니다. 4번째 줄은 "node야, `tools/serve.cjs` 파일을 돌려라, 첫 페이지는 index.html 로" 라는 뜻.
- **서버**란 거창한 게 아닙니다. 브라우저가 "index.html 줘" 하면 폴더에서 찾아 건네주는 심부름꾼입니다. `PORT = 8777`은 그 심부름꾼이 서 있는 문 번호. 그래서 주소창이 `localhost:8777`로 시작합니다.
- 주석 1번째 줄의 "의존성 0"은 "남의 부품을 안 빌렸다"는 뜻. 이 파일은 node 만 있으면 돕니다.

> **읽는 요령**
>
> 파일 이름을 들으면 폴더부터 봐라. `js/core/`면 판단, `js/ui/`면 연출, `supabase/`면 서버, `docs/`면 문서. 이 네 글자만 봐도 "무엇이 바뀌었을 수 있는가"의 범위가 정해진다.

✎ 미니 시험 1

1. AI가 "`js/ui/lumen_scene_objects.js` 를 고쳤다"고 합니다. 판단 엔진이 바뀌었을 가능성이 있습니까? 폴더 이름으로 답하세요.
2. 관객이 `play-test.html`을 열었습니다. 이때 `js/index.js`의 코드가 같이 돕니까?
3. "기억 표에 열을 하나 추가했다"는 기록은 어느 폴더에 남아야 합니까?

정답은 부록 E.

내 손으로 정리 · 폴더 다섯 개의 역할을 내 말로

웹페이지의 세 겹: html · css · 자바스크립트

> **목표** 화면 하나가 세 종류의 파일로 이루어진다는 걸 알고, 세 겹이 **이름(id · class)**으로 서로를 찾는다는 것을 실제 메뉴 화면에서 본다.

비유 하나

겹	비유	답하는 질문	이 프로젝트에서
html	뼈대	화면에 **무엇이** 있나	`index.html`, `play-test.html`
css	옷	**어떻게 보이나** (색 · 크기 · 숨김)	`css/index.css`, `css/main-menu.css`
자바스크립트 (js)	신경	무엇을 **하나** (누르면 · 시간이 지나면)	`js/` 전체

html: 상자 속 상자

메뉴 화면의 뼈대입니다. 실제 파일에서 그대로 잘라 왔습니다.

index.html : 144~152, 168~169

```
<div class="intro-screen main-menu-intro" id="introScreen">
    <div class="main-menu-wrap">
        <div class="bg-image"></div>
        <div class="left-gradient"></div>
        <div class="main-content">
            <div class="main-sidebar">
                <div class="main-header">
                    <h1>THE ETCHED<br><span>MUTATION</span></h1>
                <div class="menu-list" id="mainMenuList">
                    <div class="menu-item active" data-menu-id="play" data-action="enterPlayInterview">
```

읽는 법

- **꺾쇠 한 쌍이 상자 하나.** `<div>`로 열고 `</div>`로 닫습니다. 사선(`/`)이 있으면 닫는 쪽. 안에 다른 상자가 들어가고, 들어갈수록 오른쪽으로 밀려 적힙니다

(6장의 들여쓰기와 같은 원리).

- `id` = **이름표, 한 화면에 하나뿐.** `id="introScreen"`은 "이 상자 이름은 introScreen". 자바스크립트가 이 이름으로 찾아옵니다.
- `class` = **종류표, 여러 상자가 같이 가질 수 있음.** `class="menu-item active"`는 종류표 두 장("메뉴 항목"이면서 "켜진 상태"). css가 이 종류표를 보고 옷을 입힙니다.
- `data-…` = **자바스크립트에게 남긴 쪽지.** `data-action="enterPlayInterview"`는 "이걸 누르면 enterPlayInterview 를 하라"고 코드가 읽을 쪽지. 이 프로젝트 메뉴는 전부 이 방식입니다.
- `<h1>`은 큰 제목 상자, `<br>`은 줄바꿈(닫는 짝 없음), `<span>`은 글자 일부만 감싸는 작은 상자.

css: "누구에게" + "무엇을"

css/index.css : 1~8, 4269~4271

```
1 :root {
2   --bg-deep: #0a0a0c;
3   --bg-surface: #12121a;
4   --text-primary: #e8e4df;
5   --text-muted: #6b6b7b;
6   --accent-memory: #c4a882;
7   --accent-interpretation: #7b8fa8;
8 }
9
10 .hidden {
11   display: none !important
12 }
```

읽는 법

- css 한 덩어리는 항상 **"누구에게 { 무엇을: 어떻게; }"** 모양입니다. 10번째 줄: `.hidden`(종류표가 hidden 인 상자들에게) `display: none`(보이지 마라).
- **앞의 점(`.`)은 class, 앞의 우물 정(`#`)은 id**를 가리킵니다. `.hidden`은 종류표, `#selectScreen`은 이름표.
- `!important`는 "다른 규칙과 부딪혀도 이게 이긴다". 이 프로젝트에서 화면을 확실히 숨길 때 씁니다.

- 1번째 줄 `:root` 안의 `--이름: 값`은 **색 이름표**입니다. `--accent-memory`가 기억의 금색. 다른 곳에서 `var(--accent-memory)`라고 쓰면 이 색을 꺼내 씁니다. 색을 한 군데서 바꾸면 전체가 바뀌게 한 장치.
- `#0a0a0c` 같은 것은 색을 숫자로 적은 것. 앞 두 자리 빨강, 가운데 초록, 뒤 파랑. `#0a0a0c`는 거의 검정.

세 겹이 서로를 찾는 방법

이 프로젝트의 화면 전환은 거의 전부 한 가지 동작입니다. **자바스크립트가 id로 상자를 찾아 → class(종류표)를 붙이거나 떼면 → css 가 보이거나 숨긴다.** 실제 코드에서 그 세 줄을 보겠습니다.

`js/index.js : 377 (restart 함수 안, 일부)`

```
1 const introScreen = document.getElementById('introScreen');
2 if (introScreen) { introScreen.classList.remove('hidden');
3                   introScreen.classList.add('visible'); }
```

읽는 법

1. `document.getElementById('introScreen')` = "문서에서 이름표가 introScreen인 상자를 찾아와". 2장 첫 코드의 1번째 줄, 바로 그 상자입니다.
2. `if (introScreen)` = "찾았으면"(못 찾으면 아무것도 안 함). 이 프로젝트 코드에 이 문지기가 수백 번 나옵니다. 없으면 오류로 멈춥니다(14장).
3. `classList.remove('hidden')` = 종류표 hidden 을 뗀다 → css 의 `.hidden` 규칙이 더 이상 안 붙음 → 보인다. `classList.add('visible')` = 종류표 visible 을 붙인다.

즉 **html 이 이름을 달고, js 가 이름으로 찾아 종류표를 바꾸고, css 가 종류표를 보고 옷을 갈아입힙니다.** 이 삼각형만 알면 "메뉴가 안 사라져요" 같은 문제를 어느 겹에서 찾을지 정할 수 있습니다.

겹을 불러오는 줄: link 와 script

`index.html : 15, 17, 599, 612`

```
1 <link rel="stylesheet" href="css/index.css?v=260714">
2 <link rel="stylesheet" href="css/tem_cursor.css?v=260809">
3 <script src="js/ui/strataSection.js?v=260714"></script>
```

```
4 <script type="module" src="js/index.js?v=260909c"></script>
```

읽는 법

- `<link rel="stylesheet" href="…">` = "저 css 파일을 옷으로 입혀라". `<script src="…">` = "저 js 파일을 신경으로 붙여라". html 파일이 다른 두 겹을 여기서 끌어옵니다.
- `?v=260809` 는 **캐시버스터**. 날짜를 적어 두고, 파일을 고치면 이 숫자도 올립니다. 안 올리면 브라우저가 "전에 받아둔 옛 파일"을 계속 씁니다. 이 저장소에서 여러 번 사고가 났고, 12장에서 따로 다룹니다.
- `type="module"` 이 붙은 script 와 안 붙은 script 는 **서로 이름을 못 봅니다.** 이것도 12장.

> **함정**
>
> `play-test.html` 은 세 겹이 한 파일에 다 들어 있습니다(안에 `<style>` 수백 줄, `<script>` 수천 줄). 그래서 30만 바이트, 8천 줄이 넘습니다. 파일 하나가 크다고 겁먹을 것 없습니다. 안에서도 세 겹은 그대로고, 18장의 "큰 파일 지도" 기술로 다룹니다.

✎ 미니 시험 2

1. `<link rel="stylesheet" href="css/tem_cursor.css?v=260809">` 이 줄은 세 겹 중 어느 겹을 불러옵니까?
2. css 에 `.opening-screen.hidden { … }` 이 있습니다(점이 두 번). "종류표 opening-screen 을 가진 상자" 인가요, "종류표 opening-screen 과 hidden 을 **둘 다** 가진 상자" 인가요? 감으로 답하고 이유를 쓰세요.
3. `document.getElementById('landingShelf')` 는 어느 겹이 어느 겹을 찾는 동작입니까?

정답은 부록 E.

내 손으로 정리 · id 와 class 의 차이를 한 줄로

상자 · 목록 · 메모 · 글자

> **목표** 가만히 있는 데이터를 읽는다. 값을 담는 **상자**, 줄 세운 **목록**, 사람만 읽는 **메모**, 따옴표 안의 **글자**. (7월 1교시)

실제 코드: 안전 필터의 단어 목록

js/safety.js : 1~19

```
1  // Safety system — crisis keyword detection and response
2
3  export const SAFETY_KEYWORDS = {
4    block_high: [
5      "suicide", "kill myself", "want to die", "wanna die", "end my life",
6      "end it all", "jump off", "slit my wrist", "cut my wrist",
7      "self-harm", "self harm", "overdose", "hanging", "hang myself",
8      "murder", "kill someone", "stab", "arson", "bomb"
9    ],
10   block_mid: [
11     "gonna kill", "kill them", "beat to death", "blood gushing",
12     "school violence", "bullying", "ostracize"
13   ],
14   monitor_only: [
15     "depressed", "want to disappear", "suffering", "losing my mind", "feel
   empty"
16   ]
17 };
```

읽는 법

- `//` **뒤는 메모.** 컴퓨터는 무시하고 사람만 읽습니다. 막히면 메모부터 찾으세요. 제일 쉬운 단서입니다. 여러 줄 메모는 `/* … */`, 함수 설명용 메모는 `/** … */`로 감쌉니다(모양만 다르고 다 메모).
- `const 이름 =` **는 상자에 이름표 붙이기.** "SAFETY_KEYWORDS 라는 상자를 만들고 `=` 오른쪽 것을 담아라." `export`는 "다른 파일도 이 상자를 쓰게 내놓는다"(9장).
- `[ … ]` **는 목록.** 쉼표로 줄 세운 값들. 4번째 줄부터 `]`까지가 "고위험 단어 목록".

- `" … "` **는 글자 그대로.** `"suicide"`는 계산이 아니라 그냥 그 단어. 따옴표는 큰따옴표 `"`, 작은따옴표 `'`, 뒤로 기운 따옴표 `` ` `` 세 종류가 있고 뜻은 거의 같습니다(뒤로 기운 것은 8장).
- `{ 이름: 값, 이름: 값 }` **는 이름표 달린 묶음.** 3번째 줄의 `{`부터 마지막 `}`까지가 하나의 묶음이고, 안에 `block_high`, `block_mid`, `monitor_only` 세 칸이 있습니다. 목록과의 차이는 10장에서 자세히. 지금은 "이름표로 꺼내는 서랍장" 정도로.

통째로 읽으면: **"안전 키워드 상자 안에 고위험 · 중위험 · 감시만 세 서랍이 있고, 각 서랍에 단어 목록이 들어 있다."** 설계 문서의 3계층이 코드에서 이 모양입니다.

숫자 상자와 글자 상자

index.html : 29~31 (마스터 음량) · js/lib/supabaseClient.js : 7 · tools/serve.cjs : 20

```
var VOL_KEY = 'tem_volume';
var master = parseFloat(localStorage.getItem(VOL_KEY));
if (isNaN(master)) master = 1;
const MAX_WAIT_TIME = 10000; // 10초
const PORT = 8777;
```

읽는 법

- 따옴표가 있으면 글자(`'tem_volume'`), 없으면 숫자(`10000`, `8777`). 숫자에는 따옴표를 안 씁니다.
- `var`는 `const`의 옛날 말입니다. 오래된 코드(play-test.html, 지형 엔진)에 많이 나옵니다. 뜻은 "상자 만들기"로 같습니다. 셋의 차이는 8장.
- `// 10초` 처럼 숫자 옆 메모는 단위를 알려 줍니다. `10000`이 밀리초(천분의 일 초)라 10초. 이런 메모가 없는 숫자를 만나면 AI에게 "단위가 뭐냐"고 물으세요.
- 대문자로만 된 이름(`MAX_WAIT_TIME`, `PORT`)은 관례상 **"안 바뀌는 설정값"**이라는 신호입니다.

목록 속 묶음

js/safety.js : 56~60 (2026-09-09 추가분)

```
export const SAFETY_RESOURCES_KO = [
```

```
  { name: "자살예방 상담전화", number: "109", desc: "24시간", action: "tel:109" },
  { name: "정신건강 위기상담전화", number: "1577-0199", desc: "", action: "tel:15770199" },
  { name: "청소년 상담", number: "1388", desc: "", action: "tel:1388" },
];
```

바깥은 `[ ]` 목록, 안의 각 줄은 `{ }` 묶음. "상담 자원 목록, 항목 하나마다 이름 · 번호 · 설명 · 동작 네 칸." 데이터베이스에서 표를 읽어 오면 정확히 이 모양으로 옵니다(16장). 그러니 이 모양은 이 프로젝트에서 가장 자주 만나는 모양입니다.

읽는 요령

모르는 코드는 먼저 **메모(`//`)**를 찾고, 다음으로 **이름표(`const 이름`)**를 찾고, 그 다음에 오른쪽 값이 **글자인지 숫자인지 목록인지 묶음인지**만 본다. 그 값이 무엇에 쓰이는지는 다음 장부터.

✎ 미니 시험 3

처음 보는 코드입니다. 지형 엔진의 물리 상수.

`js/shared/tem_variant_strata.js : 129~133`

```
var PHYS = {
  inertia: 0.08, capacity: 4, minSlope: 0.01,
  erode: 0.06, deposit: 0.10, evaporate: 0.02,
  gravity: 4, maxSteps: 48, radius: 3,
};
```

1. 상자 이름은?
2. 목록입니까, 이름표 달린 묶음입니까? 어떻게 알았습니까?
3. 칸(이름표)이 몇 개입니까? 값들은 글자입니까 숫자입니까?
4. 이름이 전부 대문자입니다. 무슨 신호입니까?

정답은 부록 E.

내 손으로 정리 · 목록 [ ] 과 묶음 { } 을 구별하는 내 방법

갈림길: if

> **목표** 코드가 "판단하고 나뉘는" 자리를 읽는다. `if`를 보면 두 가지만 묻는다.
> **무슨 질문인가**(괄호 안), **맞으면 뭘 하나**(뒤). (7월 2교시 확장)

실제 코드: "예전에" 인가 "지금" 인가

안전 필터는 "예전에 죽고 싶었다"(과거 이야기)와 "지금 죽고 싶다"(현재 의도)를 구분합니다. 그 판단이 이 세 줄입니다.

`js/safety.js : 78~80 (detectTense 안)`

```
1 if (hasIntent && !hasPast) return 'present';
2 if (hasPast && !hasIntent) return 'past';
3 return 'ambiguous';
```

읽는 법

- `if ( … )` = "괄호 안이 맞으면 뒤엣것을 해라". 괄호 안은 항상 **예/아니오로 답할 수 있는 질문**입니다.
- `hasIntent`, `hasPast`는 안에 예/아니오만 든 상자("의도가 있나?", "과거 얘긴가?"). 영어로 has 가 앞에 붙은 이름은 대개 예/아니오 상자입니다(is, has, can, should 로 시작하면 의심하세요).
- `&&` = 그리고(둘 다 맞아야). `!` = 아니다(뒤집기). 그래서 1번째 줄은 "의도가 있고 **그리고** 과거 얘기가 **아니면** → 답을 '현재'로 정하고 끝".
- `return` = 답을 정해 돌려주고 **그 자리에서 함수를 끝냄**. 1번째 줄에서 return 했으면 2, 3번째 줄은 안 봅니다. 그래서 3번째 줄은 "앞의 두 질문에 다 안 걸리면 → 애매함".

갈림길의 네 가지 생김새

같은 갈림길인데 겉모양이 넷입니다. 다 실제 코드입니다.

① 여러 갈래: if · else if · else

`js/core/ContaminationTracker.js : 243~253`

```
if (
  next.cont_fixation >= HYPERCOMPLETION_FIXATION &&
  next.cont_fixation > next.cont_drift
) {
  next.cont_stage = 'hypercompletion';
} else if (next.cont_drift >= BIASED_INCLINATION_DRIFT) {
  next.cont_stage = 'biased_inclination';
} else {
  next.cont_stage = 'stable';
}
```

`if` → 아니면 `else if` → 그것도 아니면 `else`. 위에서부터 차례로 물어서 **처음 맞는 갈래 하나만** 갑니다. 여기서는 오염 단계 이름을 셋 중 하나로 정합니다.
질문이 길어서 괄호 안을 세 줄로 나눠 적었지만, 1~4번째 줄이 질문 하나입니다.

② 한 줄 갈림길: 물음표와 쌍점

`js/core/ByeoriEngine.js : 81`

```
const voidMod = (userVoid && !origVoid) ? 0.7 : 1.0;
```

`질문 ? 맞으면 : 아니면`. "관객이 회피했고 원본은 회피 안 했으면 0.7, 아니면 1.0". 별이엔진 공식의 void_mod 가 정확히 이 한 줄입니다. 값 하나를 고를 때 if 대신 자주 씁니다.

③ 문지기: 조건이 틀리면 바로 나가기

`js/shared/api.js : 8~13`

```
export async function fetchMemories() {
  const supabaseClient = getSupabaseClient();
  if (!supabaseClient) {
    console.error('fetchMemories: Supabase client not initialized');
    return [];
  }
```

"연결이 없으면 → 오류 메모 남기고 → 빈 목록 돌려주고 끝". 함수 첫머리에 서서 자격 없는 경우를 돌려보내는 **문지기**. 이 프로젝트에서 가장 흔한 if 모양이고, 문지기를 통과해야 아래 진짜 일이 시작됩니다. `return []`가 "빈 목록을 답으로"라는 것도 기억하세요. 화면에 "기억이 하나도 안 떠요"의 원인이 여기일 수 있습니다.

④ 있으면 한다: 짧은 문지기

js/index.js : 354 (backToIntro 안)

```
if (window.soundscape) window.soundscape.stop();
```

"소리 장치가 있으면 멈춰라". 중괄호 없이 한 줄이면 그 한 줄만 조건에 묶입니다. 2장에서 본 `if (introScreen) { … }`도 같은 모양.

질문에 쓰는 기호

기호	뜻	실제 예
`===`	같다 (셋이 정확히 같은지)	`tense === 'past'`
`!==`	다르다	`typeof text !== 'string'`
`<` `>` `<=` `>=`	작다 · 크다 · 작거나 같다 · 크거나 같다	`elapsed >= MAX_WAIT_TIME`
`&&`	그리고 (둘 다)	`hasIntent && !hasPast`
`||`	또는 (하나라도)	`!text || typeof text !== 'string'`
`!`	아니다	`!supabaseClient` (연결이 없으면)
`==` (둘)	대충 같다 (옛 방식). `x != null`은 "비어 있지 않다"	`_enabled != null`

함정

`=` 하나는 "담아라"(3장), `===` 셋은 "같은가?"(질문). 완전히 다른 뜻입니다. if 괄호 안에 `=` 하나가 보이면 십중팔구 실수이니 AI에게 되물으세요.

읽는 요령

if 를 보면 ① 괄호 안 질문을 우리말로 옮기고 ② 맞으면 하는 일을 한 줄로 적는다. `return`이 보이면 "거기서 끝"이라는 것까지. 계산의 속은 몰라도 된다.

✎ 미니 시험 4

처음 보는 코드 둘입니다.

js/lib/supabaseClient.js : 40~46

```
const elapsed = now - initStartTime;
if (elapsed >= MAX_WAIT_TIME) {
    console.error('Supabase CDN load timeout exceeded (10초)');
    throw new Error('Supabase 클라이언트 초기화 실패: CDN 스크립트가 로드되지 않았습니다.');
}
```

tools/serve.cjs : 11~14

```
const cmd =
  process.platform === 'darwin' ? `open "${url}"` :        // 맥
  process.platform === 'win32'  ? `start "" "${url}"` :    // 윈도우
                                  `xdg-open "${url}"`;      // 리눅스
```

1. 첫 코드의 질문을 우리말로. 맞으면 무슨 일이 일어납니까? (`throw`는 "오류를 던져 멈춘다"입니다.)
2. 둘째 코드는 물음표 갈림길이 두 번 이어져 있습니다. 윈도우에서 실행하면 `cmd` 상자에 무엇이 담깁니까?
3. 둘째 코드에서 `process.platform`이 'linux' 면?

정답은 부록 E.

내 손으로 정리 · 문지기 if 의 모양을 그려 보기

함수: 기능 한 덩어리

> **목표** "기능 하나"의 **이름 · 재료 · 몸통 · 답**을 본다. 검증의 단위가 함수다. AI가 "X 함수 고쳤어요" 하면 **그 `{` 부터 `}` 까지만** 보면 된다. (7월 3교시 확장)

실제 코드: 가장 작은 함수

index.html : 32 (마스터 음량 안)

```
1 function clamp01(x) { return Math.max(0, Math.min(1, x)); }
```

읽는 법: 네 부분

부분	여기서는	뜻
이름	`clamp01`	이 기능을 부를 때 쓰는 이름. clamp = 죄다, 01 = 0과 1 사이로.
재료	`(x)`	이 기능이 받는 것. 하나.
몸통	`{ … }`	여는 중괄호부터 닫는 중괄호까지가 이 기능의 전부. **그 밖은 남의 일.**
답	`return …`	돌려주는 값. 여기서는 "x를 0~1 사이로 가둔 값".

속(`Math.max`, `Math.min`)은 몰라도 됩니다. 이름과 메모가 "0~1로 가둔다"고 말해 주니까요. `Math.` 뒤에 붙는 것들은 전부 수학 도구(최대 · 최소 · 반올림 · 제곱근)입니다.

실제 코드: 재료가 둘, 답이 셋 중 하나

js/safety.js : 68~81

```
1 function detectTense(originalText, keywordIndex) {
2   const before = originalText.substring(Math.max(0, keywordIndex - 40),
  keywordIndex).toLowerCase();
3   const after = originalText.substring(keywordIndex,
  Math.min(originalText.length, keywordIndex + 60)).toLowerCase();
4   const window = before + ' ' + after;
5
6   const hasPast = KO_PAST.test(window) || EN_PAST.test(window);
```

```
  const hasIntent = KO_INTENT.test(window) || EN_INTENT.test(window);

  if (hasIntent && !hasPast) return 'present';
  if (hasPast && !hasIntent) return 'past';
  return 'ambiguous';
}
```

읽는 법

- 이름 `detectTense`(시제 감지), 재료 둘(`originalText` 원문, `keywordIndex` 위험 단어가 있던 위치), 몸통은 1번째 줄 `{`부터 12번째 줄 `}`까지.
- 2~7번째 줄은 재료를 손질하는 자리(위험 단어 앞 40글자와 뒤 60글자를 잘라 창을 만들고, 그 창에 과거 표현과 의도 표현이 있는지 봄). 속은 지금 몰라도 됩니다.
- 9~11번째 줄이 4장에서 본 갈림길. **답은 세 글자 중 하나**: 'present', 'past', 'ambiguous'. 함수를 읽을 때 "답이 될 수 있는 값이 뭐뭐인가"를 세면 그 함수를 다 읽은 겁니다.

실제 코드: 답이 없는 함수

tools/serve.cjs : 10~16

```
function openBrowser(url) {
  const cmd =
    process.platform === 'darwin' ? `open "${url}"` :
    process.platform === 'win32'  ? `start "" "${url}"` :
                                    `xdg-open "${url}"`;
  exec(cmd);
}
```

`return`이 없습니다. 이 함수는 답을 돌려주는 게 아니라 **일을 하고 끝**(브라우저를 연다). 답 없는 함수는 "무엇을 바꾸는가"로 읽습니다. 화면을 바꾸거나, 저장하거나, 소리를 내거나. 이 프로젝트의 화면 함수 대부분이 이 모양입니다.

함수의 다른 생김새 세 가지

생김새	실제 예	읽는 법

화살표 함수	`m => ({ ...m, is_live: !!m.is_live })` js/app/archive.js	`재료 => 답`. "m 을 받아 → 오른쪽을 답으로". 이름 없는 짧은 함수. 7장의 반복 안에서 아주 자주 나옵니다.
이름 없는 함수	`card.addEventListener('click', function () { … })` play-test.html : 5230	"click 이 일어나면 이 몸통을 돌려라". 이름이 없고 그 자리에서 바로 넘겨줍니다 (13장).
묶음 안의 함수 (메서드)	`calculateStep(input, context = {}) { … }` js/core/ByeoriEngine.js : 40	`function` 글자가 없어도 `이름(재료) {` 면 함수. `context = {}`는 "안 주면 빈 묶음으로"라는 **기본값**.

실제로 있었던 일

7월 8일 교재에 남은 말: "play-test.html 이 6,965줄 된 게 '이 함수만 바뀌었나'를 안 물어서 생긴 일." 지금은 8,400줄이 넘습니다. 함수 단위로 경계를 묻는 습관이 파일이 불어나는 속도를 늦춥니다.

읽는 요령

AI가 "X 고쳤어요" 하면 `Ctrl`+`F` 로 `function X` 또는 `X(` 를 찾아, 여는 `{` 에서 닫는 `}` 까지만 본다. 답이 될 수 있는 값을 센다. 그 밖은 안 봐도 된다.

✎ 미니 시험 5

js/shared/tem_variant_strata.js : 60~65

```
1 function seedFromString(s) {
2   var h = 5381;
3   s = String(s || 'tem');
4   for (var i = 0; i < s.length; i++) h = ((h << 5) + h +
  s.charCodeAt(i)) >>> 0;
5   return h >>> 0;
6 }
```

1. 이름, 재료 개수, 몸통의 시작과 끝 줄은?
2. 답을 돌려줍니까? 돌려준다면 숫자입니까 글자입니까? (힌트: 2번째 줄에서 `h`가 무엇으로 시작하나)
3. 3번째 줄 `s || 'tem'`: 재료 `s`가 비어 있으면 무엇이 됩니까?
4. 4번째 줄의 속(비트 연산)은 몰라도 됩니다. 그래도 이 함수가 "글자를 받아 숫자 하나로 바꾼다"는 걸 어디서 알 수 있습니까?

정답은 부록 E.

내 손으로 정리 · 함수의 네 부분

들여쓰기: 목차처럼 훑기

> **목표** 큰 파일을 처음부터 읽지 않고, **왼쪽 끝만 보며** "여기 한 뭉치, 저기 다음 뭉치"를 그린다. 덩어리 속 덩어리를 본다. (7월 4교시 확장)

실제 코드: 문지기 안의 갈림길 셋

js/core/ContaminationTracker.js : 236~ (수렴 오염 계산, 속은 생략)

```
if (depth >= MIN_DEPTH_FOR_CONVERGENCE) {
  if (fixation_level >= FIXATION_THRESHOLD) {
    delta_conv = ...
  } else if (effectiveShape >= 0.7 && level >= 0.7) {
    delta_conv = ...
  } else if (alignment >= 0.5) {
    delta_conv = ...
  }
}
```

읽는 법

- `{ }` 안에 또 `{ }`. **안에 든 것일수록 오른쪽으로 밀려 적힙니다.** 그게 들여쓰기입니다.
- 왼쪽 끝만 보면 목차가 보입니다: "바깥 조건 하나(1번째 줄) 안에, 갈림길 세 개(2 · 4 · 6번째 줄)가 나란히."
- **바깥 if 는 문지기.** 궤적이 충분히 깊지 않으면(1번째 줄 질문이 틀리면) 안쪽 세 갈래는 구경도 못 합니다. 그래서 "수렴 오염이 안 올라가요"의 첫 용의자는 안쪽 계산이 아니라 바깥 문지기입니다.
- 속(`delta_conv = ...`)은 몰라도 됩니다. 목표는 **어디부터 어디까지가 한 덩어리인가**뿐.

실제 코드: 네 겹 들여쓰기

15장에서 다시 볼 서가 카드 코드입니다. 이번엔 **계단**만 보세요.

play-test.html : 5209~5240 (일부 생략)

```
function _renderShelf() {
  const shelf = document.getElementById('landingShelf');
```

```
  const enter = document.getElementById('landingEnter');
  if (!shelf) return;

  game.memories.forEach(m => {
    const card = document.createElement('div');
    card.className = 'shelf-card';
    ...
    card.addEventListener('click', function () {
      shelf.querySelectorAll('.shelf-card').forEach(c =>
c.classList.remove('chosen'));
      card.classList.add('chosen');
      document.getElementById('memorySelect').value = m.id;
      if (enter) {
        enter.classList.add('ready');
        try { enter.scrollIntoView({ block: 'nearest', behavior: 'smooth'
}); } catch (_) {}
      }
    });
    shelf.appendChild(card);
  });
}
```

읽는 법

왼쪽 계단을 세면 이렇게 읽힙니다.

1. 0계단: 함수 `_renderShelf`(서가 그리기). 1~21번째 줄 전체.
2. 1계단: 서가 상자를 찾고, 없으면 나감(문지기). 그리고 **기억마다 반복**(6번째 줄 `forEach`).
3. 2계단: 반복 한 번마다 카드 하나를 만들고, 카드에 "클릭되면" 사건을 붙이고(10번째 줄), 서가에 붙임(19번째 줄).
4. 3계단: 클릭됐을 때 하는 일(11~17번째 줄). 다른 카드의 chosen 을 떼고, 이 카드에 붙이고, 숨은 값에 기억 번호를 넣고, 진입 문장이 있으면…
5. 4계단: 진입 문장에 ready 를 붙이고, 그 줄까지 화면을 끌어내림.

`try { … } catch (_) {}`는 "해 보고, 안 되면 조용히 넘어가라"는 안전망(14장). 여기서는 오래된 브라우저에서 부드러운 스크롤이 안 되어도 멈추지 않게 한 것.

짝 찾기

여는 `{`의 짝은 **같은 계단**에 있는 `}`입니다. 편집기에서 여는 괄호에 커서를 두면 짝이 밝게 표시됩니다. VS Code 에서는 `Ctrl`+`Shift`+`\` 로 짝으로 점프합니다. 괄호 짝이 어긋나면 파일 전체가 죽습니다(14장의 SyntaxError).

읽는 요령

큰 파일은 처음부터 읽지 말고, 왼쪽 끝을 훑어 **0계단 줄(function, const, if 가 맨 왼쪽에 있는 줄)**만 세어 지도를 그린다. 바깥 if 가 틀리면 안쪽은 실행 안 된다는 것을 잊지 않는다.

✎ 미니 시험 6

`js/index.js : 124~127, 167 (initApp 안, 중간 생략)`

```
1 if (urlParams.get('strata') === '1' && urlParams.get('memory')) {
2     const strataMemoryId = urlParams.get('memory');
3     ...
4     function _hideAllUI() { ... }
5     function _waitForStrata() { ... }
6     ...
7     return;
8 }
9 if (urlParams.get('demo') === '1' && urlParams.get('memory')) {
```

1. 바깥 if 의 질문을 우리말로 옮기세요. (`urlParams.get('strata')`는 "주소창의 strata 값".)
2. 그 안에 함수 정의가 둘 들어 있습니다. 주소창에 strata=1 이 없으면 이 두 함수는 만들어집니까?
3. 7번째 줄 `return;` 이 실행되면 9번째 줄(demo 검사)은 봅니까?

정답은 부록 E.

내 손으로 정리 · 계단 세는 법

반복: for · forEach · map · filter

> **목표** "목록 전체에 같은 일 다 하기"를 읽는다. 생김새가 다섯 가지지만 뜻은 하나다. **어떤 목록을 · 하나씩 뭐라 부르며 · 뭘 하나.** (7월 5교시 확장)

실제 코드: 단어를 하나씩 검사

js/safety.js : 100~104 (detectCrisis 안)

```
for (const keyword of SAFETY_KEYWORDS.block_high) {
  const normalizedKeyword = keyword.replace(/\s/g, '').toLowerCase();
  const matchIdx = lowerText.indexOf(normalizedKeyword);
  if (matchIdx >= 0) {
    const tense = detectTense(originalLower, matchIdx);
    ...
```

읽는 법

- `for (const keyword of 목록)` = "목록에서 하나씩 꺼내 `keyword`라 부르며 `{ }`를 반복". 단어가 20개면 몸통이 20번 돕니다.
- 반복 **안에** 갈림길(4번째 줄)이 있습니다: "각 위험 단어마다 → 글에 있으면 → 시제를 판별". 3장 + 4장 + 5장(`detectTense` 호출)이 한자리에 모였습니다.

다섯 가지 생김새

생김새	실제 예	읽는 법
`for … of`	`for (const keyword of SAFETY_KEYWORDS.block_high) { … }`	목록에서 하나씩 꺼내며 몸통 반복. 가장 읽기 쉬운 모양.
`.forEach( )`	`game.memories.forEach(m => { … })` play-test.html : 5214	같은 뜻. "기억마다 m 이라 부르며 몸통 반복". 목록 뒤에 점 찍고 붙임.
`.map( )`	`fromSupabaseOther.map(m => ({ ...m, is_live: !!m.is_live }))` js/app/archive.js : 804	반복하면서 **새 목록을 만든다**. 항목마다 화살표 오른쪽 값을 답으로 모아 같은 길이의 새 목록. "기억마다 is_live 칸을 보태 새 목록으로".

`.filter( )`	`readdirSync('.').filter(f => f.endsWith('.html'))` vite.config.js : 6	반복하면서 **골라낸다**. 화살표 오른쪽이 참인 것만 남긴 새 목록. "파일 중 이름이 .html 로 끝나는 것만".
옛날 for	`for (let i = 0; i < scenesData.length; i++) { const scene = scenesData[i]; … }` js/index.js : 423	번호 `i`를 0부터 길이 직전까지 올리며 반복. `목록[i]`가 i 번째 항목. 오래된 코드에 많음. 뜻은 같음.

for · forEach 와 map · filter 의 차이

앞의 둘은 **일을 하고 끝**(화면에 카드를 붙이거나 저장). 뒤의 둘은 **새 목록을 답으로 돌려줌**. 그래서 `map`과 `filter`는 거의 항상 `const 새이름 = 목록.map(…)` 처럼 왼쪽에 상자가 있고, 뒤에 점을 더 찍어 이어 붙이기도 합니다: `목록.filter(…).map(…)` = "골라낸 다음 바꿔라".

실제 코드: 반복 속 반복

js/shared/tem_variant_strata.js : 136~144 (붓 모양 만들기)

```
1 var BRUSH = (function () {
2   var r = PHYS.radius, arr = [], tw = 0, dx, dz, d, w;
3   for (dz = -r; dz <= r; dz++) for (dx = -r; dx <= r; dx++) {
4     d = Math.sqrt(dx * dx + dz * dz);
5     if (d > r) continue;
6     w = 1 - d / r;
7     arr.push({ dx: dx, dz: dz, w: w });
8     tw += w;
9   }
```

3번째 줄에 for 가 두 개 붙어 있습니다. 바깥이 세로(dz), 안이 가로(dx). 격자를 한 칸씩 전부 훑는 모양이고, 지형 코드에 아주 많습니다. `continue`는 "이번 항목은 건너뛰고 다음으로", `arr.push(…)`는 "목록 끝에 하나 추가". 원 밖 칸은 버리고, 원 안 칸만 무게를 매겨 목록에 담는 붓입니다.

읽는 요령

반복을 보면 세 가지만 적는다. ① 어떤 목록을 ② 하나씩 뭐라 부르며 ③ 뭘 하나. `map`이면 "새 목록으로 무엇을 만드나", `filter`면 "무엇을 남기나"를 추가로.

✎ 미니 시험 7

1. 위 붓 코드에서 `PHYS.radius`가 3 이면(3장 시험 참고) 안쪽 몸통은 최대 몇 번 돕니까? (dz 가 -3부터 3까지, dx 도 -3부터 3까지)
2. 처음 보는 코드입니다.

   js/core/store.js : 36~42

   ```
   1 listeners.forEach(listener => {
   2   try {
   3     listener(state, prevState);
   4   } catch (error) {
   5     console.error('[Store] Listener error:', error);
   6   }
   7 });
   ```

   어떤 목록을, 하나씩 뭐라 부르며, 뭘 합니까? 하나가 실패하면 나머지는 돕니까?
3. `['matchingSelection', 'modeSelection', 'sessionSetup'].forEach(id => { const el = document.getElementById(id); if (el) el.style.display = 'none' })` — 우리말로 한 줄.

정답은 부록 E.

내 손으로 정리 · map 과 filter 를 내 말로

이름 읽기: 점 · 낙타 · 밑줄 · let / const / var

> **목표** 영어투성이 이름에서 뜻을 뽑고, 점(`.`)을 "~의"로 읽고, 물음표 점(`?.`)과 세로줄 둘(`||`)의 "없으면" 뜻을 안다. (7월 6교시 확장)

이름을 자르는 세 가지 규칙

규칙	실제 이름	잘라 읽기

낙타 등
(대문자에서 끊기)
	`emotionSimilarity`
	`terrainTintLerp`
	`getSupabaseClient`
		emotion + Similarity = 감정 유사도
		terrain + Tint + Lerp = 지형 · 색조 · 보간 (서서히 섞기)
		get + Supabase + Client = Supabase 연결을 얻어라

밑줄 (밑줄에서 끊기)	`cont_divergence`
	`alignment_score`
	`scene_order`
		cont(contamination 오염) + divergence(발산)
		alignment(정렬) + score(점수)
		scene(씬) + order(순서). 밑줄 이름은 데이터베이스 열 이름에 많음.

앞 밑줄 하나	`_renderShelf`
	`_temEnterSceneMode`
	`_enabled`
		"안에서만 쓰는 것"이라는 관례. 밖에서 부르지 말라는 신호. 뜻은 밑줄 뗀 것과 같음.

앞에 `is`, `has`, `can`이 붙으면 예/아니오 상자(`isEnabled`, `hasPast`). `get`이 붙으면 "얻어 오는 함수", `set`이면 "바꾸는 함수", `render`면 "그리는 함수", `fetch`/`load`면 "서버에서 가져오는 함수", `save`/`record`면 "저장하는 함수". 이 여섯 접두어만 알아도 이 프로젝트 함수의 절반은 이름으로 읽힙니다.

점은 "~의"

play-test.html : 4896 · js/core/ContaminationTracker.js · js/app/archive.js : 809 · js/demo/demoFlow.js : 20

```
game.terrainTintLerp = Math.min(1, game.terrainTintLerp + 0.032);
next.cont_divergence = clamp01(next.cont_divergence + delta_div);
const _uid = appStore.getState().currentUser.id;
window.location.href = 'index.html?action=record';
```

읽는 법

- `game.terrainTintLerp` = "game(회차 상태 묶음)**의** 지형 색조 보간값". 점은 큰 묶음에서 한 칸 꺼내는 것.
- 점이 여러 개면 왼쪽부터 차례로: `appStore.getState().currentUser.id` = "appStore **의** 현재 상태(를 얻어서) **의** 현재 사용자 **의** 번호". 중간에 `( )`가 있으면 그건 함수를 돌려 나온 답에 다시 점을 찍은 것.
- `window.location.href = '…'` = "창 **의** 주소 **의** 값을 이걸로 바꿔라" = **페이지 이동**. 이 한 줄이 나오면 다른 문(1장)으로 넘어가는 자리입니다.
- 1번째 줄처럼 `x = … x + …` 모양은 "x 를 조금 늘려 다시 x 에 담기". 애니메이션에서 매 프레임 조금씩 움직이는 전형(17장).

"없으면" 을 다루는 세 기호

play-test.html : 5223 · js/lib/supabaseClient.js : 59 · test/e2e/playthrough.mjs : 36, 43

```
const n = Number(m._depth) || 0;
console.log(`[Auth] 상태 변경: ${event}`, session?.user?.email || 'no user');
return data?.[0]?.id;
return rt?.isFirstPerson?.();
```

기호	뜻	위 코드에서
`||`	"왼쪽이 비어 있으면 오른쪽을 써라" (기본값)	`Number(m._depth) || 0` = 깊이를 숫자로, 못 바꾸면 0

`?.`	"있으면 점 찍고, 없으면 조용히 비움" (안전한 점)	`session?.user?.email` = 세션이 없거나 사용자가 없어도 오류 없이 빈 값. 그냥 `session.user.email`이면 세션이 없을 때 멈춤(14장의 TypeError)
`??`	"왼쪽이 정말 없을 때만 오른쪽" (0 이나 빈 글자는 그대로 둠)	이 프로젝트엔 드묾. `||`와의 차이만 기억: `0 || 5`는 5, `0 ?? 5`는 0

4번째 줄 `rt?.isFirstPerson?.()`은 "rt 가 있으면 → 그 안에 isFirstPerson 함수가 있으면 → 돌려라". 없으면 어느 단계에서든 조용히 빈 값. 자동 검사 코드가 아직 준비 안 된 화면을 물을 때 쓰는 모양입니다.

뒤로 기운 따옴표: 글자 속에 값 끼우기

js/core/ByeoriEngine.js : 105

```
1 formula: `L${level.toFixed(2)} × S${shapeResult.clamped.toFixed(2)} ×
  V${voidMod.toFixed(2)} = ${alignment.toFixed(3)}`,
```

`` ` … ` `` 안에서 `${ }`는 "여기에 이 값을 끼워라". 별이엔진이 콘솔에 `L0.72 × S0.91 × V1.00 = 0.655` 같은 공식 문장을 찍는 자리입니다. `.toFixed(2)`는 소수 둘째 자리까지. 14장에서 콘솔을 읽을 때 이 문장을 보게 됩니다.

let · const · var

말	뜻	읽을 때 신호
`const`	한 번 담으면 다른 걸로 못 바꾸는 상자	"이 이름은 이 함수 안에서 계속 이 값". 안심하고 읽어도 됨.
`let`	나중에 값이 바뀔 상자	`let delta_conv = 0` = "일단 0, 아래에서 바꿀 예정". 바뀌는 자리를 찾아 읽어야 함.
`var`	`let`의 옛날 말	play-test.html, 지형 엔진, 데이터베이스 연결 코드에 많음. 오래된 코드라는 신호일 뿐 뜻은 `let`과 같다고 보면 됨.

함정

`const`는 "상자를 다른 것으로 못 바꾼다"이지, 상자 안의 묶음 내용은 바꿀 수 있습니다. `const game = {…}` 다음에 `game.terrainTintLerp = 0.5` 는 됩니다. 그래서 이 프로젝트의 `game` 묶음은 const 인데도 매 프레임 바뀝니다.

읽는 요령

모르는 이름은 ① 대문자 · 밑줄에서 자르고 ② 접두어(get/set/is/has/render/fetch/save)로 성격을 잡고 ③ 점은 "의"로 읽는다. `?.`와 `||`는 "없을 때 대비"라고만 읽고 넘어간다.

✎ 미니 시험 8

1. `next.lifetime_drift_sum += driftSignal;` (ContaminationTracker.js : 227) — 이름을 잘라 우리말로. `+=`는 "오른쪽만큼 늘려 담기"입니다.
2. `game.terrainAnimRaf = requestAnimationFrame(tick);` (play-test.html : 4916) — 점 왼쪽은 무엇의 무엇입니까? Raf 는 requestAnimationFrame 의 머리글자입니다. 무슨 값이 담기는지 추측하세요.
3. `const title = m.title || '(untitled)';` — 기억 제목이 비어 있으면 화면에 무엇이 뜹니까?
4. `appStore.getState().currentStoryData?.scenes?.[0]` — 씬 목록이 아직 없으면 오류로 멈춥니까, 조용히 빕니까?

정답은 부록 E.

내 손으로 정리 · 접두어 여섯 개

만들기와 부르기: import / export

> **목표** 같은 기능이 "만들어진 곳"(정의, 한 곳)과 "쓰이는 곳"(호출, 여러 곳)을 구별한다. 파일 첫머리의 `import` 줄만 읽고 **이 파일이 누구에게 기대는지** 지도를 그린다. (7월 7교시 확장)

정의는 하나, 호출은 여럿

`js/safety.js : 68 (정의) · 104 (호출) · js/app/opening.js : 17 (다른 파일에서 꺼내오기)`

```
1 // 만든 곳 - 앞에 function
2 function detectTense(originalText, keywordIndex) { ... }
3
4 // 쓴 곳 - 이름 뒤에 바로 ( )
5 const tense = detectTense(originalLower, matchIdx);
6
7 // 다른 파일의 기능을 꺼내오는 곳
8 import { getSupabaseClient } from '../lib/supabaseClient.js';
```

읽는 법

- **정의(만들기)**: 앞에 `function`이 있거나(2번째 줄), 묶음 안에서 `이름(재료) {` 모양. 보통 딱 한 곳.
- **호출(부르기)**: 이름 뒤에 바로 `( )`(5번째 줄). "그 기계를 지금 돌려라." 여러 곳에서 부를 수 있음. 재료를 괄호 안에 넣어 줌.
- **import**(8번째 줄): "저 파일에서 이 이름을 꺼내 온다". 짝은 그 파일 쪽의 `export`(3장에서 본 `export const`, `export function`).

왜 중요한가: AI가 "어디서 일어나요?"에 답할 때 정의 자리를 말하기 쉽지만, **진짜 도는 자리는 호출 자리**입니다. 정의는 한 곳이라도 호출이 세 곳이면 세 곳 다 봐야 합니다(1장의 진입점 세 곳과 같은 얘기).

실제 코드: 파일의 의존 지도

`js/index.js : 1~10`

```
1 import { initI18n, t, setLanguage, setBrightness, getBrightness,
  getCurrentLanguage } from './lib/i18n.js';
2 import { getSupabaseClient, onAuthStateChange, getSession, getAccessToken }
  from './lib/supabaseClient.js';
3 import { SUPABASE_ANON_KEY, SUPABASE_URL, SUPABASE_FUNCTION_URL } from
  './lib/config.js';
4 import { detectCrisis, getRandomDialogue, CRISIS_DIALOGUES,
  SAFETY_RESOURCES } from './safety.js';
5 import { NPC_DIALOGUES } from './npc-dialogues.js';
6
7 // Shared modules
8 import { fetchMemories, fetchScenes, savePlay, saveNote, fetchNotes,
  activateMemoryIfFetus } from './shared/api.js';
9 import { playSound, stopSound, setVolume, SOUNDS } from
  './shared/audio.js';
10 import { cosineSimilarity, normalizeVector, addVectors, getBucket,
  checkFixated, getDominantEmotion, normalizeAnchor, projectEmotionToVAD,
  emotionVectorToWaveStyle } from './shared/math.js';
```

읽는 법

- 파일 첫머리의 import 줄들 = **이 파일이 남에게 빌려 쓰는 목록**. index.js 는 언어(i18n), 데이터베이스 연결, 설정, 안전 필터, 유령 대사, 서버 심부름, 소리, 수학을 빌립니다. 이 열 줄만 읽어도 index.js 가 무슨 일을 하는 파일인지 절반은 보입니다.
- `from './lib/i18n.js'`의 `./`는 "지금 파일과 같은 폴더에서", `../`는 "한 폴더 위에서". `'../lib/supabaseClient.js'`(opening.js 에서)는 "app 폴더에서 한 칸 올라가 lib 폴더로".
- 중괄호 안 이름들은 저쪽 파일이 `export`한 것 중 골라 온 것. 8번째 줄에서 `fetchMemories`를 가져왔으니, 4장 ③에서 본 그 함수가 index.js 안에서 불립니다.

세 가지 변형

모양	실제 예	뜻
이름 없이 통째로	`import './js/core/DriftPicker.js';` play-test.html : 719	"저 파일을 돌리기만 해라". 꺼내 오는 이름이 없음. 그 파일이 스스로 `window.DriftPicker`를 달아 두는 방식(12장).

필요할 때 꺼내오기	`_liveModule = await import('./app/live.js');` js/index.js : 79	파일 첫머리가 아니라 **함수 안에서** 필요할 때 가져옴. 라이브 모드는 안 쓰니 처음엔 안 읽어 두는 것. `await`는 11장.
옛 방식 (node 도구)	`const http = require('http');` tools/serve.cjs : 4	`require` = import 의 옛날 말. `.cjs` 파일과 도구 코드에서. 뜻은 같음.

함정

import 경로 끝에 `?v=260727`이 붙은 것(`from '../shared/math.js?v=260727'`)은 2장의 캐시버스터가 import 에도 붙은 것입니다. 파일을 고쳤는데 이 숫자를 안 올리면 **브라우저가 옛 math.js 를 씁니다.** 12장에서 전수 대조법을 배웁니다.

읽는 요령

이름 뒤에 `( )`가 붙어 있으면 "지금 돌리는 중"(호출). `function` 뒤에 붙어 있으면 "만드는 중"(정의). 파일을 열면 **import 줄부터 읽어** 의존 지도를 그린다.

✎ 미니 시험 9

js/app/opening.js : 8~14

```
1 import { NPC_DIALOGUES } from '../npc-dialogues.js';
2 import { setLanguage } from '../lib/i18n.js';
3 import { setUserName, getUserName } from './userIdentity.js';
4 import { isTesterMode, TESTER_MEMORY_CODE, TESTER_MEMORY_ID,
  testerMemoryFor } from './testerMode.js';
5 import { buildDoor } from './confession.js';
6 import { _pickTopMemoryForLumen } from './archive.js';
7 import { pickTopMemory, pickGhostVariant } from
  '../core/SeekerMatchEngine.js';
```

1. 이 파일(오프닝)이 기대는 파일은 몇 개입니까?
2. `'./testerMode.js'`와 `'../core/SeekerMatchEngine.js'`는 각각 어느 폴더에 있습니까? (opening.js 는 js/app 안에 있습니다.)
3. `buildDoor`는 이 파일에서 만들어진 함수입니까, 빌려 온 함수입니까? 만들어진 곳은 어느 파일입니까?
4. `getUserName()` 이라고 쓰인 줄을 만났습니다. 정의입니까 호출입니까?

정답은 부록 E.

내 손으로 정리 · 정의와 호출을 구별하는 표시

데이터 묶음: 객체와 JSON

> **목표** `{ }`의 두 번째 뜻(이름표 달린 묶음)을 확실히 하고, 묶음을 풀어 꺼내는 세 가지 모양과 "묶음을 글자로 만든 것"(JSON)을 안다. TEM 의 기억 · 씬 · 회차 한 줄이 전부 이 모양이다. (7월 8교시 확장)

실제 코드: 회차의 상태 전부

js/app/appStore.js : 3~15, 27~33 (일부)

```
export const appStore = createStore({
    // Mode/Session
    currentMode: null,
    currentRole: null,
    sessionCode: null,

    // Memory/Scene
    allMemoriesData: [],
    currentMemory: null,
    currentScene: 0,

    // Alignment/Bucket
    currentAlignment: 0,
    currentBucket: null,
    emotionHistory: [],
    userEmotionTrajectory: [],
});
```

읽는 법

- `{ }`가 두 가지 뜻: (6장) **코드 덩어리**, 안에 명령이 줄줄이. (10장) **데이터 묶음**, 안에 `이름: 값` 짝이 쉼표로. 구별법은 **쌍점(`:`)**. 안에 `이름: 값`이 보이면 묶음.
- 여기는 "회차 상태 묶음". 현재 모드는 아직 없음(`null`), 기억 목록은 빈 목록(`[]`), 현재 씬은 0번, 정렬도는 0. 프로그램이 켜질 때의 초기값입니다.
- `null`은 "값이 없음"을 뜻하는 특별한 값. 8장의 `?.`와 `||`는 이 null 을 만났을 때를 대비한 것.
- 목록 `[ ]`과의 차이: 목록은 **순서**로 꺼냄(`목록[0]`), 묶음은 **이름표**로 꺼냄(`묶음.이름`).

실제 코드: 함수가 묶음을 답으로

js/core/ByeoriEngine.js : 110~115 (calculateStep 의 답)

```
return {
  affective_position: affectivePosition,
  alignment_score: alignment,
  alignment_bucket: bucket,
  transition_pattern: transitionPattern,
  mismatch_type: mismatchType,
```

별이엔진은 숫자 하나를 답하지 않고 **묶음**을 답합니다. 정렬도 점수, 구간, 전이 패턴, 어긋남 종류가 한 묶음에. 부른 쪽은 `result.alignment_score`, `result.transition_pattern`으로 꺼냅니다. 설계 문서의 "엔진 출력"이 코드에서 이 모양입니다.

묶음을 푸는 세 가지 모양

모양	실제 예	뜻
점으로 꺼내기	`result.alignment_score`	8장. "묶음의 그 칸".
한꺼번에 풀기	`const { data, error } = await supabaseClient.from('memories').select('*');` js/shared/api.js : 14 `const { store, engine, ui, visualizer } = deps;` js/app/bindEvents.js : 21	왼쪽 `{ }`는 만드는 게 아니라 **푸는 것**. "오른쪽 묶음에서 data 칸과 error 칸을 꺼내 같은 이름 상자에 담아라". 데이터베이스 호출 뒤에 거의 항상 이 모양(16장).
펼쳐서 새로 만들기	`state = { ...state, ...partial };` js/core/store.js : 31 `{ ...m, is_live: !!m.is_live }` js/app/archive.js	점 셋(`...`) = "이 묶음의 칸을 전부 펼쳐 넣어라". "옛 상태를 다 펼치고 그 위에 바뀐 칸을 덮어 새 묶음". 원본을 안 건드리고 복사본을 만드는 이 프로젝트의 습관.

JSON: 묶음을 글자로

js/index.js : 128~131 (initApp 안)

```
let playTraces = [];
try {
    const raw = localStorage.getItem('tem_play_traces');
    if (raw) playTraces = JSON.parse(raw);
} catch (_e) {}
```

묶음이나 목록은 브라우저 저장소나 데이터베이스에 **글자**로만 넣을 수 있습니다. 그 글자 형식이 JSON 입니다. 생김새는 코드의 묶음과 거의 같고(`{"이름": 값}`, 이름에도 큰따옴표), `JSON.parse(글자)`가 글자를 다시 묶음으로, `JSON.stringify(묶음)`이 묶음을 글자로 바꿉니다. 4번째 줄은 "저장소에서 꺼낸 글자를 목록으로 되돌려라". 데이터베이스의 `meta`, `emotion_dist`, `height_delta` 열이 전부 JSON 을 담는 열(jsonb)입니다(16장).

TEM 의 묶음 세 가지 (참고 카드)

묶음	주요 칸 (데이터베이스 열)	안에 또 묶음
기억 (memories)	id, code, title, description, status, is_public, cont_depth, cont_divergence, cont_convergence, cont_heterogeneity	`meta` = { emotion_entries, key_scenes, author … }
씬 (scenes)	id, memory_id, scene_order, text, echo_words, original_emotion, anchor_emotions, text_stage_1~3	`meta` = { scene_code, motif_tags, stage_position, sound_url, object_models … }
회차 (plays)	id, memory_id, user_id, run_id, alignment, user_reason, created_at	sky_stains, dialog_turns 등 jsonb

CLAUDE.md 7절과 docs/자료스키마_명세_v1-260717.md 가 정본. 이 표는 읽기용 요약입니다.

읽는 요령

`{ }` 안에 **쌍점**이 보이면 묶음. 왼쪽에 `const { a, b } =` 이 보이면 "푸는 중". `...`이 보이면 "펼치는 중". JSON 은 "묶음을 글자로 적은 것"이라고만 기억.

✎ 미니 시험 10

js/demo/demoFlow.js : 29~32

```
const MONOLOGUES = [
  { text: 'I shouldn\'t have done that', entryLine: 'I thought that was the right thing at the time.', vad: { v: -0.5, a: 0.2, d: -0.6 }, x: 0.3, y: 0.35 },
  { text: 'I just wanted to run away', entryLine: 'That was the only thing I could do.', vad: { v: -0.6, a: 0.7, d: -0.4 }, x: 0.65, y: 0.45 },
];
```

1. 바깥은 목록입니까 묶음입니까? 항목이 몇 개입니까?
2. 항목 하나의 칸(이름표)은 몇 개입니까? 그중 **안에 또 묶음**인 칸은?
3. 둘째 항목의 각성도(a)를 꺼내려면 어떻게 씁니까? (목록 번호는 0부터)
4. `enter.scrollIntoView({ block: 'nearest', behavior: 'smooth' })` — 괄호 안에 넘긴 것은 무엇입니까? 칸이 몇 개입니까?

정답은 부록 E.

내 손으로 정리 · 코드 덩어리 { } 와 묶음 { } 을 구별하는 표시

기다림: async · await · setTimeout

> **목표** 코드에서 **시간이 걸리는 자리**를 알아본다. 서버에 물어보는 일, 몇 초 뒤에 하는 일, 매 프레임 하는 일. "저장이 안 됐어요"의 원인 절반이 여기 있다.

왜 기다림이 따로 있나

코드는 위에서 아래로 한 줄씩 돕니다. 그런데 어떤 줄은 **답이 바로 안 옵니다.** 데이터베이스에 "기억 목록 줘"라고 하면 인터넷 건너 서버가 답할 때까지 0.1초든 2초든 걸립니다. 그동안 화면이 멈추면 안 되니까, 자바스크립트는 "답 오면 이어서 해"라고 표시하고 다른 일을 합니다. 그 표시가 `await` 입니다.

실제 코드: 서버에 물어보기

js/shared/api.js : 8~26

```
1  export async function fetchMemories() {
2    const supabaseClient = getSupabaseClient();
3    if (!supabaseClient) {
4      console.error('fetchMemories: Supabase client not initialized');
5      return [];
6    }
7
8    const { data, error } = await supabaseClient
9      .from('memories')
10     .select('*')
11     .eq('is_public', true)
12     .order('created_at', { ascending: false });
13
14   if (error) {
15     console.error('fetchMemories error:', error);
16     return [];
17   }
18
19   return data || [];
20 }
```

읽는 법

- 1번째 줄 `async function` = "이 함수 안에는 기다리는 줄이 있다"는 선언. `await`는 `async` 붙은 함수 안에서만 쓸 수 있습니다.

- 8번째 줄 `await` = "오른쪽 일이 끝날 때까지 여기서 기다렸다가, 답을 왼쪽에 담아라". 8~12번째 줄은 점으로 이어진 한 문장: "memories 표에서 · 모든 열을 · is_public 이 참인 것만 · 만든 시각 내림차순으로" (16장에서 자세히).
- 답이 `{ data, error }` 묶음으로 오고(10장의 "한꺼번에 풀기"), 오류면 빈 목록, 아니면 `data`.

함정

await 가 빠지면 답 대신 "약속 상자"(Promise)라는 빈 껍데기가 옵니다. 오류도 안 나고 저장도 안 됩니다. "저장했다는데 데이터베이스에 없어요"를 들으면 첫 용의자는 await 빠진 줄입니다. AI에게 "그 호출 앞에 await 있어?"라고 물으세요. 반대로 **await 없이 부르는 게 의도인 경우**도 있습니다(기다리지 않고 뒤에서 저장). 그땐 보통 `// 비동기`, `_background…` 같은 표시가 있습니다.

실제 코드: 몇 초 기다리기

js/app/opening.js : 84 (playNpcIntro, 줄 나눔)

```
1 async function playNpcIntro() {
2   ...
3   await new Promise(r => setTimeout(r, 2000));
4   centerWrapper.classList.add('lifted');
5   await new Promise(r => setTimeout(r, 1000));
6   await typeTextAsync(dialogue, NPC_DIALOGUES.intro.firstVisit, 100);
7   ...
```

`await new Promise(r => setTimeout(r, 2000))`는 통째로 **"2초 기다려"**라고 읽으면 됩니다(속은 몰라도 됨). 오프닝이 2초 쉬고 → 가운데 상자를 띄우고 → 1초 쉬고 → 대사를 한 글자씩 치는 흐름이 위에서 아래로 그대로 읽힙니다. `await` 덕분에 시간 흐름이 줄 순서와 같습니다.

시간을 다루는 세 가지 도구

도구	뜻	실제 예
`setTimeout(함수, 밀리초)`	그만큼 뒤에 **한 번** 함수를 돌려라	`setTimeout(initSupabaseClient, 100);` js/lib/supabaseClient.js : 48 — 0.1초 뒤 다시 연결 시도

`setInterval(함수, 밀리초)`	그 간격으로 **계속** 돌려라 (`clearInterval`로 멈춤)	`const timer = setInterval(() => { … }, speed)` js/app/opening.js : 67 — 글자 한 자씩 타이핑
`requestAnimationFrame(함수)`	다음 화면 그리기 직전에 **한 번**. 함수 안에서 또 부르면 **매 프레임** (초당 약 60번)	`game.terrainAnimRaf = requestAnimationFrame(tick);` play-test.html : 4916 — 지형 색이 서서히 바뀌는 애니메이션 (17장)

밀리초는 천분의 일 초. 1000 = 1초, 100 = 0.1초, 50 = 타이핑 한 글자 간격.

옛 모양: then 과 catch

js/app/opening.js : 92 (fadeInSound 안, 줄 나눔)

```
1 const playPromise = audio.play();
2 playPromise.then(() => {
3     console.log('opening 사운드 재생 시작');
4     ...
5 }).catch(e => {
6     console.error('opening 사운드 재생 Failed:', e);
7 });
```

`await`가 생기기 전 방식. `.then(함수)` = "성공하면 이걸 해", `.catch(함수)` = "실패하면 이걸 해". 뜻은 await + try/catch 와 같습니다. 브라우저가 소리 재생을 거부하면(사용자가 아직 화면을 안 눌렀을 때) catch 로 갑니다. 오프닝 소리가 안 나는 문제는 이 catch 의 콘솔 메모에서 시작됩니다.

읽는 요령

`async`는 "안에 기다림 있음", `await`는 "여기서 답 올 때까지". `await new Promise(r => setTimeout(r, N))`는 통째로 "N 밀리초 쉬기". 저장이 안 됐다는 말을 들으면 **await 부터** 의심.

✎ 미니 시험 11

1. `await new Promise(r => setTimeout(r, 1500));` — 우리말로.
2. 자동 검사 코드입니다. 무엇을 기다려 `memId`에 담습니까?

test/e2e/playthrough.mjs : 32~36

```
const memId = await page.evaluate(async () => {
  const sb = await window.getSupabaseClient();
  const { data } = await sb.from('memories').select('id').limit(1);
  return data?.[0]?.id;
});
```

3. `setTimeout(_waitForStrata, 200);` (js/index.js : 158) — 몇 초 뒤에 무엇을 합니까? 이 줄이 `_waitForStrata` 함수 **안에** 있다면 무슨 모양이 됩니까?

정답은 부록 E.

내 손으로 정리 · await 가 빠지면 생기는 일

모듈과 전역: 진입점 세 곳과 캐시버스터

> **목표** 같은 페이지 안에서도 **서로 이름을 못 보는 방**이 있다는 것을 알고, 방 사이를 잇는 `window.`를 읽는다. 그리고 이 저장소 최다 사고 두 가지, **진입점 누락**과 **캐시버스터 미갱신**을 검사하는 법을 익힌다.

두 종류의 script: 방과 마당

2장에서 `<script type="module">`과 그냥 `<script>`를 봤습니다. 차이는 이겁니다.

	일반 script (마당)	module script (방)
이름이 보이는 범위	거기서 만든 이름은 페이지 전체(`window`)에서 보임	그 파일 안에서만 보임. 밖에서 못 부름
import / export	못 씀	씀 (9장)
이 프로젝트에서	`js/ui/lumen_*.js`, `js/shared/tem_*.js`, 지형 엔진 등 대부분의 연출 파일	`js/index.js`, `js/app/*`, `js/core/*`, play-test.html 의 703번째 줄 script

실제 코드: 방에서 마당으로 내놓기

play-test.html : 703, 724~731

```
<script type="module">
  ...
  // Non-module 스크립트(fpAmbientWave 등)에서 접근 가능하게 전역 노출
  window.sharedEmotionVectorToWaveStyle = sharedEmotionVectorToWaveStyle;
  window.GHOST_PRESETS = GHOST_PRESETS;
  const sb = window.supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);
  // strataView.js가 Supabase client를 찾을 수 있도록 HUB 쪽 API를 흉내낸다.
  window.getSupabaseClient = function () { return sb; };
  ...
```

읽는 법

- `window` = 브라우저 창 자체 = **마당**. `window.이름 = 값`은 "마당에 이 이름표를 꽂아 누구나 쓰게 하라".

- 4~5번째 줄: 방(module) 안에서 import 한 것들을 마당에 내놓습니다. 그래야 일반 script 로 된 연출 파일들이 씁니다. 메모(3번째 줄)가 정확히 그 이유를 적어 뒀습니다.
- 9번째 줄: 데이터베이스 연결(`sb`)을 돌려주는 함수를 마당에 꽂음. 그래서 다른 파일이 `window.getSupabaseClient()`라고 부를 수 있습니다(11장 시험의 검사 코드가 바로 그걸 씀).
- 반대로 마당의 것을 방에서 쓸 땐 그냥 `window.supabase`처럼 앞에 window 를 붙입니다(6번째 줄). 마당은 어디서나 보이니까요.

> **실제로 있었던 일**
>
> play-test.html 707~709번째 줄 메모: "calculateFixationLevel 은 이 파일이 이미 인라인 사본을 갖고 있다 — import 하면 이름 충돌로 모듈 전체가 SyntaxError 로 죽는다." 같은 방에 같은 이름이 두 번 생기면 그 방 전체가 안 열립니다. 화면이 통째로 검게 멈추는 사고의 한 종류(14장).

play-test.html : 3533, 5508

```
window._temEnterSceneMode = enterSceneMode;
// 260708 fix: 이 함수는 module 스코프라 뒤쪽 일반 <script> 블록에서 안 보임 — 전역 공유.
```

이런 줄이 이 프로젝트에 수십 개 있습니다. 읽는 법은 하나: **"방 안의 X 를 마당에 꽂음, 이유는 메모에".** 디버깅 때 콘솔에서 `window._temEnterSceneMode(…)`로 강제 진입할 수 있는 것도 이 줄 덕분입니다.

script 줄의 순서

index.html : 598~612 (일부)

```
<script src="data/memories.js"></script>
<script src="js/ui/strataSection.js?v=260714"></script>
<script src="js/shared/tem_af_strata_terrain.js?v=260809a"></script>
<script src="js/ui/strataView.js?v=260810a"></script>
<script src="js/contamination.js?v=260714"></script>
<script src="js/ui/lumen_dialog_phase1.js?v=260909a"></script>
<script type="module" src="js/index.js?v=260909c"></script>
```

위에서 아래로 읽힙니다. 아래 파일은 위 파일이 마당에 꽂은 이름을 쓸 수 있지만, 반대는 안 됩니다. `strataView.js`가 `tem_af_strata_terrain.js`보다 아래 있는 이유입니다. AI 가 "script 줄 순서를 바꿨다"고 하면 "아래 것이 위 것을 쓰나?"를 물으세요.

진입점 세 곳 검사

같은 부품이 세 문에서 각각 불립니다. 예를 들어 유령 대화 껍데기 파일은 두 문에 있습니다.

Ctrl+Shift+F 로 "lumen_dialog_phase1.js" 를 찾으면

```
1 index.html:611      <script src="js/ui/lumen_dialog_phase1.js?v=260909a">
2 play-test.html:99   <script src="js/ui/lumen_dialog_phase1.js?v=260909a">
```

이걸 **grep**(폴더 전체 글자 찾기)이라고 부릅니다. VS Code 에서는 `Ctrl`+`Shift`+`F`. 어떤 기능을 "붙였다"고 하면, 그 기능 이름이나 파일 이름으로 grep 해서 **세 문(index.html · play-test.html · js/demo/demoFlow.js)에 다 나오는지** 셉니다. 그 기능이 어느 문에 필요한지는 작품 판단이지만, "세 곳 중 몇 곳에 있나"는 눈으로 셀 수 있습니다.

캐시버스터 검사

위 grep 결과에서 하나 더 봅니다. **`?v=` 뒤 숫자가 두 줄 모두 같은가.** 260909a 와 260909a, 같습니다. 만약 한쪽만 옛 숫자면 그 문의 관객은 옛 파일을 받습니다.

규칙	검사
파일을 고치면 `?v=`를 올린다 (날짜 + 글자)	AI 가 고쳤다는 파일 이름을 grep → 그 줄의 `?v=`가 오늘 날짜인가
같은 파일은 세 문에서 같은 `?v=`	grep 결과 줄들의 `?v=`가 서로 같은가
import 경로의 `?v=`도 같은 규칙	`from './js/shared/math.js?v=260727'` 처럼 붙은 것도 대상

실제로 있었던 일

2026-07-17 감수 실연 기록: "결함 2건 발견·봉합: 캐시버스터 미갱신 + 접촉-plays 시차 403." 캐시버스터를 안 올려서 지휘 세션이 옛 대화 파일을 보고 있었습니다. 그날 이후 "배포 전 캐시버스터 전수 확인(수정 파일 ↔ ?v= 대조)"이 체크리스트에 올랐고, 이 책 18장의 검사 순서 5번이 그것입니다.

읽는 요령

`window.X =` 는 "마당에 꽂기". 기능을 붙였다는 말을 들으면 파일 이름으로 **grep → 세 문에 있나 → ?v= 같나** 두 가지를 센다. 계산 없이 세는 것만으로 이 저장소 사고의 큰 두 종류를 막는다.

✎ 미니 시험 12

1. `window.QuiltDemoState = QuiltDemoState; // non-module 영역 (footprints, mock test)에서 사용` (play-test.html : 718) — 무엇을 어디에 왜?
2. grep "strataView.js" 결과가 `index.html:603 …?v=260810a`, `play-test.html:6598 …?v=260810a` 입니다. 문제가 있습니까? 만약 한쪽이 260714 였다면 관객에게 무슨 일이 생깁니까?
3. play-test.html 94번째 줄은 `<script type="module" src="js/safety.js">` 입니다(방). 그런데 일반 script 인 유령 대화 파일이 `detectCrisis`를 씁니다. 어떻게 가능합니까? (힌트: safety.js 의 맨 끝 열 줄을 열어 보세요. 열어 보는 것 자체가 답입니다.)

정답은 부록 E.

내 손으로 정리 · 기능 붙였다는 말을 들으면 세는 두 가지

화면과 사건: DOM과 이벤트

> **목표** 자바스크립트가 화면을 **찾고 · 만들고 · 바꾸는** 네댓 개의 동사와, "눌렀을 때 · 키를 쳤을 때" 같은 **사건**을 붙이는 한 가지 모양을 읽는다. 2장의 삼각형을 완성한다.

DOM: 브라우저가 html 로 만든 나무

브라우저는 html 을 읽어 상자 나무를 만듭니다. 그 나무의 이름이 DOM 입니다. 자바스크립트가 "화면을 바꾼다"는 건 전부 이 나무의 상자를 찾아 만지는 일입니다. `document`가 나무의 뿌리.

동사	모양	실제 예

찾기	`document.getElementById('이름표')`
	`document.querySelector('.종류표')`
	`상자.querySelectorAll('.종류표')`
		`document.getElementById('landingShelf')` 하나 찾기
		`document.querySelector('.footer')` 첫 하나
		`shelf.querySelectorAll('.shelf-card')` 서가 안 카드 전부(목록)

만들기	`document.createElement('div')`	카드 상자 하나 새로 만듦 (아직 나무에 안 붙음)
붙이기	`부모.appendChild(자식)`	`shelf.appendChild(card)` 서가 끝에 카드 붙임

바꾸기	`상자.textContent = '글'`
	`상자.classList.add / remove / toggle`
	`상자.style.display = 'none'`
	`상자.value = 값`
	`상자.innerHTML = '<div>…'`
		글자 바꾸기
		종류표 붙이기 · 떼기 · 뒤집기 (2장)
		직접 숨기기
		입력칸 값 넣기
		안쪽 html 통째로 갈아끼우기 (`= ''`는 비우기)

없애기	`상자.remove()`	`modal.remove()` 창 닫기

실제 코드: 카드 만들기 (DOM 동사만 보기)

play-test.html : 5210~5213, 5215~5228, 5239

```
const shelf = document.getElementById('landingShelf');
...
shelf.innerHTML = '';
game.memories.forEach(m => {
  const card = document.createElement('div');
  card.className = 'shelf-card';
  card.dataset.memoryId = m.id;
  const title = document.createElement('span');
  title.className = 'shelf-title';
  title.textContent = m.title || '(untitled)';
  ...
  card.appendChild(title);
  card.appendChild(depth);
  card.addEventListener('click', function () { ... });
  shelf.appendChild(card);
});
```

찾고(1) → 비우고(3) → 기억마다 상자를 만들어(5) 종류표를 달고(6) 쪽지를 달고(7, 2장의 `data-`가 코드에서는 `dataset.`) → 안에 제목 상자를 만들어 글을 넣고(8~10) → 붙이고(12, 13) → 사건을 달고(14) → 서가에 붙임(15). **서가 화면 전체가 이 동사 여섯 개로 그려집니다.**

사건 붙이기

js/demo/demoFlow.js : 17~23

```
window.addEventListener('keydown', function (e) {
  if (!(e.ctrlKey && e.shiftKey) || e.altKey || e.metaKey) return;
  if ((e.code || '') !== 'KeyB' && String(e.key || '').toLowerCase() !==
'b') return;
  e.preventDefault();
  e.stopImmediatePropagation();
  window.location.href = 'index.html?action=record';
}, true);
```

읽는 법

- `대상.addEventListener('사건이름', function (e) { … })` = "대상에서 그 사건이 일어나면 이 함수를 돌려라". 대상은 `window`(창 전체), 사건은 `keydown`(키 누름). 사건 이름은 `click`, `keydown`, `mousemove`, `timeupdate`(소리 재생 위치), `ended`, `DOMContentLoaded`(html 다 읽음) 등.

- `e` = 그 사건의 정보 묶음. `e.ctrlKey`(Ctrl 눌렸나), `e.key`(어떤 키), `e.target`(어디서 일어났나).
- 2~3번째 줄: 문지기 둘. "Ctrl+Shift 가 아니거나 Alt/Meta 가 섞였으면 나가", "B 키가 아니면 나가". 통과하면 Ctrl+Shift+B.
- `e.preventDefault()` = 브라우저가 원래 하려던 일(이 키 조합의 기본 동작) 막기. `e.stopImmediatePropagation()` = 같은 사건을 기다리는 다른 함수들에게 안 넘기기.
- 6번째 줄: 페이지 이동(8장). 마지막 `true`는 "남들보다 먼저 받겠다"는 옵션.

이 일곱 줄이 세 문에 똑같이 있습니다(1장 · 12장). grep "KeyB" 로 셀 수 있습니다.

html 에 직접 적힌 사건

js/index.js : 426 (showOriginalMemory 안, 일부)

```
<button class="close-original-btn" onclick="this.closest('.original-memory-modal').remove()">Close</button>
```

옛 방식. html 속성 `onclick="…"`에 코드를 바로 적음. 뜻은 같습니다("눌리면 가장 가까운 modal 상자를 없애라"). 그리고 2장의 `data-action="enterPlayInterview"` 방식은 `js/app/bindEvents.js`가 메뉴 항목 전체에 사건을 한 번에 붙이고, 눌린 항목의 쪽지를 읽어 무슨 함수를 부를지 정합니다. 메뉴가 안 눌리면 그 파일부터.

시작 사건

js/index.js : 160~164

```
if (document.readyState === 'loading') {
    document.addEventListener('DOMContentLoaded', _waitForStrata);
} else {
    _waitForStrata();
}
```

"html 을 아직 읽는 중이면 다 읽은 뒤에, 이미 다 읽었으면 지금 바로." 상자를 찾기 전에 나무가 완성돼야 하니까요. **찾았는데 없다**(14장의 null 오류)의 원인 중 하나가 이 순서입니다.

함정

같은 상자에 같은 사건을 **두 번** 붙이면 두 번 실행됩니다. 이 프로젝트의 "누적 가드"(260731, 떨림 · 호흡 연출)가 그 사고의 흔적입니다. AI 가 "이벤트 붙였어요" 하면 "화면이 다시 그려질 때 또 붙나?"를 물으세요.

읽는 요령

DOM 동사는 여섯 개(찾기 · 만들기 · 붙이기 · 바꾸기 · 없애기 · 사건 붙이기). `addEventListener('X', function (e) {…})`를 보면 "X 가 일어나면 → 문지기 → 하는 일" 순으로 읽는다.

✎ 미니 시험 13

1. `modal.addEventListener('click', (e) => { if (e.target === modal) { modal.remove() } })` (js/index.js : 426) — 어디를 눌러야 창이 닫힙니까? 창 안의 글을 누르면?
2. `const footer = document.querySelector('.footer'); if (footer) footer.classList.remove('visible');` — 우리말로. `if (footer)`가 없으면 무슨 위험이 있습니까?
3. 12장에서 본 `audio.addEventListener('timeupdate', crossfadeTimeUpdateHandler);` (js/app/opening.js) — 대상과 사건은? 이 사건은 언제 일어날 것 같습니까?

정답은 부록 E.

내 손으로 정리 · 2장의 삼각형에 13장의 동사를 얹어 그리기

오류 읽기: 콘솔 · 네트워크 · 숫자

> **목표** `F12`를 눌러 빨간 글자를 읽는다. 오류 문장의 **종류 · 내용 · 위치** 세 부분을 나누고, 네트워크 숫자(404 · 403 · 409)를 뜻으로 옮긴다. 7월 17일의 사고를 그날 잡을 수 있었던 눈을 만든다.

여는 법

브라우저에서 `F12` (또는 `Ctrl`+`Shift`+`I`). 위쪽 탭에서 **Console**(코드가 남긴 말)과 **Network**(서버와 주고받은 기록)만 봅니다. 나머지 탭은 이 책 범위 밖.

콘솔의 네 가지 줄

색	누가 만드나	뜻	실제 예
흰/검정	`console.log`	그냥 기록. 정상.	`[variant-strata] ON (기본, 260802) – 이본 지층 경로 활성.`
노랑	`console.warn`	경고. 돌긴 하는데 이상함.	`[ByeoriEngine V4] userVector 또는 originalVector 없음`
빨강	`console.error`	코드가 스스로 잡은 오류. 돌긴 함.	`fetchScenes error: {…}`
빨강 + Uncaught	브라우저	아무도 안 잡은 오류. **그 자리에서 멈춤.**	`Uncaught TypeError: …`

대괄호 접두어(`[variant-strata]`, `[Auth]`, `[Store]`, `[door]`)는 이 프로젝트 관례로 **어느 부품이 말하는지**입니다. 콘솔이 수백 줄이어도 대괄호만 훑으면 어느 부품이 시끄러운지 보입니다. 8장에서 본 별이엔진의 `L0.72 × S0.91 × V1.00 = 0.655` 문장도 여기 찍힙니다.

오류 문장 해부

아래는 이 프로젝트 코드 모양으로 만든 **예시**입니다(실제 기록된 문장이 아니라 형태 연습용).

콘솔 예시

```
Uncaught TypeError: Cannot read properties of null (reading 'classList')
    at restart (index.js?v=260909c:377:23)
    at HTMLDivElement.<anonymous> (bindEvents.js:212:9)
```

부분	여기서는	읽는 법
종류	`TypeError`	"값의 종류가 안 맞음". 아래 표 참고.
내용	`Cannot read properties of null (reading 'classList')`	"null(없음)에서 classList 를 읽으려 했다" = **상자를 못 찾았는데 점을 찍었다**. 2장의 `if (introScreen)` 문지기가 빠진 자리.
위치	`restart (index.js:377:23)`	"restart 함수, index.js 파일 377번째 줄 23번째 칸". 첫 `at` 줄이 사고 현장, 그 아래는 거기까지 온 길(누가 불렀나).

종류	뜻	첫 용의자
`TypeError … of null / undefined`	없는 것에 점을 찍음	id 오타 · html 에 그 상자 없음 · 아직 안 만들어짐(13장 시작 사건) · 데이터베이스 답이 비었음
`TypeError: X is not a function`	함수가 아닌 것에 `( )`를 붙임	마당에 안 꽂힘(12장) · 이름 오타 · 옛 파일(캐시버스터)
`ReferenceError: X is not defined`	그런 이름이 없음	방/마당 경계(12장) · script 순서 · 오타
`SyntaxError`	문법이 깨짐. **그 파일 전체가 안 돎**	괄호 짝 어긋남(6장) · 같은 이름 두 번(12장 실제 사례) · 따옴표 안 닫힘
`Failed to load resource`	파일을 못 받음	Network 탭으로 가서 숫자 확인

네트워크 탭의 숫자

Network 탭의 각 줄은 "무엇을 달라고 했고(Name) 뭐라고 답했나(Status)". 빨간 줄만 보면 됩니다.

숫자	뜻	이 프로젝트에서 흔한 원인
200	됐음	정상
304	전에 받은 것 그대로 써	정상. 단, 파일을 고쳤는데 304 면 **캐시버스터 미갱신** (12장)
404	그런 파일 없음	경로 · 파일 이름 오타, 파일 이동. `?v=` 뒤 숫자는 서버가 무시하므로 404 의 원인이 아님
401	누군지 몰라	로그인 만료, 키 없음
403	넌 이거 못 해	**데이터베이스 권한(RLS)**이 거부. 16장. 7월 17일 사고
409	충돌	같은 것을 동시에 두 번 저장. `record-erosion` 함수가 세대 번호 역행을 일부러 409 로 거부(설계)
500 · 502	서버 쪽 고장	Edge Function 안 오류. 브라우저 코드로는 못 고침

실제로 있었던 일

2026-07-17: 접촉이 일어나 기록을 쓰려는데, 그 회차의 plays 행이 아직 저장되기 전이라 권한 규칙이 "네 회차가 아니네" 하고 **403**으로 거부했습니다. 콘솔에는 빨간 403 한 줄이 떴을 겁니다. 그날 지휘 세션이 실제 순서로 밟다가 발견해 "봉인 뒤 재시도"로 봉합(커밋 c850d6c). Network 탭에서 403 을 읽을 줄 알았다면 누구든 그날 잡을 수 있었던 사고입니다. 이 장이 있는 이유.

코드가 오류를 다루는 모양: try / catch

`js/index.js : 128~131 · play-test.html : 5236`

```
try {
    const raw = localStorage.getItem('tem_play_traces');
    if (raw) playTraces = JSON.parse(raw);
} catch (_e) {}

try { enter.scrollIntoView({ block: 'nearest', behavior: 'smooth' }); }
  catch (_) {}
```

`try { 해 봐 } catch (오류) { 실패하면 이거 }`. `catch (_) {}`처럼 **비어 있으면 오류를 조용히 삼킵니다.** 멈추지는 않지만 콘솔에도 안 남습니다. 편리하지만 함정: "왜 안 되지?"의 원인이 빈 catch 안에 숨을 수 있습니다. AI 에게 "이 catch 가 삼키는 오류가 뭐냐"고 물을 수 있는 자리입니다. 반대로 `catch (e) { console.error('…', e) }`는 기록을 남기니 콘솔에서 보입니다.

읽는 요령

빨간 줄은 **종류 · 내용 · 첫 at 줄** 세 조각으로 자른다. null/undefined 면 "못 찾았는데 점 찍음", not defined 면 "이름 없음", SyntaxError 면 "파일 통째로 죽음". Network 는 404 = 없음, 403 = 권한, 409 = 충돌.

✎ 미니 시험 14

1. 콘솔에 `[Auth] 상태 변경: SIGNED_IN sojeong@…` 이 떴습니다. 어느 부품이, 무슨 말을 한 겁니까? 문제입니까?
2. 콘솔에 빨간 `fetchScenes error: {code: '42501', message: 'permission denied for table scenes'}` 가 떴습니다. 어느 함수의 어느 갈래에서 나온 말입니까(4장 ③ 참고)? 화면에는 무엇이 보일 것 같습니까?
3. Network 탭에 `js/ui/strataView.js?v=260810a` 가 빨간 **404** 입니다. 원인 후보를 두 개 쓰고, **아닌** 원인 하나를 쓰세요.

정답은 부록 E.

내 손으로 정리 · 403 과 404 의 차이를 내 말로

비교표(diff) 읽기

> **목표** AI 가 "고쳤다"고 할 때 **전과 후를 나란히** 본다. 비교표의 머리글 · 덩어리(hunk) · 빨강 · 초록을 읽고, "세 곳 고쳤다"가 정말 세 곳인지 **센다**. 이 책에서 가장 중요한 장. (7월 9교시 확장)

비교표의 생김새

아래는 9월 15일 커밋 하나의 실제 비교표 전문입니다. 서가가 창보다 길면 진입 문장이 잘리던 문제를 고친 것.

git show b894eb1 (play-test.html, 세 덩어리 전부)

```
diff --git a/play-test.html b/play-test.html
--- a/play-test.html
+++ b/play-test.html
@@ -111,14 +111,18 @@
     * { box-sizing: border-box; }
-    #selectScreen { min-height: 100vh; display: flex; flex-direction: column;
align-items: center; justify-content: center; background: #0a0a0e; }
+    /* 260915: 서가가 화면보다 길어지면(기억 11개째부터 ~920px 창에서 발생) "Enter
this memory" 줄이
+       body overflow:hidden 아래로 밀려 누를 수 없었다. 서가 화면만 스스로 스크롤하
고, 세로 가운데는
+       justify-content:center 대신 첫/끝 자식의 margin:auto 로 — 넘치면 위에서부
터 보이고 끝까지 스크롤된다. */
+    #selectScreen { height: 100vh; overflow-y: auto; display: flex; flex-
direction: column; align-items: center; justify-content: flex-start;
background: #0a0a0e; }

     #selectScreen { gap: 0; padding: 6vh 24px; position: relative; }
-    #landingHead { text-align: center; margin-bottom: clamp(2.4rem, 6vh,
4rem); }
+    #landingHead { text-align: center; margin-top: auto; margin-bottom:
clamp(2.4rem, 6vh, 4rem); flex-shrink: 0; }
+    #landingShelf { flex-shrink: 0; }
@@ -156,7 +160,7 @@
     #landingEnter {
-      margin-top: clamp(2rem, 5vh, 3.2rem); min-height: 2.6em;
+      margin-top: clamp(2rem, 5vh, 3.2rem); margin-bottom: auto; min-height:
2.6em; flex-shrink: 0;
       display: flex; align-items: center;
@@ -5225,7 +5229,11 @@
         card.classList.add('chosen');
         document.getElementById('memorySelect').value = m.id;
```

```
-        if (enter) enter.classList.add('ready');
+        if (enter) {
+          enter.classList.add('ready');
+          // 260915: 서가가 화면보다 길면 진입 문장이 아래로 숨는다 — 고른 순간 그
줄까지 끌어내린다.
+          try { enter.scrollIntoView({ block: 'nearest', behavior: 'smooth'
}); } catch (_) {}
+        }
       });
```

읽는 법: 네 부분

1. **머리글** `--- a/play-test.html` / `+++ b/play-test.html`: a 가 전, b 가 후. 어느 파일인지.
2. **덩어리(hunk) 표시** `@@ -111,14 +111,18 @@`: "전 파일 111번째 줄부터 14줄 ↔ 후 파일 111번째 줄부터 18줄". 이 비교표엔 `@@`가 **세 개** = 세 자리를 고쳤다. 두 자리는 css 쪽(111, 156번째 줄 근처), 한 자리는 자바스크립트 쪽(5225번째 줄 근처).
3. **빨강(`-`)** = 지운 줄, **초록(`+`)** = 새로 넣은 줄, 앞에 아무 표시 없는 줄 = 안 바뀐 문맥(어디인지 알려 주려고 같이 보여 줌).
4. **초록만 읽으면 "새로 뭘 하기로 했나"**가 나옵니다. 첫 덩어리: 서가 화면 높이를 딱 화면만큼 하고 넘치면 스스로 스크롤(`overflow-y: auto`), 가운데 정렬을 위쪽 정렬로. 둘째: 진입 문장 아래 여백 자동. 셋째: 카드를 고르면 `ready`를 붙이는 한 줄이 `{ }` 블록이 되고, **그 줄까지 화면을 끌어내리는 한 줄이 추가**(6장에서 본 그 코드).

빨강과 초록이 짝으로 있으면 "바꿈", 초록만 있으면 "추가", 빨강만 있으면 "삭제". 첫 덩어리의 `#selectScreen` 줄은 빨강 하나 초록 하나로 **바꿈**이고, 그 사이 초록 세 줄은 메모 **추가**.

요약 숫자: --stat

```
git show --stat b894eb1
```

```
1 play-test.html | 16 ++++++++++++----
2 1 file changed, 12 insertions(+), 4 deletions(-)
```

"파일 하나, 12줄 늘고 4줄 줄었다". `+`와 `-` 개수가 대략 비율. 이 숫자는 **양**만 말합니다. "한 줄 고쳤어요"라는 말에 stat 이 `+340 -2` 면 되물을 자리.

커밋 메시지 읽기

git log --oneline (최근 4개)

```
b894eb1 fix(play-test): 서가가 창보다 길면 진입 문장이 잘리던 문제 — 서가 화면 자체 스크롤 + 카드 선택 시 자동 스크롤 (260915)
4401fdf feat(authoring): 소설→기억 저작 파이프라인 v0.1 + 권역(GWON-001) 첫 실전 (260915)
1afeee4 fix(reveal): 봉인 서사 이중 노출 제거 — 엔딩 후속 화면은 문장 생략 (260810)
c7f99d2 feat(dialog): EN 연결사 8종 — 이어쓰기 선택지도 언어를 탄다 (260810)
```

`번호 종류(범위): 요약 (날짜)`. 종류는 `fix`(고침) · `feat`(새 기능) · `docs`(문서) · `chore`(잡일) · `refactor`(정리, 동작 불변). 앞의 일곱 글자 번호(`b894eb1`)가 그 커밋의 이름이라 AI 에게 "b894eb1 비교표 보여줘"라고 할 수 있습니다.

아직 커밋 안 한 변경: git diff

오늘(9월 25일) 저장소에는 커밋 안 된 변경이 12개 파일에 있습니다. 그중 안전 필터 파일의 비교표입니다.

git diff js/safety.js (미커밋)

```
@@ -52,6 +52,18 @@ export const SAFETY_RESOURCES = [
   }
 ];

+// 260909 RECORD v4 §5-1 안전 단일화 — recordChat.js 에만 있던 한국 상담전화를 여기로 이관.
+export const SAFETY_RESOURCES_KO = [
+  { name: "자살예방 상담전화", number: "109", desc: "24시간", action: "tel:109" },
+  { name: "정신건강 위기상담전화", number: "1577-0199", desc: "", action: "tel:15770199" },
+  { name: "청소년 상담", number: "1388", desc: "", action: "tel:1388" },
+];
+
+/** 언어별 상담 자원 — ko 면 한국, 그 외 미국 목록. */
+export function getSafetyResources(lang) {
+  return String(lang || '').slice(0, 2) === 'ko' ? SAFETY_RESOURCES_KO : SAFETY_RESOURCES;
+}
+
 // ── Tense detection (시제 판별) ──
@@ -269,6 +281,8 @@ if (typeof window !== 'undefined') {
     safetyForAbsorb,
     detectCrisis,
     ABSORB_GENERIC_KIN,
```

```
+    getSafetyResources,   // 260909 RECORD v4
+    resetSafetySession,
   };
 }
```

덩어리 둘. 첫째는 초록만(**추가**): 한국 상담전화 목록과, 언어에 따라 목록을 고르는 함수. 둘째도 초록만: 파일 끝의 "마당에 꽂는 묶음"(12장)에 두 이름 추가. `@@` 줄 오른쪽 끝에 `export const SAFETY_RESOURCES = [`, `if (typeof window …` 처럼 **그 덩어리가 어느 덩어리 안인지** 힌트가 붙습니다. 12장 시험 3번의 답이 여기 보입니다.

어디서 보나

- VS Code 왼쪽 **소스 제어** 아이콘(가지 모양) → 바뀐 파일 목록 → 파일을 누르면 전/후가 나란히. 빨강 · 초록 색도 같습니다.
- AI 에게: "요약 말고 `git diff` 출력을 그대로 붙여 줘." 부록 D 에 문구를 모아 뒀습니다.

검증 다섯 질문

1. **바꾼 함수 이름이 뭐야?** (5장 · `@@` 줄 오른쪽 힌트)
2. **덩어리(`@@`)가 몇 개야? 각각 어느 함수 안이야?** "세 곳 고쳤다" ↔ `@@` 세 개.
3. **초록 줄이 새로 하는 일을 내 말로.** 못 하겠으면 AI 에게 한 줄씩 풀어 달라고.
4. **그 함수 밖도 건드렸어?** 건드렸으면 왜.
5. **stat 숫자가 말과 맞아?** "한 줄"인데 +340 이면 되묻기.

> **읽는 요령**
> 비교표는 `@@` 개수부터 센다. 그다음 초록 줄만 읽어 "새로 하는 일"을 적는다. 빨강 · 초록 짝 = 바꿈, 초록만 = 추가. 속은 몰라도 세 모양(if · for · 이름)만 보면 성격이 잡힌다.

✎ 미니 시험 15

git diff js/app/archive.js (미커밋)

```
@@ -804,7 +804,10 @@ async function loadMemoriesFromSupabase() {
     console.log('[loadMemoriesFromSupabase] merged', merged.length,
'memories');
-    appStore.setState({ allMemoriesData: merged });
+    // 260909 RECORD v4 §6-①: v4 기록 씨앗(Fetus)은 기록자 본인에게만 — PLAY
풀·ARCHIVE 목록 공통.
+    const { filterVisibleMemories } = await
import('../shared/tem_record_visibility.js?v=260909a');
+    const _uid = (appStore.getState().currentUser &&
appStore.getState().currentUser.id) || null;
+    appStore.setState({ allMemoriesData: filterVisibleMemories(merged, {
userId: _uid }) });
     const state = appStore.getState();
```

1. 어느 함수 안입니까? 덩어리는 몇 개?
2. 빨강 한 줄과 초록 네 줄. **전에는** 무엇을 했고 **지금은** 무엇을 합니까? (힌트: 9장의 "필요할 때 꺼내오기", 7장의 filter)
3. stat 에 `js/index.js | 44 ++++` 만 보입니다. 이 숫자만으로 알 수 있는 것과 없는 것을 하나씩.

정답은 부록 E.

내 손으로 정리 · 검증 다섯 질문을 내 문장으로

데이터베이스: 표 · SQL · 권한 · jsonb

> **목표** 데이터베이스가 **표**라는 것, 표를 만들고 고치는 글(SQL)을 읽는 법, 코드가 표를 읽고 쓰는 한 문장, 403 의 근원인 **권한 규칙(RLS)**, 그리고 "저장돼 있다 ≠ 쓰인다"를 안다.

표 · 행 · 열

Supabase 의 데이터베이스는 엑셀과 같습니다. **표**(memories, scenes, plays …)가 있고, **행** 하나가 기억 하나 · 씬 하나 · 회차 하나, **열**이 칸(title, scene_order, alignment …). 행마다 `id`라는 주민번호가 있고 모양은 `8fe034ef-6db0-4ba1-b291-66954fea2e08` 같은 36글자(UUID). 코드에서 `memory_id`, `scene_id`는 전부 이 번호로 다른 표의 행을 가리키는 것.

실제 SQL: 표 만들기

supabase/migrations/20250107000000_create_notes_table.sql : 4~14

```sql
CREATE TABLE IF NOT EXISTS notes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  memory_id UUID REFERENCES memories(id) ON DELETE CASCADE,
  sender_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  recipient_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
  message TEXT NOT NULL,
  note_type TEXT DEFAULT 'player_to_author',
  is_read BOOLEAN DEFAULT false,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT now(),
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT now()
);
```

읽는 법

- SQL 은 영어 문장에 가깝습니다. `CREATE TABLE IF NOT EXISTS notes ( … )` = "notes 표를 만들어라, 이미 있으면 말고". 괄호 안이 열 목록, 한 줄에 열 하나: **이름 · 종류 · 규칙**.
- 종류: `UUID`(주민번호), `TEXT`(글), `BOOLEAN`(참/거짓), `TIMESTAMP WITH TIME ZONE`(시각), `INT`(정수), `JSONB`(묶음을 통째로).

- `PRIMARY KEY` = 이 열이 주민번호. `DEFAULT …` = 안 주면 이 값 (`gen_random_uuid()` 새 번호 자동, `now()` 지금 시각, `false`).
- `REFERENCES memories(id) ON DELETE CASCADE` = "이 칸은 memories 표의 id 를 가리킨다. 그 기억이 지워지면 이 쪽지도 같이 지워라". 표 사이의 끈. (2026-07-14 에 씬 삭제 때 이 끈이 회차까지 지우던 것을 끊는 마이그레이션이 있었습니다. 끈은 편리하지만 위험.)
- `NOT NULL` = 비워 둘 수 없음.

실제 SQL: 표 고치기와 권한

`supabase/migrations/20260716000000_terrain_layers_and_contact_cols.sql : 11~17, 24~31, 34~36`

```
1  create table if not exists public.terrain_layers (
2    memory_id    uuid primary key references public.memories(id) on delete
   cascade,
3    height_delta jsonb       not null default '{}'::jsonb,
4    foot_map     jsonb       not null default '{}'::jsonb,
5    generation   int         not null default 0,
6    updated_at   timestamptz not null default now()
7  );
8
9  alter table public.terrain_layers enable row level security;
10
11 -- SELECT 공개(anon/authenticated). INSERT/UPDATE/DELETE 정책은 두지 않는다.
12 -- = 관객(anon)의 쓰기는 매칭 행 0개로 조용히 통과. Edge service_role 만 RLS
   우회.
13 create policy "terrain_layers public read"
14 on public.terrain_layers for select
15 to anon, authenticated
16 using (true);
17
18 alter table public.trajectory_bridges
19   add column if not exists contact_utterance text,
20   add column if not exists contact_turn      int;
```

읽는 법

- 대소문자는 상관없습니다(위 파일은 소문자, 앞 파일은 대문자). `--` 뒤는 메모 (자바스크립트의 `//`).
- 9번째 줄 **RLS**(행 단위 권한) 켜기. 켜면 **정책이 허락한 것만** 됩니다. 13~16번째 줄 정책: "읽기(select)는 익명(anon)이든 로그인(authenticated)이든 누구나(using true)". **쓰기 정책이 없으니 관객은 못 씁니다.** 쓰려면 서버 함수

(`supabase/functions/record-erosion`)가 관리자 열쇠(service_role)로 대신 씁니다. 메모(11~12번째 줄)가 그 설계를 적어 뒀습니다.

- 14장의 **403 이 태어나는 자리**가 바로 이 정책입니다. 정책이 "아니오" 하면 403. notes 표의 정책은 `auth.uid() = recipient_id`("지금 로그인한 사람이 받는 사람일 때만 읽기") 같은 모양.
- 18~20번째 줄 `alter table … add column if not exists` = "표에 열을 더해라, 이미 있으면 말고". 표 모양이 바뀌었다는 말은 **반드시** 이런 파일이 `migrations` 폴더에 새로 생겨야 합니다(1장 시험 3번). 파일 이름 앞 숫자가 날짜시각이고 그 순서로 적용됩니다.

코드에서 표를 읽고 쓰는 한 문장

`js/shared/api.js : 36~40, 59~63`

```
1  const { data, error } = await supabaseClient
2    .from('scenes')
3    .select('*, choices(*)')
4    .eq('memory_id', memoryId)
5    .order('scene_order');
6
7  const { data, error } = await supabaseClient
8    .from('plays')
9    .insert(playData)
10   .select()
11   .single();
```

조각	뜻
`.from('scenes')`	scenes 표에서
`.select('*, choices(*)')`	모든 열을, 그리고 끈으로 이어진 choices 표의 행들도 같이
`.eq('memory_id', memoryId)`	memory_id 가 이 값인 행만 (eq = equal). 비슷한 것: `.neq` 아닌, `.gt` 보다 큰, `.in` 목록 안
`.order('scene_order')`	scene_order 순으로 정렬
`.insert(playData)`	이 묶음을 새 행으로 넣어라
`.select().single()`	넣은 행을 돌려받되, 하나만
`.update({…}).eq(…)`	조건에 맞는 행의 칸을 고쳐라 (이 파일 다른 곳)

답은 항상 `{ data, error }`(10장). `error`가 차 있으면 콘솔 빨간 줄(14장), 그 안의 `code: '42501'`은 "권한 거부"입니다.

jsonb: 열 안의 묶음

`meta`, `emotion_dist`, `height_delta`처럼 종류가 `jsonb`인 열은 안에 묶음을 통째로 담습니다. 코드에서는 `scene.meta.stage_position`처럼 점 두 번. 장점은 열을 안 늘리고 칸을 더할 수 있다는 것. 단점은 **넣기만 하고 아무도 안 읽는 칸**이 생겨도 표에는 안 보인다는 것.

> **실제로 있었던 일**
>
> CLAUDE.md 4절, 죽은 레버 `pin_override`: 편집기에서 핀을 끌면 `meta.pin_override`에 저장은 됩니다. 그런데 상영 쪽에 그 칸을 **읽는 코드가 없습니다.** 진짜 핀 위치는 `meta.stage_position`. "데이터베이스에 저장된다"와 "쓰인다"는 다른 말입니다. 검사법: 칸 이름(`pin_override`)으로 grep 해서 **읽는 자리**(`meta.pin_override`가 `=` 오른쪽에 있는 줄)가 있는지 셉니다. 저장하는 자리(`=` 왼쪽)만 있으면 죽은 레버.

서버 함수(Edge Function)의 머리

supabase/functions/record-erosion/index.ts : 1~6, 33~41

```ts
// record-erosion — 이본 지층 W2-2 (2026-07-16): 봉인 시 변형층 저장 + 스냅샷 로깅.
//
// 배경: 회차 봉인 시 그 관객 몫의 침식(누적 변형층)을 굽고 저장한다.
//   memories/terrain_layers RLS = 익명 관객 직접 쓰기 불가 → service_role Edge 경유가 유일한 길

const MAX_BODY_BYTES = 512 * 1024;    // 스냅샷 포함분
const HEIGHT_SCALE = 20;              // |height_delta 읽| ≤ 이 값

interface RequestBody {
  memoryId: string;
  height_delta: unknown;
  generation: number;
}
```

파일 끝이 `.ts`(TypeScript) = 자바스크립트에 **종류 표시**를 더한 것. `memoryId: string`은 "memoryId 는 글자여야 한다". 읽는 법은 자바스크립트와 같고, 쌍점 뒤

종류만 덤으로 읽으면 됩니다. 머리 메모가 **왜 이 함수가 존재하는지**(관객은 못 쓰니 서버가 대신)를 적어 둔 좋은 예. 이 프로젝트 서버 함수 19개는 거의 다 이 이유로 존재합니다.

읽는 요령

표 모양이 바뀌었다 → `migrations`에 새 파일이 있나. 403 → 그 표의 `create policy`를 읽어 누가 무엇을 할 수 있는지 본다. 저장했다 → 그 칸을 **읽는** 코드가 있는지 grep.

✎ 미니 시험 16

1. 위 마이그레이션 18~20번째 줄은 어느 표에 어떤 열을 더합니까? 이미 있으면 어떻게 됩니까?
2. `supabaseClient.from('plays').insert(playData).select().single()` — 우리말 한 문장으로.
3. notes 표 정책이 `ON notes FOR SELECT USING (auth.uid() = recipient_id)` 입니다. 내가 남에게 보낸 쪽지를 내가 다시 읽을 수 있습니까?

정답은 부록 E.

내 손으로 정리 · "저장된다"와 "쓰인다"의 차이

3D의 겉모양: Three.js

> **목표** 지형 · 문 · 유령을 그리는 코드의 **겉모양**만 읽는다. 무대 · 카메라 · 그리는 장치 세 가지, 물체 = 모양 + 재질, 그리고 매 프레임 도는 루프. 삼각형을 어떻게 그리는지는 안 배운다.

무대 세 가지

`js/shared/tem_af_strata_terrain.js : 1639, 1644, 1647`

```
scene3 = new THREE.Scene();
camera = new THREE.PerspectiveCamera(50, vp0.w / vp0.h, 0.1, 500);
renderer = new THREE.WebGLRenderer({ canvas: canvas, antialias: true });
```

것	비유	읽는 법
`THREE.Scene`	무대	물체를 올려놓는 곳. `scene.add(물체)`로 올림.
`THREE.PerspectiveCamera`	카메라	재료 넷: 시야각 50도 · 화면 비율 · 가장 가까운 거리 0.1 · 가장 먼 거리 500. 이 밖은 안 보임 (지형 끝이 사라지면 여기 숫자).
`THREE.WebGLRenderer`	그리는 장치	무대를 카메라로 찍어 `canvas`(html 의 그림판 상자)에 냄. `antialias: true` = 테두리 부드럽게.

`new THREE.X(…)` = "X 를 하나 만든다". `THREE.` 뒤 이름만 읽으면 무엇을 만드는지 압니다. 이 프로젝트는 Three.js **r128**(2021년판)을 씁니다. 최신 예제 문법이 안 맞는 이유이고, 9월 23일 실험 기록에 "스플랫은 r128 위험"이라고 적힌 이유입니다.

물체 = 모양 + 재질

`play-test.html : 7422~7450 (출구 문, 일부)`

```
var doorGrp = new THREE.Group();
doorGrp.userData._exitDoor = true;

// Door frame (two pillars + lintel)
```

```
var frameMat = new THREE.MeshStandardMaterial({ color: 0x8a7a65, roughness: 0.9, metalness: 0.05 });
var pillarL = new THREE.Mesh(new THREE.BoxGeometry(0.15, 3.2, 0.15), frameMat);
pillarL.position.set(-0.7, 1.6, 0);
doorGrp.add(pillarL);
var pillarR = new THREE.Mesh(new THREE.BoxGeometry(0.15, 3.2, 0.15), frameMat);
pillarR.position.set(0.7, 1.6, 0);
doorGrp.add(pillarR);
var lintel = new THREE.Mesh(new THREE.BoxGeometry(1.55, 0.15, 0.15), frameMat);
lintel.position.set(0, 3.25, 0);
doorGrp.add(lintel);

// Door panel (slightly inset, darker)
var panelMat = new THREE.MeshStandardMaterial({ color: 0x1a1518, roughness: 0.95, transparent: true, opacity: 0.85 });
var panel = new THREE.Mesh(new THREE.PlaneGeometry(1.3, 3.0), panelMat);

var doorLight = new THREE.PointLight(0xc4a882, 0.3, 6, 2);
doorLight.position.set(0, 3.5, 0.3);
doorGrp.add(doorLight);
```

읽는 법

- `THREE.Group`(1번째 줄) = 물체 여러 개를 담는 **묶음**. 문 전체를 한 덩어리로 옮기려고. `userData._exitDoor = true`는 물체에 쪽지(html 의 `data-`, 13장의 `dataset`과 같은 발상). 나중에 "출구 문이냐?"를 이 쪽지로 묻습니다.
- `THREE.MeshStandardMaterial({ color, roughness, metalness })`(5번째 줄) = **재질**. 색 · 거칠기 · 금속성. `0x8a7a65`는 색을 16진수로 적은 것 = css 의 `#8a7a65`와 같은 색(2장). 흙빛 나무색.
- `THREE.Mesh(모양, 재질)`(6번째 줄) = **물체**. 모양은 `BoxGeometry(가로, 높이, 깊이)`: 가로 0.15 · 높이 3.2 · 깊이 0.15 = 가늘고 긴 기둥. `PlaneGeometry(가로, 높이)`는 판.
- `.position.set(x, y, z)`(7번째 줄) = 어디에 놓나. **y 가 위**. 왼쪽 기둥 x=-0.7, 오른쪽 기둥 x=0.7, 높이 1.6 은 기둥 가운데(3.2 의 절반). 상인방(lintel)은 y=3.25 로 위에 얹힘.
- `.add(물체)` = 묶음이나 무대에 올림.
- `THREE.PointLight(색, 세기, 거리, 감쇠)`(20번째 줄) = 전구. 색 `0xc4a882`... 2장의 `--accent-memory`와 같은 금색입니다. 문 위 3.5 에 떠서 6 거리까지

은은하게.

이 스무 줄이 상영에서 보이는 그 문입니다. 기둥 둘, 상인방 하나, 어두운 판 하나, 금색 전구 하나. **3D 코드는 "무엇을 · 어떤 재질로 · 어디에" 세 가지의 반복**입니다.

자주 보는 이름표

이름	뜻
`BoxGeometry` · `PlaneGeometry` · `SphereGeometry` · `BufferGeometry`	상자 · 판 · 공 · 직접 짠 모양(지형이 이것)
`MeshStandardMaterial` · `MeshBasicMaterial` · `ShaderMaterial`	빛 받는 재질 · 빛 무시 재질 · 직접 짠 재질(안개 · 하늘 얼룩)
`AmbientLight` · `DirectionalLight` · `PointLight`	사방 빛 · 햇빛(방향만) · 전구(위치 있음)
`Fog` · `FogExp2`	안개(거리 따라 흐려짐)
`Vector3(x, y, z)`	좌표 하나. 위치 · 방향 · 속도 다 이걸로
`GLTFLoader` · `FBXLoader`	바깥에서 만든 모형(.glb · .fbx) 불러오기. 사물 앵커 · 유령 몸
`OrbitControls`	마우스로 카메라 돌리기(편집기 · 지형 조감)
`.traverse(o => …)`	무대 위 모든 물체를 하나씩 방문(7장의 반복). 11장 시험의 검사 코드가 씬 핀을 이걸로 찾음

매 프레임 도는 루프

play-test.html : 4893~4916 (지형 띠 애니메이션, 일부) · js/ui/strataView.js : 717

```
function tick() {
  if (game.terrainTintFrom && game.terrainTintTo && game.terrainTintLerp < 1) {
    game.terrainTintLerp = Math.min(1, game.terrainTintLerp + 0.032);
    tint = lerpRgb(game.terrainTintFrom, game.terrainTintTo, easeOutCubic(game.terrainTintLerp));
  }
  ...
```

```
7   renderTerrainBar(game.currentSceneIndex, tint, { rippleAmp: ripplePx,
  rippleTime: game.terrainRipplePhase });
8   const tintAnimating = game.terrainTintFrom && game.terrainTintTo &&
  game.terrainTintLerp < 1;
9   if (tintAnimating || game.terrainRipple > 0.004) {
10    game.terrainAnimRaf = requestAnimationFrame(tick);
11  } else {
12    game.terrainAnimRaf = null;
13  }
14 }
15 game.terrainAnimRaf = requestAnimationFrame(tick);
```

읽는 법

- 애니메이션은 전부 이 모양입니다: **함수 하나(tick) 안에서 → 값을 조금 바꾸고 (3번째 줄, 0.032 씩) → 그리고(7번째 줄) → 아직 덜 됐으면 다음 프레임에 자기를 다시 부른다(9~10번째 줄).** 15번째 줄이 첫 시동.
- `requestAnimationFrame(tick)`(11장) = "다음 그리기 직전에 tick 을 한 번". tick 안에서 또 부르니 초당 60번 돕니다. 다 되면(else) 안 부르니 멈춤. 12번째 줄 `null`은 "지금 도는 애니메이션 없음" 표시.
- `lerp`(8장의 Lerp)는 "두 값 사이를 비율만큼 섞기", `easeOutCubic`은 "처음엔 빠르고 끝은 천천히". 연출 코드에 늘 나오는 두 단어. 속은 몰라도 됨.
- 3D 무대라면 7번째 줄 자리에 `renderer.render(scene, camera)`("무대를 카메라로 찍어 내라")가 있습니다. strataView.js 717번째 줄의 `animId = requestAnimationFrame(animateLoop)`가 지형 뷰의 같은 자리.

함정

이 프로젝트 기록(디버깅 지름길 메모): 카메라를 콘솔에서 옮겨도 `_fpTick`이 매 프레임 되돌립니다. 루프가 도는 동안은 루프 밖에서 바꾼 값이 다음 프레임에 덮입니다. "값을 바꿨는데 안 바뀌어요"의 3D 판 원인. 루프 안을 봐야 합니다.

읽는 요령

3D 코드는 `new THREE.이름`의 이름만 읽는다. 물체는 "모양 + 재질 + 위치". 애니메이션은 "값 조금 바꿈 → 그림 → 자기 다시 부름". `0x` 색은 css `#` 색과 같다.

✎ 미니 시험 17

1. `var pillarR = new THREE.Mesh(new THREE.BoxGeometry(0.15, 3.2, 0.15), frameMat); pillarR.position.set(0.7, 1.6, 0);` — 무엇을 어떤 재질로 어디에? 왼쪽 기둥과 다른 점 하나.
2. `new THREE.PointLight(0xc4a882, 0.3, 6, 2)` — 이 색은 CSS 의 어느 색 이름표와 같습니까? (2장 `:root` 참고)
3. 어떤 화면에서 "매 프레임 도는 함수"를 찾고 싶습니다. `Ctrl`+`F`에 무슨 글자를 넣겠습니까?
4. `doorGrp.userData._exitDoor = true` — 나중에 무대 위 물체를 하나씩 돌며(`traverse`) 출구 문만 찾고 싶다면 무엇을 확인하면 됩니까?

정답은 부록 E.

내 손으로 정리 · 물체 하나를 그리는 세 줄

큰 파일 지도 + "됐어요" 검사 순서

> **목표** 8,400줄짜리 파일도 처음부터 안 읽는다. 세 가지 기술로 지도를 그리고, 이 책 전체를 **일곱 단계 검사 순서** 하나로 접는다. 졸업 장. (7월 10교시 확장)

큰 파일 읽는 세 가지 기술

1. **이름으로 점프.** `Ctrl`+`F`에 AI 가 말한 함수 이름을 넣고 `function 이름` 또는 `이름(` 를 찾습니다. 함수 이름이 이정표.
2. **계단 훑기.** 6장. 왼쪽 끝만 보며 0계단 줄을 셉니다. VS Code 에서 `Ctrl`+`K` 다음 `Ctrl`+`0` 을 누르면 모든 덩어리가 접혀 목차만 남습니다(`Ctrl`+`K` `Ctrl`+`J` 로 다시 펼침).
3. **`function` 세기.** `Ctrl`+`F`에 "function " 을 넣으면 오른쪽에 "1/312" 처럼 개수가 뜹니다. 그 파일이 가진 기능 수. `Ctrl`+`Shift`+`O`는 함수 이름 목록을 띄웁니다(html 안 script 에서는 일부만).

실제 지도: play-test.html

줄 근처	무엇	찾는 글자
14~102	부품 목록 (script 태그 약 40개, 캐시버스터 포함)	`<script src=`
~100~530	옷 (style)	`<style>`
531	일반 script 시작 (마당)	`<script>`
703	module script 시작 (두뇌, 방). import 목록	`<script type="module">`
730	데이터베이스 연결을 마당에 꽂음	`window.getSupabaseClient`
3533	씬 진입 함수를 마당에 꽂음 (디버깅 지름길)	`_temEnterSceneMode`
4893	지형 띠 애니메이션 루프	`terrainAnimRaf`
5209	서가 그리기	`_renderShelf`

7420	출구 문 만들기	`_exitDoor`
6598	지형 뷰 부품 (맨 아래)	`strataView.js`

줄 번호는 2026-09-25 기준. 밀리면 "찾는 글자"로 점프.

grep: 폴더 전체에서 찾기

`Ctrl`+`Shift`+`F`. 파일 하나가 아니라 저장소 전체에서 글자를 찾습니다. 이 책에서 grep 이 필요한 자리 셋:

- **진입점 세 곳**(12장): 기능 이름이나 파일 이름 → 세 문 중 몇 곳에 있나.
- **캐시버스터**(12장): 파일 이름 → 각 줄의 `?v=`가 같고 새로운가.
- **죽은 레버**(16장): 열 이름 → 읽는 자리(`=` 오른쪽)가 있나.

검색 결과 창에서 파일별로 몇 개인지 숫자가 나옵니다. **세는 것**이 전부입니다.

"됐어요" 검사 순서

AI 가 작업을 마쳤다고 하면 **이 순서대로** 묻습니다. 각 단계가 이 책 어느 장인지 적어 뒀습니다. 문구는 부록 D 에 그대로 있습니다.

#	묻는 것	눈으로 확인하는 것	장
1	어느 파일, 어느 함수?	폴더로 층을 짐작. 함수 이름으로 점프해 `{ }` 경계 확인	1 · 5
2	비교표 그대로 보여줘	`@@` 개수 = 말한 자리 수. 각 덩어리가 어느 함수 안인지	15
3	초록 줄이 새로 하는 일	if · for · 이름으로 내 말로 한 줄. 못 하면 한 줄씩 풀어 달라고	4 · 7 · 8
4	진입점 세 곳	grep → 세 문 중 몇 곳. 빠진 문이 의도인지	12
5	캐시버스터	고친 파일 이름 grep → `?v=` 오늘 날짜 · 세 문 동일	12
6	콘솔 · 네트워크 빨강 0	F12. 빨간 줄 없음. 있으면 종류 · 내용 · 위치 읽기	14

7	실제 순서로 한 번	콘솔에 값을 심어 놓고 보는 게 아니라, 관객이 밟는 순서(서가 → 진입 → 걷기 → 대화 → 봉인)로 끝까지. 각 단계 캡처	13 · 14

실제로 있었던 일

7단계가 있는 이유. 2026-07-17 기록: "console 세팅 · 심어둔 전제는 그 전제 깨지는 실전 타이밍 못 봄." 접촉 기록 403 은 콘솔에서 값을 심어 시험할 땐 안 났고, 실제 순서로 밟을 때만 났습니다(접촉이 회차 저장보다 먼저 일어나는 타이밍). 자동으로 밟아 주는 도구가 이미 있습니다: `npm run qa`(`tools/qa.cjs`, 스냅샷 찍고 → 풀 플레이 → 되돌림)와 `test/e2e/playthrough.mjs`. AI 에게 "qa 돌리고 캡처 붙여"라고 시킬 수 있습니다.

덧붙여: 죽은 코드를 만났을 때

이 저장소는 안 쓰는 레버를 **일부러 남겨 둡니다**(라이브 모드, 옛 archive 플레이, pin_override …). 읽다가 "이거 아무 데서도 안 부르는데?"를 발견하면 결함이라고 단정하지 말고 **의도를 먼저 묻습니다.** 2026-08-02 에 죽은 재진입 스킵을 "고쳤다가" 되돌린 사고가 있습니다. 읽는 눈이 생기면 이런 걸 발견하게 되고, 발견은 좋은 일이지만 **발견 → 질문 → 결정** 순서를 지킵니다.

졸업

여기까지 오면 당신은 **읽고 검사하는 사람**입니다. 코드를 쓰지는 못해도(계속 AI 가 씁니다), "됐어요"를 들었을 때 파일을 열어 `@@`를 세고, grep 으로 세 문을 세고, F12 의 빨강을 읽고, 실제 순서로 밟게 시킬 수 있습니다. 처음 목표가 정확히 그거였습니다.

남은 건 반복입니다. 다음 작업부터 검사 순서 일곱 단계를 한 번씩 밟으세요. 4~6주면 비교표에서 "이상한데?"가 저절로 걸립니다. 그때 이 책의 장 번호가 손에 익어 있을 겁니다.

읽는 요령

큰 파일 = 이름 점프 · 계단 · function 세기. 됐어요 = 일곱 단계. 전부 **세는 일**이다. 계산이 아니라.

✎ 미니 시험 18 (졸업)

AI 가 이렇게만 말했습니다. "오염 계산 로직 좀 고쳤어요. play-test 에서 잘 돼요."

1. 검사 순서 일곱 단계로 되물을 질문을 **다섯 개** 만드세요. 각 질문 옆에 이 책 장 번호를 적으세요.
2. "play-test 에서 잘 돼요"라는 말에서 빠진 문이 있습니까? 있다면 무엇을 시킵니까?
3. 비교표를 받았더니 `@@`가 다섯 개이고 그중 둘이 `js/ui/` 폴더 파일입니다. "오염 계산만 고쳤다"는 말과 맞습니까? 무엇을 묻습니까?

정답 예시는 부록 E. 정답이 하나가 아닙니다.

내 손으로 정리 · 다음 작업에서 밟을 일곱 단계를 내 순서로

엔진이 내놓는 네 가지

> **목표** 엔진이 **무슨 질문**을 받고 **무슨 답**을 돌려주는지. 답은 네 개뿐이고, 그 네 개가 어디로 흘러가는지까지만 본다. 계산은 다음 장부터.

여기서부터는 속을 봅니다

앞의 18장에서 제가 계속 "속은 몰라도 된다"고 했죠. 경계만 보라고. 그건 지금도 맞아요. AI가 "고쳤어요" 할 때 진짜인지 확인하는 데는 경계만 보면 됩니다.

그런데 10월에 질문이 바뀌었어요. "내 엔진이 어떤 원리로 도는지 나도 모른다. 인정받으려면 내가 설명할 수 있어야 한다." 이건 경계를 보는 눈으로는 답이 안 나와요. 속을 봐야 해요. 그래서 이 부(部)를 붙였습니다.

미리 안심시켜 드릴게요. 속이라고 해 봐야 수학은 세 개예요. **평균**, **코사인**(두 화살표 사이 각도), **지수이동평균**(새 값 10%, 옛 값 90%). 미적분 없어요. 행렬 없어요. 이 셋을 모르는 사람은 없어요. 다만 코드 안에서 그 셋이 어떤 옷을 입고 있는지를 모를 뿐이에요. 그 옷을 벗기는 게 이 부의 일입니다.

엔진은 질문 하나에 답합니다

별이엔진이 받는 질문은 딱 하나예요. 설계 문서 첫 줄에 그대로 적혀 있어요.

> **"A가 기억을 말했다. B가 그 기억을 읽고 반응했다. A와 B의 경험이 얼마나 비슷한가?"**

답은 0에서 1 사이 숫자 하나. 1이면 똑같이 겪은 것, 0이면 전혀 다르게 겪은 것. 이 숫자 이름이 **정렬도**(`alignment`)예요. 엔진이 하는 일은 이 숫자를 내는 것뿐이고, "그러니까 당신은 이런 사람이다"라든가 "다음엔 이렇게 하라" 같은 말은 절대 안 합니다. 설계 원칙에 "엔진은 판단하지 않는다, 관찰하고 보고할 뿐"이라고 박아 뒀어요. 7월 16일에 "높은 정렬 = 승리"라는 말을 걷어내고 "접촉"이라는 말로 바꾼 것도 같은 맥락이에요. 숫자가 높은 건 잘한 게 아니라 일어난 거예요.

답은 네 개가 한 묶음으로 나옵니다

엔진 파일 맨 아래쪽, 함수가 답을 돌려주는 자리를 먼저 보세요. 10장에서 배운 "묶음 하나를 답으로"의 모양이에요.

js/core/ByeoriEngine.js : 110~118

```
return {
  affective_position: affectivePosition,
  alignment_score: alignment,
  alignment_bucket: bucket,
  transition_pattern: transitionPattern,
  mismatch_type: mismatchType,
  current_scene_score: currentSceneScore,
  debug
};
```

노란 줄 넷이 "네 가지"예요. 하나씩 풀면 이래요.

이름	뭔가	생김새
`alignment_score` 정렬도	위의 질문에 대한 답. 얼마나 비슷한가.	0~1 숫자. 예: 0.52
`alignment_bucket` 등급	그 숫자를 네 칸으로 나눈 것. 연출이 "지금 어느 칸이냐"만 물을 때 씀.	`HIGH` · `MID` · `LOW` · `FIXATED`(고착)
`transition_pattern` 전이 패턴	"비슷한가"가 아니라 "어떻게 움직였나". 다음에 어느 장면이 열릴지를 이게 정해요.	`echo_follow` · `bridge` · `contradiction` · `displacement` · `avoidance` · `fixation` 중 하나
`mismatch_type` 어긋남 종류	안 비슷했다면 뭐가 안 비슷했나.	`emotion_mismatch`(감정) · `attribution_mismatch`(탓) · `target_displacement`(대상) · `void_mismatch`(회피) · 없으면 `null`

나머지 셋은 곁다리예요. `affective_position`은 관객 감정을 지도 위 점으로 바꾼 것(지형 핀 자리용), `current_scene_score`는 "이번 장면 하나만" 봤을 때 점수, `debug`는 계산 과정을 다 풀어 적은 쪽지. `debug` 안에 `formula: "L0.73 × S0.71 × V1.00 = 0.52"` 같은 글자가 들어 있어요. 콘솔(14장)에서 이 글자를 보면 엔진이

그 순간 뭘 곱했는지 그대로 보이는 거예요. 저는 이게 이 파일에서 제일 친절한 줄이라고 생각해요.

그 네 개가 어디로 가나

이걸 알아야 "측정이 어디 남느냐"는 질문에 답할 수 있어요. 흐름은 이렇습니다.

1. 관객이 장면 하나에서 감정을 입력해요. (장면마다 한 번.)
2. play-test 안의 **관문 함수**가 엔진을 불러요. 2,611번째 줄쯤에 있는 `byeoriCalculateStep`. 모든 엔진 호출이 이 한 자리를 지나요. 12장에서 본 "한 곳만 지나게 하는" 설계예요.
3. 엔진이 네 가지를 돌려줘요.
4. play-test가 그중 둘을 `plays` 표(16장)에 한 행으로 넣어요.

play-test.html : 4708~4714 (plays 표에 넣는 행)

```
const row = {
  memory_id: game.memoryId,
  scene_id: scene.id,
  user_emotion: vec,
  user_reason: isVoid ? 'void' : (reasonLabel || ''),
  alignment: result.alignment_score,
  mismatch_type: isVoid ? 'void_mismatch' : (result.mismatch_type || null),
```

보이죠. 표에 남는 건 `alignment`(정렬도)와 `mismatch_type`(어긋남 종류) 둘이에요. 그리고 4번째 줄, 관객이 입력한 감정 `vec`가 **원시 그대로** 남아요. 등급과 전이 패턴은 그 순간 연출과 다음 장면 고르기에 쓰이고 표에는 안 남습니다.

함정

"표에 안 남는다"는 건 결함이 아니라 **선택**이에요. 정렬도와 원시 감정이 남아 있으면 등급과 패턴은 나중에 다시 계산할 수 있거든요. 다만 9월 26일 기록 공백 제안이 지적한 대로, "그때 엔진이 어떤 설정이었나"(문턱값, 손잡이 값)는 안 남아요. 그래서 설정을 바꾸면 과거 행을 같은 자로 못 재요. 이건 20장 뒤에서 다시 나옵니다.

한 가지만 미리

엔진은 **상태가 없어요**. 파일 21~23번째 줄에 `constructor() { // 상태 없음 }`이라고 적혀 있어요. 무슨 뜻이냐면, 엔진은 "지난번에 뭘 계산했는지" 기억하지 않아요. 매번 재료를 전부 받아서 처음부터 계산해요. 지금까지의 궤적, 지금까지의 점수, 이전 등급, 전부 부르는 쪽(play-test)이 들고 있다가 넘겨줘요. 그래서 같은 재료를 넣으면 언제 어디서 돌려도 같은 답이 나와요. 이게 "측정 장치"라고 부를 수 있는 이유예요. 측정 장치는 기분이 없어야 하니까요.

> **읽는 요령**
> 엔진 = 질문 하나(얼마나 비슷한가), 답 네 개(정렬도 · 등급 · 전이 패턴 · 어긋남 종류). 표에 남는 건 정렬도와 어긋남 종류, 그리고 원시 감정. 엔진은 기억이 없다.

✎ 미니 시험 19

1. 콘솔에 `formula: "L0.40 × S1.00 × V0.70 = 0.280"`이 찍혔습니다. 이 관객은 지금 몇 번째 장면쯤에 있을 가능성이 높습니까? (힌트: S가 1.00인 이유.) 그리고 V가 0.70이라는 건 무슨 일이 있었다는 뜻입니까?
2. `plays` 표에서 어떤 회차의 행들을 꺼내 봤더니 `alignment`는 있는데 등급(`HIGH` / `MID` …)은 없습니다. 고장입니까?
3. AI가 "엔진이 이전 회차의 결과를 기억하고 있어서 이번 결과가 이상해요"라고 말했습니다. 이 말은 성립합니까? 파일 어느 줄을 보여 주며 되묻겠습니까?

정답은 부록 E.

내 손으로 정리 · 엔진이 받는 질문 한 줄, 돌려주는 네 가지 이름

코사인: 두 화살표 사이 각도

> **목표** 감정 하나가 코드 안에서 어떤 모양인지(17칸 묶음), "비슷하다"를 숫자로 바꾸는 코사인이 뭔지, 그리고 손으로 한 번 계산해 본다. 이 장을 넘기면 엔진의 수학 절반이 끝난다.

감정 하나 = 칸 17개짜리 묶음

엔진이 "감정"이라고 부르는 건 느낌이 아니라 **칸 17개에 숫자가 든 묶음**이에요. 10장에서 배운 그 객체요. 칸 이름은 공용 수학 파일에 목록으로 적혀 있어요.

js/shared/math.js : 19~25

```
export const DEFAULT_EMOTION_ANCHORS = [
 // negative/pain
  'fear', 'sadness', 'anger', 'guilt', 'shame',
  'isolation', 'numbness', 'moral_pain', 'helplessness', 'despair',
 // positive/recovery
  'joy', 'hope', 'relief', 'gratitude', 'love', 'peace', 'comfort'
];
```

고통 쪽 열 개(공포 · 슬픔 · 분노 · 죄책 · 수치 · 고립 · 무감 · 도덕적 고통 · 무력 · 절망), 회복 쪽 일곱 개(기쁨 · 희망 · 안도 · 감사 · 사랑 · 평화 · 위안). 그래서 관객이 어떤 장면에서 "무섭고 조금 슬펐다"고 하면 코드 안에선 이렇게 생긴 묶음이 돼요.

(예시 · 실제 저장되는 모양)

```
{ fear: 0.8, sadness: 0.3, joy: 0 }
```

안 쓴 칸은 그냥 없거나 0이에요. 이 목록을 "17축"이라고 불러요. 축이라는 말이 어렵게 들리면 "칸"이라고 읽으세요. 똑같은 뜻이에요.

왜 "벡터"라고 부르나

문서에 자꾸 "감정 벡터"라는 말이 나오죠. 벡터는 그냥 **칸마다 숫자가 든 묶음을 화살표로 보는 것**이에요. 칸이 두 개면 종이 위에 그릴 수 있어요. 가로가 공포, 세로가 기쁨이라고 치면, `{ fear: 0.8, joy: 0.2 }`는 원점에서 오른쪽으로 0.8, 위로 0.2 간 자리를 가리키는 화살표예요. 칸이 17개면 종이에 못 그리지만 수학은

똑같이 돌아가요. 그래서 저는 머릿속으로는 항상 두 칸짜리로 생각해요. 그걸로 충분해요.

"비슷하다"를 숫자로: 코사인

화살표 두 개가 있으면 "비슷하다"를 두 가지로 잴 수 있어요. 하나는 끝점 사이 **거리**. 다른 하나는 두 화살표 사이 **각도**. 엔진은 각도를 써요. 각도를 0~1 숫자로 바꾼 게 코사인이에요.

- 두 화살표가 **같은 쪽**을 가리키면 코사인 = 1.
- **직각**이면 0.
- **정반대**면 -1. (엔진은 음수가 나오면 0으로 깎아요. 이유는 22장.)

왜 거리가 아니라 각도냐. 각도는 **세기를 무시**하거든요. `{ fear: 0.4, joy: 0.1 }`과 `{ fear: 0.8, joy: 0.2 }`는 길이는 두 배 차이지만 방향은 똑같아요. 코사인은 1. "덜 무서웠다"와 "많이 무서웠다"를 같은 결로 치는 거예요. 이건 설계 선택이에요. 결이 같으면 같은 경험으로 본다. 세기 차이는 안 본다. 좋다 나쁘다가 아니라, 이 엔진이 그렇게 정했다는 걸 아는 게 중요해요.

손으로 한 번

계산법은 세 줄이에요. 두 칸짜리로 해 볼게요.

1. **칸끼리 곱해서 전부 더한다.** (이걸 내적이라고 불러요. 이름은 몰라도 돼요.)
2. **각 화살표의 길이**를 구한다. 길이 = 칸마다 제곱해서 더한 뒤 제곱근.
3. 1번을 2번 둘의 곱으로 나눈다.

	A = { fear 0.8, joy 0.2 } **B = { fear 0.7, joy 0.3 }**	**A = { fear 0.9, joy 0.1 }** **C = { fear 0.1, joy 0.9 }**	**A = { fear 1, joy 0 }** **D = { fear 0, joy 1 }**
① 곱해서 더함	0.8×0.7 + 0.2×0.3 = 0.56 + 0.06 = **0.62**	0.9×0.1 + 0.1×0.9 = 0.09 + 0.09 = **0.18**	1×0 + 0×1 = **0**
② 길이	A: √(0.64+0.04) = 0.825 B: √(0.49+0.09) = 0.762	A: √(0.81+0.01) = 0.906 C: 똑같이 0.906	A: 1 D: 1

③ 나눔	0.62 ÷ (0.825×0.762) = **0.99**	0.18 ÷ (0.906×0.906) = **0.22**	0 ÷ 1 = **0**
읽으면	거의 같은 결	많이 다름	완전히 다른 결

이게 다예요. 17칸이면 ①에서 17개를 곱해 더하고 ②에서 17개를 제곱해 더할 뿐, 나머지는 똑같아요.

그 세 줄이 코드에선 이렇게 생겼어요

js/shared/math.js : 183~209 (cosineSimilarity)

```
1 export function cosineSimilarity(vec1, vec2, anchorEmotions = null) {
2   if (!vec1 || !vec2) return 0;
3
4   let keys;
5   if (anchorEmotions && Array.isArray(anchorEmotions) &&
  anchorEmotions.length > 0) {
6     keys = anchorEmotions.map(anchor => normalizeAnchor(String(anchor)));
7   } else {
8     keys = DEFAULT_EMOTION_ANCHORS;
9   }
10
11  let dotProduct = 0;
12  let mag1 = 0;
13  let mag2 = 0;
14
15  keys.forEach(key => {
16    const v1 = vec1[key] || 0;
17    const v2 = vec2[key] || 0;
18    dotProduct += v1 * v2;
19    mag1 += v1 * v1;
20    mag2 += v2 * v2;
21  });
22
23  mag1 = Math.sqrt(mag1);
24  mag2 = Math.sqrt(mag2);
25
26  if (mag1 === 0 || mag2 === 0) return 0;
27  return dotProduct / (mag1 * mag2);
28 }
```

읽는 법

- 4~9번째 줄: **어느 칸들로 잴지** 정해요. 셋째 재료(`anchorEmotions`)로 칸 목록이 들어오면 그걸 쓰고, 안 들어오면 기본 17칸. 이게 7월 27일에 정한 "기억별

좌표계"가 들어오는 자리예요. 기억마다 실제로 쓰는 감정 어휘만으로 재겠다는 결정. 그 결정이 코드 한 줄짜리 갈림길(4장)로 들어와 있는 거예요.

- 10~13번째 줄: 빈 통 세 개. 곱해서 더할 통, A 길이 통, B 길이 통.
- 15~21번째 줄: 칸 목록을 하나씩 돌며(7장) 세 통을 채워요. `vec1[key] || 0`은 8장의 "없으면 0" 모양. 그래서 `{ fear: 0.8 }`처럼 칸이 비어 있어도 안 터져요.
- 22~23번째 줄: 길이 = 제곱근. 위 표의 ②.
- 25번째 줄: 둘 중 하나라도 길이가 0이면(아무 감정도 없으면) 0을 답으로. 0으로 나누면 터지니까요.
- 26번째 줄: 표의 ③. 끝.

27줄이 전부 위의 세 줄짜리 계산이에요. 코드가 길어 보이는 건 "없을 때" 대비(2 · 25번째 줄)와 "어느 칸으로" 고르기(4~9번째 줄)가 붙어서예요. 계산 자체는 15~26번째 줄, 열두 줄.

함정

"비슷하다"는 말이 코드에선 **항상 코사인**이에요. 장면 하나의 점수도 코사인, 궤적 모양 비교도 코사인(다음 장), 고착 감지도 코사인(최근 셋이 서로 얼마나 같은 결이냐). 그러니까 어떤 문서가 "유사도"라고 쓰면 "코사인이겠구나, 세기는 무시하겠구나"라고 읽으면 거의 맞아요. 안 맞는 자리가 하나 있는데, 지형 배치용 VAD 거리(`calculateVADSimilarity`)예요. 그건 판단에 안 쓰고 그림 그릴 때만 써요. 7월에 "VAD 안 쓰기로"라고 한 건 정렬도 점수에 안 쓴다는 뜻이었어요.

읽는 요령

감정 = 17칸 묶음. 비슷함 = 코사인 = 각도. 세기는 무시, 결만 본다. 계산은 "곱해서 더하고, 길이로 나눈다" 세 줄.

✎ 미니 시험 20

1. A = { fear 0.6, sadness 0.8 }, B = { fear 0.8, sadness 0.6 }. 코사인을 손으로 구하세요. (길이가 둘 다 깔끔하게 떨어집니다.)
2. C = { fear 0, sadness 1 }, D = { fear 1, sadness 0 }. 코사인은? 이 둘은 "둘 다 고통"인데 왜 그 값이 나옵니까?
3. `cosineSimilarity(a, b, ['fear', 'joy'])`처럼 셋째 재료를 넣으면 무엇이 달라집니까? 관객 감정에 `sadness: 0.9`가 들어 있었다면 그 값은 어떻게 됩니까?

정답은 부록 E.

내 손으로 정리 · 코사인 세 줄을 내 말로

별이엔진 190줄, 한 덩어리씩

> **목표** `js/core/ByeoriEngine.js`의 중심 함수 `calculateStep`을 재료 받기부터 답 내보내기까지 아홉 덩어리로 끊어서 읽는다. 다 읽으면 "L × S × V"가 어디서 어떻게 나오는지 손가락으로 짚을 수 있다.

식부터 외워 두세요

파일 맨 위 주석에 식이 통째로 적혀 있어요. 이걸 먼저 외우고 코드를 읽으면, 코드는 이 식의 재료를 준비하는 과정일 뿐이라는 게 보여요.

`js/core/ByeoriEngine.js : 4~7`

```
// 공식: alignment = level × shape × void_mod
// level = 장면별 감정 유사도 평균
// shape = 감정 변화 궤적의 형태 유사도 (3장면 미만이면 1.0)
// void_mod = 회피 보정 (기본 1.0, 회피 시 0.7)
```

한국어로 다시 쓰면 이래요.

정렬도 = 수준 × 형태 × 회피

- **수준**(level): 장면마다 "작가와 얼마나 같은 결이었나"를 코사인으로 재고, 지금까지 본 장면 전부의 **평균**.
- **형태**(shape): 장면에서 다음 장면으로 감정이 **어느 쪽으로 움직였나**를 작가의 움직임과 비교한 코사인. 장면이 셋 미만이면 비교할 게 없어서 1.
- **회피**(void_mod): 관객은 감정을 안 내놨는데 작가는 내놨으면 0.7을 곱해서 깎음. 아니면 1(그대로).

셋을 곱해요. 더하는 게 아니라. 왜 곱이냐는 22장에서 말할게요. 지금은 "하나라도 0이면 전체가 0"이라는 성질만 기억하세요.

덩어리 ① 재료 받기 (40~51)

`js/core/ByeoriEngine.js : 40~51`

```
calculateStep(input, context = {}) {
```

```
  const {
    userVector, originalVector, anchorEmotions = null,
    userTrajectory = null, originalTrajectory = null,
    sceneScores = null, emotionKeys = null
  } = input;
  const { previousBucket = null, emotionHistory = null } = context;

  if (!userVector || !originalVector) {
    console.warn('[ByeoriEngine V4] userVector 또는 originalVector 없음');
    return this._createEmptyResult();
  }
```

읽는 법

- 재료는 두 묶음이에요. `input`(이번 장면 것 + 지금까지 것)과 `context`(이전 등급, 감정 이력). 2~6번째 줄은 10장의 "묶음 풀기". 묶음에서 이름표대로 꺼내 변수로 만들어요. `= null`은 "안 들어왔으면 없음으로".
- 꺼낸 것 중 핵심 둘: `userVector`(관객이 이번 장면에서 느낀 것), `originalVector`(작가가 이 장면에 심어 둔 원본 감정). 나머지는 "지금까지"예요. `userTrajectory`는 관객이 지금까지 느낀 것들의 목록, `originalTrajectory`는 그 장면들의 원본 목록, `sceneScores`는 지금까지 장면마다 매긴 점수 목록.
- 9~11번째 줄: 핵심 둘 중 하나라도 없으면 콘솔에 경고 쓰고 빈 답(전부 0, 등급 LOW)을 돌려주고 끝. 4장의 "문지기" 모양.

19장에서 "엔진은 기억이 없다"고 했죠. 여기서 보이죠. 지금까지의 궤적과 점수를 전부 재료로 받아요. 엔진이 들고 있지 않아요.

덩어리 ② 자 고르기 (53~58)

js/core/ByeoriEngine.js : 53~58

```
// 판단 좌표계 (260727 기억별 좌표계 — docs/기억별_좌표계_결정-260727.md):
// 씬 anchor(가장 구체) > emotionKeys(기억의 저작 어휘) > 판단 기본 17축.
// level·shape·mismatch 가 전부 같은 자로 재야 V4 §9.1 (곱셈 결합의 동일 공간) 유지.
const frameKeys = (anchorEmotions && anchorEmotions.length)
  ? anchorEmotions
  : ((emotionKeys && emotionKeys.length) ? emotionKeys : null);
```

20장 끝에서 "어느 칸들로 잴지"가 셋째 재료로 들어온다고 했죠. 여기서 그 칸 목록을 정해요. 우선순위가 셋이에요. 장면에 앵커 감정이 박혀 있으면 그것, 아니면

이 기억이 실제로 쓰는 감정 어휘(`emotionKeys`), 둘 다 없으면 `null`, 즉 기본 17칸. 4장의 "물음표 갈림길" 두 개가 겹친 모양이에요. 주석 세 줄에 결정 날짜와 문서 이름까지 적혀 있어요. 이런 주석이 있는 자리는 **설계 결정이 코드로 내려온 자리**예요. 그냥 지나치지 말고 "아, 7월 27일 결정이 여기구나" 하고 표시해 두세요.

덩어리 ③ 이번 장면 점수, 그리고 수준 (60~66)

js/core/ByeoriEngine.js : 60~66

```
1 const currentSceneScore = calculateEmotionScore(
2   userVector, originalVector, frameKeys
3 );
4
5 const allSceneScores = sceneScores ? [...sceneScores, currentSceneScore] :
  [currentSceneScore];
6
7 const level = calculateLevelSimilarity(allSceneScores);
```

- 1번째 줄: 이번 장면 점수. `calculateEmotionScore`는 속을 열면 20장의 `cosineSimilarity`를 부르고 음수면 0으로 깎는 게 다예요. (임베딩이 둘 다 있으면 그걸 쓴다는 갈림길이 하나 있는데, 지금 상영 경로에선 임베딩을 안 넘겨서 항상 코사인 쪽으로 가요.)
- 5번째 줄: 지금까지 점수 목록 끝에 이번 점수를 붙여요. `[...sceneScores, x]`는 "목록을 복사해서 뒤에 x 하나 더"(10장). 목록이 없으면 이번 점수 하나짜리 목록.
- 7번째 줄: **수준 = 평균.** 함수 이름이 거창하지만 속은 이래요.

js/shared/math.js : 302~305

```
1 export function calculateLevelSimilarity(sceneScores) {
2   if (!sceneScores || sceneScores.length === 0) return 0;
3   return sceneScores.reduce((a, b) => a + b, 0) / sceneScores.length;
4 }
```

`reduce((a, b) => a + b, 0)`는 "0에서 시작해 전부 더해라"(7장의 반복 한 종류), 그걸 개수로 나누면 평균. 끝이에요. 장면 네 개 점수가 0.9 · 0.8 · 0.6 · 0.7이면 수준은 0.75.

덩어리 ④ 궤적, 그리고 형태 (68~75)

여기가 이 엔진의 **알맹이**예요. 다른 데는 다 평범한데 여기만 TEM 고유예요. 천천히 갈게요.

js/core/ByeoriEngine.js : 68~75

```
const userTraj = userTrajectory
  ? [...userTrajectory, userVector.base || userVector]
  : [userVector.base || userVector];
const origTraj = originalTrajectory
  ? [...originalTrajectory, originalVector.base || originalVector]
  : [originalVector.base || originalVector];

const shapeResult = calculateShapeSimilarity(userTraj, origTraj,
frameKeys);
```

1~6번째 줄은 덩어리 ③의 5번째 줄과 같은 모양이에요. 지금까지 궤적 끝에 이번 감정을 붙여서 두 목록을 만들어요. 관객 목록과 원본 목록. `userVector.base || userVector`는 "묶음 안에 `base` 칸이 있으면 그걸, 없으면 묶음 자체를"(8장). 이 파일은 감정이 `{ base: {...}, reason_analysis: {...} }`로 포장돼 들어올 때도 있고 맨 묶음으로 들어올 때도 있어서 둘 다 받아요.

중요한 건 **두 목록이 같은 순서**라는 거예요. 관객이 장면을 3번, 1번, 4번 순서로 봤으면 관객 목록도 그 순서, 원본 목록도 그 순서로 재배열된 원본이에요. 이 재배열은 엔진 밖(play-test)에서 해서 넘겨요. 문서 §2에 그림으로 있어요.

8번째 줄에서 형태를 계산해요. 속을 보죠.

js/shared/math.js : 283~297 (calculateShapeSimilarity)

```
export function calculateShapeSimilarity(userTrajectory,
originalTrajectory, emotionKeys = null) {
  // emotionKeys (260727 기억별 좌표계): level 과 같은 자로 재도록 호출자가 축
목록을
  // 넘길 수 있다. 생략 시 판단 기본 17축 — V4 §9.1 (level·shape 동일 좌표계)
유지.
  if (!userTrajectory || !originalTrajectory || userTrajectory.length < 3)
{
    return { raw: 1.0, clamped: 1.0, active: false };
  }
  const deltaUser = [];
  const deltaOrig = [];
  for (let i = 1; i < userTrajectory.length; i++) {
    deltaUser.push(subtractVectors(userTrajectory[i], userTrajectory[i -
1]));
```

```
    deltaOrig.push(subtractVectors(originalTrajectory[i],
originalTrajectory[i - 1]));
  }
  const raw = cosineFlat(flattenDeltas(deltaUser, emotionKeys),
flattenDeltas(deltaOrig, emotionKeys));
  return { raw, clamped: Math.max(0, raw), active: true };
}
```

읽는 법

- 4~6번째 줄: 장면이 **셋 미만이면 형태 = 1**, 그리고 `active: false`("형태는 아직 안 쟀음" 표시). 왜 셋이냐. 움직임을 하나 재려면 장면 둘이 필요하고, 움직임 둘을 비교하려면 장면 셋이 필요해요. 움직임이 하나뿐이면 "방향이 비슷한가"를 물을 수 없어요. 그래서 1을 돌려주면 정렬도 = 수준 × 1 × 회피가 되어 초반엔 수준만 반영돼요. 이게 19장 시험 1번의 답이에요.
- 9~12번째 줄: **움직임(delta) 만들기.** 둘째 장면부터 "이번 감정 빼기 직전 감정"을 구해 목록에 넣어요. `subtractVectors`는 칸마다 빼는 것. 관객 것 한 목록, 원본 것 한 목록.
- 13번째 줄: 움직임 목록을 **한 줄로 펴서**(`flattenDeltas`) 코사인(`cosineFlat`). 펴는 건 그냥 "움직임1의 17칸, 움직임2의 17칸, ..."을 쭉 이어 붙이는 거예요. 그러면 긴 화살표 하나가 되고, 관객 것과 원본 것 두 화살표의 각도를 재요. 20장과 완전히 같은 계산.
- 14번째 줄: 날것(`raw`, -1~1)과 깎은 것(`clamped`, 음수면 0) 둘 다 돌려줘요. 엔진은 깎은 걸 식에 넣고 날것은 `debug`에만 남겨요.

손으로 한 번: 장면 셋

두 칸짜리로 해 볼게요. 관객이 장면 셋을 봤고, 갈수록 무서워졌어요. 작가도 그랬어요.

	장면 1	장면 2	장면 3
관객	{ fear 0.2, joy 0.8 }	{ fear 0.6, joy 0.4 }	{ fear 0.9, joy 0.1 }
작가 원본	{ fear 0.3, joy 0.7 }	{ fear 0.7, joy 0.3 }	{ fear 0.8, joy 0.2 }
장면 점수(코사인)	0.99	0.98	0.99

수준 = (0.99 + 0.98 + 0.99) ÷ 3 = **0.99**. 장면마다 거의 같은 결이었어요.

이제 **움직임**. 둘째 빼기 첫째, 셋째 빼기 둘째.

	움직임 1 (장면2 − 장면1)	움직임 2 (장면3 − 장면2)	한 줄로 편 것
관객	{ fear +0.4, joy −0.4 }	{ fear +0.3, joy −0.3 }	[0.4, −0.4, 0.3, −0.3]
작가	{ fear +0.4, joy −0.4 }	{ fear +0.1, joy −0.1 }	[0.4, −0.4, 0.1, −0.1]

편 두 줄의 코사인: 곱해서 더하면 0.16 + 0.16 + 0.03 + 0.03 = 0.38. 길이는 √0.50 = 0.707과 √0.34 = 0.583. 나누면 0.38 ÷ 0.412 = **0.92**. 둘 다 공포 쪽으로 움직였고, 폭만 조금 달랐으니 높게 나와요.

정렬도 = 0.99 × 0.92 × 1 = **0.91**. 등급은 HIGH, 패턴은 echo_follow가 될 거예요 (덩어리 ⑦, ⑧에서 확인).

왜 "순서"가 아니라 "움직임"인가

자주 헷갈리는 데라서 짚고 갈게요. 관객이 장면을 어떤 순서로 보든, 원본 목록도 같은 순서로 재배열해서 비교해요. 그러니까 순서 자체는 점수를 떨어뜨리지 않아요. 떨어뜨리는 건 "그 순서로 봤을 때 감정이 **어느 쪽으로 움직였나**"가 작가와 다른 경우예요. 장면마다 감정은 비슷한데 움직인 방향이 반대면, 수준은 높고 형태는 0이 돼요. 24장 두 번째 문제가 정확히 이거예요.

덩어리 ⑤ 회피 (77~81)

js/core/ByeoriEngine.js : 77~81

```
1 const userReason = userVector.reason_analysis || {};
2 const origReason = originalVector.reason_analysis || {};
3 const userVoid = !!userReason.is_void;
4 const origVoid = !!origReason.is_void;
5 const voidMod = (userVoid && !origVoid) ? 0.7 : 1.0;
```

`!!x`는 "x를 참/거짓으로 딱 떨어지게"(부록 A). 5번째 줄이 식의 셋째 항이에요. **관객은 회피했고 작가는 안 했을 때만** 0.7. 관객도 작가도 회피했으면 1. 작가만 회피했어도 1. 숫자 0.7은 이론에서 온 게 아니라 정한 값이에요. 22장에서 다시.

덩어리 ⑥ 곱하기 (83)

js/core/ByeoriEngine.js : 83 · js/shared/math.js : 310~312

```
1 const alignment = calculateTrajectoryAlignment(level, shapeResult.clamped,
  voidMod);
2
```

```
3 export function calculateTrajectoryAlignment(level, shape, voidMod = 1.0) {
4   return Math.max(0, Math.min(1, level * shape * voidMod));
5 }
```

식 그대로예요. 셋을 곱하고, 0 아래면 0, 1 위면 1로 잘라요. 이 줄이 이 파일의 심장인데 한 줄이에요. 나머지 189줄은 이 한 줄의 재료를 만들고, 결과에 이름표를 붙이는 일이에요.

덩어리 ⑦ 등급 (85)

js/shared/math.js : 362~380 (getBucket)

```
1 export function getBucket(alignment, previousBucket = null, emotionHistory
  = null) {
2  // FIXATED 체크 (bucket 판정보다 우선)
3   if (emotionHistory && emotionHistory.length >= 3) {
4     const fixLevel = calculateFixationLevel(emotionHistory);
5     if (fixLevel >= 0.85) {
6       return 'FIXATED';
7     }
8   }
9
10  // 히스테리시스 적용
11  if (previousBucket === 'HIGH' && alignment >= 0.40) return 'HIGH';
12  if (previousBucket === 'LOW' && alignment <= 0.15) return 'LOW';
13
14  // 표준 판정
15  if (alignment >= 0.50) return 'HIGH';
16  if (alignment < 0.10) return 'LOW';
17  return 'MID';
18 }
```

읽는 법

- 3~8번째 줄: **고착 먼저.** 감정 이력이 셋 이상이면 최근 셋이 서로 얼마나 같은 결인지(세 쌍의 코사인 평균)를 재서 0.85 넘으면 무조건 FIXATED. 정렬도와 무관해요. "같은 감정에 갇혔다"는 정렬도가 높든 낮든 따로 보는 거예요. 설계 원칙 3번("고착은 횟수가 아니라 유사도로")이 이 줄이에요.
- 11~12번째 줄: **끈적임**(히스테리시스). 직전이 HIGH였으면 0.40까지는 HIGH로 봐줘요. 직전이 LOW였으면 0.15까지는 LOW. 등급이 경계에서 깜빡깜빡 바뀌는 걸 막으려고. 이것도 정한 값.
- 15~17번째 줄: 기본 경계. **0.50 이상 HIGH, 0.10 미만 LOW, 사이는 MID.**

함정

문서와 코드가 다릅니다. 시스템 매뉴얼 §8.5에는 "HIGH ≥ 0.7, MID 0.4~0.7, LOW < 0.4"라고 적혀 있어요. 코드는 0.50 / 0.10이에요. 매뉴얼은 4월 10일 것이고, 코드 주석에 "V4 곱셈 분포 기준 (파일럿 후 재조정)"이라고 적혀 있으니 파일럿 뒤에 코드만 고치고 매뉴얼은 안 고친 거예요. **둘이 다르면 코드가 진실**이에요. 돌아가는 건 코드니까요. 이 책 전체가 하는 말이 이거예요. 문서를 읽고 끝내지 말고 코드를 열어라. 지금 당신이 그걸 했어요.

덩어리 ⑧ 어긋남 종류, 전이 패턴 (87~91, 121~171)

js/core/ByeoriEngine.js : 121~150 (_getMismatchType)

```
1  _getMismatchType(userVector, originalVector, frameKeys = null) {
2    if (!userVector || !originalVector) return null;
3
4    const userReason = userVector.reason_analysis || {};
5    const origReason = originalVector.reason_analysis || {};
6
7    if (!!userReason.is_void !== !!origReason.is_void) return
   'void_mismatch';
8
9    if (userReason.attribution && origReason.attribution &&
10       userReason.attribution !== origReason.attribution) return
   'attribution_mismatch';
11
12   if (userReason.target && origReason.target &&
13       userReason.target !== origReason.target) {
14     const emotionSim = cosineSimilarity(
15       userVector.base || userVector,
16       originalVector.base || originalVector,
17       frameKeys
18     );
19     if (emotionSim >= 0.5) return 'target_displacement';
20   }
21
22   const emotionSimilarity = cosineSimilarity(
23     userVector.base || userVector,
24     originalVector.base || originalVector,
25     frameKeys
26   );
27   if (emotionSimilarity < 0.5) return 'emotion_mismatch';
28
29   return null;
30 }
```

위에서 아래로 **먼저 걸리는 게 답**이에요. 4장의 "문지기 줄서기". 순서가 곧 우선순위예요.

1. 한쪽만 회피했으면 → 회피 어긋남.
2. 둘 다 "탓"(attribution)을 적었는데 다르면 → 탓 어긋남.
3. 둘 다 "대상"(target)을 적었는데 다르고, 감정은 비슷하면(코사인 0.5 이상) → 대상 옮김. "같은 감정인데 향한 데가 다르다."
4. 감정 코사인이 0.5 미만이면 → 감정 어긋남.
5. 다 통과하면 어긋남 없음(`null`).

"탓"과 "대상"은 작가가 장면에 적어 둔 것과 관객이 이유 서술에서 뽑힌 것을 글자로 비교해요. 글자가 다르면 다른 거예요. 지금 상영 경로에선 관객 쪽 이 칸이 비어 있는 경우가 많아서, 실제론 1번과 4번이 주로 걸려요.

js/core/ByeoriEngine.js : 152~171 (_getTransitionPattern)

```
_getTransitionPattern(bucket, mismatchType, level, shape, userTraj, origTraj) {
  if (bucket === 'FIXATED') return 'fixation';

  if (level >= 0.5 && shape < 0.3 && userTraj.length >= 3 && origTraj.length >= 3) return 'displacement';

  if (bucket === 'HIGH') return 'echo_follow';
  if (bucket === 'MID') return 'bridge';

  if (bucket === 'LOW') {
    switch (mismatchType) {
      case 'void_mismatch': return 'avoidance';
      case 'target_displacement': return 'displacement';
      case 'emotion_mismatch': return 'contradiction';
      case 'attribution_mismatch': return 'contradiction';
      default: return 'bridge';
    }
  }

  return null;
}
```

역시 위에서 아래로. 이걸 한국어 문장으로 바꾸면 전이 패턴의 정의가 돼요. 외우지 말고 읽어 두세요.

패턴	언제	뜻

fixation	등급이 FIXATED	같은 감정에 갇힘
displacement	수준 ≥ 0.5인데 형태 < 0.3, 장면 셋 이상	장면마다 감정은 비슷한데 움직인 방향이 다름. "같은 감정, 다른 이유"의 흔적
echo_follow	등급 HIGH	잔향처럼 따라감
bridge	등급 MID, 또는 LOW인데 어긋남 종류가 없음	중립적 연결
avoidance	등급 LOW + 회피 어긋남	감정 접근 회피
contradiction	등급 LOW + 감정 어긋남 또는 탓 어긋남	반대로 튕김

함정

여기도 문서와 코드가 달라요. 설계 문서 §5 표에는 "echo_follow = 직전 두 움직임 방향 일치", "contradiction = 직전 움직임 방향 반대"라고 적혀 있어요. 코드는 그런 계산을 안 해요. 그냥 `등급`으로 갈라요. HIGH면 echo_follow, LOW면 어긋남 종류를 봐요. 문서가 처음 그린 그림이고, 구현하면서 단순화한 거예요. 어느 쪽이 더 좋냐는 별개 문제고, **지금 돌아가는 건 코드 쪽**이에요. 논문에 "전이 패턴은 움직임 방향으로 판정한다"고 쓰면 틀린 말이 돼요.

덩어리 ⑨ 내보내기 (93~118)

19장에서 봤어요. 지도 위 점(`affective_position`)을 하나 더 만들고, `debug` 쪽지에 계산 과정을 다 적고, 묶음 하나로 돌려줘요. 끝.

190줄을 한 장으로

덩어리	줄	하는 일	식의 어디
①	40~51	재료 받기, 없으면 빈 답	—
②	53~58	어느 칸으로 잴지	자
③	60~66	이번 장면 코사인 → 지금까지 평균	**수준**
④	68~75	궤적 두 목록 → 움직임 → 코사인	**형태**

⑤	77~81	관객만 회피했나	**회피**
⑥	83	셋을 곱함	**= 정렬도**
⑦	85	고착 · 끈적임 · 경계로 네 칸	등급
⑧	87~91	뭐가 어긋났나, 어떻게 움직였나	어긋남 · 패턴
⑨	93~118	묶음으로 내보내기	—

읽는 요령

190줄 = 재료 받기 → 자 고르기 → 코사인 평균(수준) → 움직임 코사인(형태) → 회피 0.7 → 곱 → 네 칸 → 이름표. 심장은 83번째 줄 한 줄.

✎ 미니 시험 21

1. 수준 0.8, 형태 날것 −0.2, 관객 회피 · 작가는 안 회피. 정렬도는? 형태 날것이 음수인데 정렬도가 음수가 안 되는 이유는 어느 줄 때문입니까?
2. 직전 등급 HIGH, 이번 정렬도 0.42. 등급은? 직전 등급이 MID였다면?
3. 관객 탓 = "self", 작가 탓 = "other", 감정 코사인 0.9. 어긋남 종류는? 왜 감정이 비슷한데도 그게 나옵니까?
4. 장면 셋, 수준 0.6, 형태 0.2, 등급 HIGH. 전이 패턴은? (함수의 몇 번째 줄에서 걸립니까?)

정답은 부록 E.

내 손으로 정리 · 아홉 덩어리 중 셋을 골라 내 말로

왜 이 식인가: 이론에서 온 것과 직접 정한 것

> **목표** 식의 모양(왜 곱인가, 왜 움직임인가)이 어디서 왔는지, 식 안의 숫자(0.7 · 0.85 · 0.50 · 셋 규칙)가 어디서 왔는지를 **둘로 나눠서** 말할 수 있게 된다. 이 둘을 섞어 말하면 심사위원이 바로 찌른다.

코드엔 "왜"가 없어요

21장을 다 읽으면 식은 읽혀요. 그런데 "왜 더하지 않고 곱하냐", "왜 0.7이냐"는 코드 어디에도 없어요. 그건 설계 문서 `docs/별이엔진_V4-궤적기반_정렬도-260327.md`에 있어요. §3이 "왜 이 공식이어야 했는가", §4가 "이론적 기반", §9가 "설계 결정 기록". 이 세 절이 이 장의 재료예요. 제가 여기서 하는 일은 그걸 "이론에서 온 것"과 "직접 정한 것"으로 가르는 거예요.

먼저 실패 이야기: V1에서 V4까지

이 식은 처음부터 이랬던 게 아니에요. 세 번 실패하고 네 번째예요. 실패 기록이 식의 모양을 설명해요.

판	식	뭐가 안 됐나
V1	감정 코사인 하나	"같은 감정이라도 이유가 다르면 다른 경험"을 못 담음. 둘 다 슬펐으면 끝.
V2	글 임베딩 코사인 + VAD 거리	"엄마한테 미안해서 울었다"와 "친구한테 미안해서 울었다"가 임베딩 공간에서 거의 같은 자리. 이유를 못 가름.
V3	감정 0.4 + 이유 0.4 + 태도 0.2	세 가지가 터짐. ① AI가 뽑는 이유 꼬리표가 같은 글에서도 흔들려 점수가 최대 0.14 널뜀. ② 이유 축이 15단계 꼬리표 맞추기라 해상도가 너무 낮음. ③ **더하기라서 한 축이 0이어도 다른 축이 메움.** 감정이 정반대(0)여도 이유 꼬리표가 우연히 같으면 0.54까지 올라감. 그리고 0.4/0.4/0.2라는 비율에 근거가 없었음.

V4	수준 × 형태 × 회피	이유를 재지 않기로 함. 대신 이유의 결과를 봄. 같은 감정이라도 이유가 다르면 다음 장면에서 다르게 움직일 테니, 움직임(형태)을 비교하면 이유 차이가 간접으로 드러남.

여기서 V4의 핵심 전환을 한 문장으로 하면 이래요. **"이유를 직접 재는 대신 이유의 결과를 관찰한다."** 장면 1에서 둘 다 슬펐어도, "내 탓"이라 느낀 사람은 장면 2에서 죄책감 쪽으로, "그 사람 탓"이라 느낀 사람은 분노 쪽으로 움직여요. 궤적이 갈라지면 이유가 달랐다는 게 드러나요. AI가 이유를 뽑을 필요가 없어져요. 이게 V3의 세 문제를 한 번에 없앤 수였어요.

이론에서 온 것: 식의 모양

식의 모양을 정한 외부 이론은 사실상 하나예요. 나머지는 그 모양이 말이 된다는 뒷받침이에요.

① 왜 두 항인가, 왜 곱인가 → Butler의 TIES

Butler(2011, 2017)가 "감정은 시간에 걸쳐 펼쳐지는 대인 시스템"이라며 한 주장이 이거예요. **감정의 수준(level)과 감정의 움직임(dynamics)은 서로 독립이고, 둘 다 따로 관계의 질을 예측한다.** 별이엔진은 이 구분을 그대로 가져왔어요. 수준 따로, 형태 따로. 그리고 독립인 두 것을 결합할 때 더하면 V3의 ③ 문제(한쪽이 메움)가 다시 생기니까 곱했어요. 곱이면 한쪽이 0일 때 전체가 0. "감정이 비슷해도 궤적이 반대면 낮고, 궤적이 비슷해도 감정이 다르면 낮다"가 구조로 보장돼요. §9.1이 이 결정이에요.

딱 하나만 읽으시겠다면 Butler 2017이에요. 여섯 쪽이에요. 이 식의 뼈대가 거기서 왔어요.

② 왜 감정을 코사인으로 재도 되나 → 정서 유사도 연구

"같은 상황에서 같은 감정을 느끼는 것"이 공감 · 조율 · 관계 만족과 연결된다는 연구 묶음(Locke & Horowitz 1990, Preston & de Waal 2002, Anderson 외 2003 등). 이건 "감정이 비슷하다는 게 의미 있는 양이다"라는 뒷받침이지, 코사인이라는 계산법을 준 건 아니에요. 코사인은 그냥 벡터 비슷함의 표준 도구예요.

③ 왜 움직임의 궤적을 비교하나 → 감정 궤적 그래프

서사 분석에서 이야기를 "시간순 감정 곡선"으로 만들어 비교하는 건 이미 있는 방법이에요(Reagan 외 2016, "이야기의 감정 곡선은 여섯 모양으로 수렴한다"). 별이엔진은 그 방법을 빌리되 **감정 값이 아니라 감정의 변화량**을 비교해요. 그래서 "얼마나 슬펐나"가 아니라 "어느 쪽으로 움직였나"가 잡혀요. 이건 빌린 방법 위에 얹은 TEM의 변형이에요.

④ 왜 이유를 안 재나 → 인지적 평가 이론의 측정 도구

Lazarus의 평가 이론은 "같은 사건도 사람마다 다르게 평가한다"고 해요. V3는 그 평가를 AI로 뽑으려 했죠. 그런데 심리학에서 평가를 재는 검증된 도구는 전부 자기보고 설문이에요. AI가 글에서 꼬리표를 뽑는 건 검증된 측정이 아니고 재현성도 확인 안 됐어요. 그래서 V4는 이 이론을 버린 게 아니라, 그 이론이 말하는 차이가 궤적에 나타날 거라 보고 간접으로 가요. §4.4가 이 얘기예요.

직접 정한 것: 식 안의 숫자

이제 반대편. 아래 숫자들은 **이론에서 유도된 게 아니에요.** 당신이(여러 세션에 걸쳐) 정한 값이에요. 이걸 "이론에서 왔다"고 말하면 틀린 말이고, 심사위원이 "어느 이론?"이라고 물으면 막혀요. 반대로 "정한 값이고, 이유는 이렇고, 바꾸면 기록한다"고 말하면 아무도 안 찔러요.

숫자	어디	왜 그 값인가 (문서에 적힌 이유)
회피 벌점 0.7	엔진 81	V3의 태도 축이 거의 항상 0.7로 고정돼 "상수 덧셈"처럼 굴던 걸, 회피가 일어났을 때만 곱하는 보정으로 강등(§9.4). 0.7이라는 값 자체의 근거는 V3에서 쓰던 값을 이어받은 것.
고착 문턱 0.85	등급 함수	"최근 셋이 서로 코사인 0.85 이상이면 같은 감정 반복"으로 정함. 근거 문장 없음. 7-28 페르소나 리트머스에서 "자 A는 병자"라고 한 수술 대상이 이 근처.
등급 경계 0.50 / 0.10	등급 함수	"V4 곱셈 분포 기준, 파일럿 후 재조정." 곱하면 값이 전체적으로 낮아지니 경계를 내린 것. 매뉴얼의 0.7/0.4는 조정 전 값.
끈적임 0.40 / 0.15	등급 함수	경계에서 등급이 깜빡이는 걸 막으려고. 폭은 정한 것.

displacement 조건 0.5 / 0.3	엔진 155	"수준은 높은데 형태는 낮다"의 높다 · 낮다 경계. 정한 것.
어긋남 문턱 0.5	엔진 139 · 147	"감정이 비슷하다 / 다르다"의 경계. 정한 것.
음수 형태 → 0	형태 함수 14	"반정렬"이라는 개념을 아직 안 정했으니 반대 방향은 그냥 0으로(§9.2). 나중에 (1+날것)/2로 부드럽게 바꿀 수 있다고 적어 둠.
17칸 목록	math.js 19	고통 10 + 회복 7. 어떤 감정 모델에서 가져온 게 아니라 TEM이 다루는 서사에 맞춰 고른 것. 7월 27일부터는 기억마다 실제 쓰는 칸만으로 좁힘.

딱 하나, 숫자가 아니라 **수학적 필연**인 게 있어요. "장면 셋 미만이면 형태 = 1" (§9.3). 움직임 둘을 비교하려면 장면 셋이 필요하다는 건 정한 게 아니라 셀 수 있는 사실이에요. 이것만은 "왜요?"에 "세어 보세요"로 답하면 돼요.

> **함정**
>
> 위 표의 값들이 지금 **어디에도 행마다 기록되지 않아요.** 손잡이를 바꾸면 그 뒤 행과 그 전 행이 다른 자로 잰 게 되는데, 어느 행이 어느 자인지 표에 안 남아요. 9월 26일 기록 공백 제안(회차 일지 표에 "출처 꼬리표" 한 열)이 정확히 이 구멍을 메우는 거예요. 이 장을 읽었으면 왜 그 제안이 "튜닝 전에" 들어가야 하는지도 보일 거예요.

"원본"이라는 말에 대해

하나 더 짚을게요. TEM 철학은 "순수한 원본은 없다"인데 식은 "원본 감정"과 비교해요. 모순 아니냐. 7월 4일에 이걸 봉합했어요. 식에서 "원본"이라 부르는 건 **작가가 처음 서술했을 때의 궤적**(`telling_trajectory`)이에요. 진짜 사건이 아니라 첫 말하기. 그러니까 정렬도는 "진실과 얼마나 같나"가 아니라 "첫 서술과 얼마나 같은 결로 겪었나"예요. 공식과 표는 안 바뀌었고 말만 바뀌었어요. 설계 원칙 7번("initial telling trajectory, not original")이 이거예요. 누가 "원본이 어딨냐"고 물으면 이렇게 답하면 돼요.

심사위원이 물으면

이 장을 한 문단으로 줄이면 면접 답이 돼요. 연습 삼아 적어 둘게요.

> "정렬도는 두 항의 곱입니다. 장면마다의 감정 유사도 평균과, 장면 사이 감정 변화 방향의 유사도. 두 항을 분리한 건 Butler의 TIES 이론, 감정의 수준과 역학이 독립이라는 주장에서 가져왔고, 더하지 않고 곱한 건 한 항이 다른 항을 메우는 걸 막기 위한 구조적 선택입니다. 변화량을 비교하는 건 감정 궤적 그래프 방법론의 변형입니다. 식 안의 문턱값들, 0.7 · 0.85 · 0.5는 이론 값이 아니라 설계 값이고, 파일럿 분포를 보고 조정했으며, 앞으로 바꿀 때는 회차마다 기록합니다."

이 문단을 안 보고 말할 수 있으면 22장은 끝이에요.

읽는 요령

모양(두 항 · 곱 · 움직임)은 이론에서, 숫자(0.7 · 0.85 · 0.50 · 0.5)는 손에서. 둘을 갈라 말하라. "셋 규칙"만 수학적 필연.

✎ 미니 시험 22

1. 누가 "0.7은 어느 논문에서 왔나요?"라고 물었습니다. 뭐라고 답합니까?
2. 식을 곱에서 더하기(수준 0.5 + 형태 0.5)로 바꾸면, 24장 두 번째 문제의 관객(수준 0.97, 형태 0)의 정렬도는 얼마가 됩니까? 그게 왜 문제입니까?
3. "TEM은 원본과 비교한다, 그런데 TEM은 원본이 없다고 한다"는 지적에 한 문장으로 답하세요.

정답은 부록 E.

내 손으로 정리 · "심사위원이 물으면" 문단을 안 보고 써 보기

오염 추적기: 새 값 10%, 옛 값 90%

> **목표** 회차가 끝날 때 한 번 도는 `js/core/ContaminationTracker.js`의 핵심인 지수이동평균을 손으로 굴려 보고, 그 값이 어떻게 "단계"로 바뀌는지, 3축이 뭔지 뼈대만 본다. 그리고 이게 측정이 아니라 **연출 제어**라는 걸 분명히 한다.

먼저 성격부터

별이엔진은 측정이에요. 오염 추적기는 아니에요. 명세 첫 줄에 이렇게 적혀 있어요. **"이것은 진실 모델이 아니라 연출 제어 모델이다. 오염 벡터의 값은 '틀릴' 수 없다. 표현 강도를 결정하는 제어 변수다."**

무슨 말이냐면, 오염 값은 "이 기억이 얼마나 변했나"의 참값이 아니에요. "지금 무대를 얼마나 흔들어 보여 줄까"를 정하는 손잡이예요. 그래서 이 파일의 숫자들은 전부 연출가가 정한 값이고, 그걸 두고 "맞다 틀리다"를 따지는 건 질문이 잘못된 거예요. 따질 건 "이 손잡이가 무대에서 어떻게 보이나"뿐이에요. 이 구분을 놓치면 논문에서 오염 값을 측정 결과처럼 쓰게 되고, 그건 ICIDS가 찌른 "제목은 궤적인데 증거는 순간"과 같은 종류의 실수예요.

그리고 또 하나. 오염은 **엔진을 절대 건드리지 않아요.** 파일 13번째 줄에 "별이엔진 V4는 변경하지 않는다. 이 모듈은 V4 출력의 소비자이다"라고 박혀 있어요. 엔진이 답을 내면, 오염이 그 답을 받아서 자기 값을 굴려요. 반대 방향은 없어요.

언제 도나

장면마다가 아니라 **회차가 끝날 때 한 번**이에요. 함수 이름이 `updateContamination`이고, 재료는 "지금까지 오염 상태"와 "이번 회차 엔진 답"이에요. 돌려주는 건 "다음 오염 상태". 재료를 고치지 않고 새 묶음을 만들어 돌려줘요(185번째 줄 `const next = { ...state }`, 10장의 "복사해서 새로"). 그 결과가 `memories` 표의 `cont_`로 시작하는 열들에 저장돼요.

지수이동평균: 새 값 10%, 옛 값 90%

이 파일의 수학은 이 한 줄이에요.

js/core/ContaminationTracker.js : 221~224

```
// 3. EMA update (replaces cumulative + decay)
//    axis = α * signal + (1 − α) * axis_old
next.cont_drift = clamp01(α * driftSignal + (1 − α) * next.cont_drift);
next.cont_fixation = clamp01(α * fixSignal + (1 − α) * next.cont_fixation);
```

`α`(알파)는 27번째 줄에 0.10으로 정해져 있어요. 그러니까 식은 이거예요.

> **새 축값 = 이번 신호 × 0.1 + 옛 축값 × 0.9**

이름이 "지수이동평균"이라 어려워 보이는데, 뜻은 "**옛것을 90% 남기고 새것을 10% 섞는다**"예요. 그게 다예요. `clamp01`은 0~1 밖으로 나가면 잘라 주는 것.

손으로 굴려 보기

처음 오염이 0이고, 어긋나는 관객이 연달아 와서 이번 신호가 매번 0.6이라고 해 봐요.

회차	계산	표류 축
1	0.6 × 0.1 + 0 × 0.9	0.060
2	0.6 × 0.1 + 0.060 × 0.9	0.114
3	0.6 × 0.1 + 0.114 × 0.9	0.163
…		
8		0.342
9		**0.368**

보이죠. 한 명이 와서는 거의 안 움직여요. 같은 쪽으로 어긋나는 사람이 **아홉 명** 연달아 와야 0.35를 넘어요. 0.35가 뭐냐면, 아래에서 나올 "1단계 켜지는 문턱"이에요. 그러니까 이 설계는 "한 사람의 반응으로 기억 전체가 기우는 일은 없게, 여러 사람이 같은 쪽으로 밀어야 기울게"예요. 27번째 줄 주석에 "반감기 약 6.6회차, 최근 7회차가 축값을 지배"라고 적힌 게 이 뜻이에요. 반감기는 "옛 값의 힘이 절반으로 줄기까지 몇 번"이고, 0.9를 몇 번 곱해야 0.5가 되나를 세면 6.6번이에요.

α를 0.5로 올리면 어떻게 되나는 24장 다섯 번째 문제예요. 미리 말하면, 두 명이면 넘어요.

신호는 어디서 오나

위 식의 "이번 신호"가 뭔지 봐야 하죠. 둘이에요. 표류 신호와 고착 신호.

js/core/ContaminationTracker.js : 211~219

```
const driftSignal =
  CONTAMINATION.DRIFT_WEIGHTS.LEVEL * (1 - level) +
  CONTAMINATION.DRIFT_WEIGHTS.SHAPE * (1 - effectiveShape) +
  CONTAMINATION.DRIFT_WEIGHTS.MISMATCH * mismatchBonus;

const fixSignal =
  CONTAMINATION.FIX_WEIGHTS.FIXATION * fixation_level +
  CONTAMINATION.FIX_WEIGHTS.ALIGNMENT * normalizeAlignmentBonus(alignment)
+
  CONTAMINATION.FIX_WEIGHTS.PATTERN * patternBonus;
```

표류 신호는 "얼마나 어긋났나"예요. 엔진의 수준이 낮을수록(1 − 수준), 형태가 낮을수록(1 − 형태), 어긋남 종류가 무거울수록 커요. 비율은 29~33번째 줄에 0.45 · 0.35 · 0.20. 어긋남 종류마다 무게가 있어요(41~50번째 줄). 감정 어긋남 1.0, 회피 0.8, 탓 0.7, 대상 0.6, 흐릿함 0.5, 없음 0.

예를 들어 수준 0.4, 형태 0.6, 감정 어긋남이면: 0.45 × 0.6 + 0.35 × 0.4 + 0.20 × 1.0 = 0.27 + 0.14 + 0.20 = **0.61**. 위 표의 0.6이 대충 이런 관객이에요.

고착 신호는 "얼마나 같은 데 갇혔나"예요. 엔진의 고착 수준(최근 셋의 코사인 평균)이 0.55, 정렬도가 0.55를 넘는 만큼이 0.25, 패턴이 fixation이면 1.0 · echo_follow면 0.5 · bridge면 0.2 하는 보너스가 0.20.

이 비율들, 0.45 · 0.35 · 0.20과 0.55 · 0.25 · 0.20, 그리고 어긋남 무게표. 전부 22장 표 쪽, **직접 정한 값**이에요. 22장에서 "모양은 이론, 숫자는 손"이라고 했죠. 이 파일은 숫자가 더 많아요. 그만큼 "연출 제어"예요.

실제로 있었던 일

2026년 7월 14일. 대화 경로(유령과 말하는 상영)로 플레이한 행은 어긋남 종류를 `resonance` · `vague` · `dissonance`라는 다른 단어로 적고 있었어요. 그런데 무게표는 엔진 단어(`emotion_mismatch` 등)만 알았어요. 그래서 실제 관객이 걷는 경로에선 어긋남 보너스가 항상 0이었고, **표류가 한 번도 쌓이지 않았어요.** 오염이 안 생기는 게 아니라 못 생기고 있었던 거예요. 고친 자리가 110~118번째 줄 `normalizeMismatch`, 두 어휘를 한 어휘로 접는 함수. 주석에 사고 경위가 그대로 적혀 있어요. 이게 "숫자가 틀릴 수 없다"는 말의 뜻이에요. 값이 틀린 게 아니라 안 흐르고 있었던 거예요. 15장의 비교표 읽기가 이런 걸 잡아요.

값에서 단계로

축값 둘이 굴러가면 그걸로 "지금 무대가 어느 단계냐"를 정해요.

js/core/ContaminationTracker.js : 245~254

```
if (
  next.cont_fixation >= HYPERCOMPLETION_FIXATION &&
  next.cont_fixation > next.cont_drift
) {
  next.cont_stage = 'hypercompletion';
} else if (next.cont_drift >= BIASED_INCLINATION_DRIFT) {
  next.cont_stage = 'biased_inclination';
} else {
  next.cont_stage = 'stable';
}
```

문턱은 61~64번째 줄. 고착 0.65, 표류 0.35. 읽으면: **고착이 0.65 이상이고 표류보다 크면 과완결, 아니면 표류가 0.35 이상이면 편향 기울기, 둘 다 아니면 안정.** 4장의 문지기 셋. 위에서부터 먼저 걸리는 게 답. 과완결을 먼저 보니까, 고착도 높고 표류도 높으면 과완결이 이겨요.

이 세 단어가 무대에서 뭐가 되냐면, 장면 글이 `text_stage_1/2/3` 중 뭘 보여 줄지, 안개와 색을 얼마나 흔들지를 정해요. 그런데 play-test 2,702번째 줄 주석에 이런 게 있어요. "관객이 120명 와도 2 · 3단계 글이 한 번도 표시되지 않았다." 위의 아홉 명 계산이 그 이유예요. 같은 쪽으로 미는 사람이 연달아 아홉이 있어야 하는데 실제

관객은 그렇게 안 오니까요. 이건 결함이 아니라 지금 설정의 결과예요. 10월 튜닝이 이 손잡이를 만질 거고, 그래서 그 전에 행마다 설정을 적어 두자는 거예요.

실제로 있었던 일

2026년 7월 9일. 관리자 화면이 단계를 `inclination`이라고 짧게 적어 넣고 있었는데, 연출 쪽은 `biased_inclination`이라는 긴 이름과 비교하고 있었어요. 글자가 안 맞으니 세기가 항상 0으로 나왔고, 작가가 써 둔 2 · 3단계 글이 한 번도 관객에게 닿지 않았어요. 고친 자리가 439~443번째 줄 `normalizeStage`. 두 철자를 하나로 접어요. 이 파일엔 이런 "접는 함수"가 둘 있는데(어긋남 하나, 단계 하나), 둘 다 사고 뒤에 생겼어요. 8장에서 "이름이 다르면 다른 것"이라고 했죠. 이게 그 비용이에요.

3축: 뼈대만

위의 두 축(표류 · 고착)은 "2축 MVP"예요. 그 뒤에 3축이 더 있어요. 256~303번째 줄. 지금은 모양만 보고 넘어가도 돼요. 10월 튜닝 뒤에 이 자리를 다시 열면 그때 자세히.

축	뭐를 쌓나	어떻게
갈라짐 `divergence`	원본에서 얼마나 멀어졌나	(1 − 형태) × (1 − 수준)을 더하되, 회차가 쌓일수록 더하는 양이 줄어듦(1 ÷ (1 + 깊이 × 0.05)). "처음 온 사람들이 더 크게 흔들고, 뒤로 갈수록 둔해짐."
수렴 `convergence`	자꾸 같은 쪽으로 모이나	깊이 3 이상부터만. 고착이 0.85 넘으면 정렬도 × 0.4, 수준 · 형태 둘 다 0.7 넘으면 × 0.15, 정렬도 0.5 넘으면 × 0.03을 더함.
다양성 `heterogeneity`	해석이 얼마나 갈리나	지금까지 정렬도들의 **분산**(흩어진 정도) × 4. 분산을 한 번에 다 안 들고 굴리는 요령(Welford)을 씀. 그래서 `_cont_align_mean` · `_cont_align_m2`라는 밑줄 시작 칸이 두 개 있어요(8장의 "밑줄 = 속사정").

그리고 291~303번째 줄에서 이 셋을 섞어 "단계 비율" 셋을 만들어요. 1단계 = 갈라짐 × (1 − 수렴), 2단계 = 다양성 × (깊이 ÷ 5, 최대 1), 3단계 = 수렴 × (1 − 다양성). 셋을 합이 1이 되게 나눠요. 그래서 무대는 "지금 1단계냐 3단계냐"가

아니라 "1단계 0.6, 2단계 0.3, 3단계 0.1만큼 섞어서" 보여 줄 수 있어요.
CLAUDE.md의 "Stage는 배타적 X, 섞어서 렌더링"이 이 줄들이에요.

바이러스 이야기에 대해

명세 §13이 이 3축을 HIV 바이러스 진화 추적법에 빗대 설명해요. 갈라짐은 "뿌리에서 끝까지 거리", 다양성은 "한 환자 안 변이체 구름의 폭", 수렴은 "면역 압력 아래 같은 변이로 모임", 깊이는 "계대 횟수". 문서는 "임의 설계가 아니다, 구조적으로 동일하다"고 꽤 세게 적었어요.

제 말은 이거예요. 이건 **같은 수학 모양을 빌려왔다**는 뜻이지, 기억이 바이러스라는 증명이 아니에요. 분산을 "구름 폭"이라 부르고 둔화 곡선을 "분자시계 포화"라 부르는 건 이름 붙이기예요. 그 이름이 설계를 떠올리는 데 도움이 됐고, 논문에서 "참고한 틀"로 적을 수 있어요. 그런데 "생물학이 근거다"라고 쓰면 생물학자가 와서 "경쟁도 선택도 없는데 왜 준종이냐"고 물어요. 실제로 6월 10일 v0.6 개정초안이 그 지적을 스스로 했고, 9월 26일 연구 명세도 "생물학 비유는 유지하되 비유라고 명시"로 정리했어요. 그러니 이 장을 읽고 남길 문장은 이거예요. **"3축은 갈라짐 · 수렴 · 다양성이고, 계산은 누적 · 조건부 누적 · 분산이며, 바이러스 진화 측정법과 같은 모양이라 그 이름을 빌렸다."**

> **읽는 요령**
> 오염 = 연출 제어. 회차 끝 1번. 새 신호 10% + 옛 값 90%. 아홉 명이 같은 쪽으로 밀어야 1단계. 엔진은 안 건드린다. 바이러스는 비유.

✎ 미니 시험 23

1. 지금 표류 축이 0.30이고 이번 회차 표류 신호가 0.8입니다. 다음 표류 축은? 단계는?
2. 그 회차에 고착 축이 0.70이 됐다면(표류는 1번 답 그대로) 단계는? 어느 줄에서 걸립니까?
3. AI가 "오염이 너무 느리게 쌓여서 엔진의 정렬도 계산을 좀 세게 바꿨어요"라고 했습니다. 이 문장에서 설계 원칙에 어긋나는 부분을 짚고, 어느 파일 몇 번째 줄 주석을 들이밀겠습니까?

정답은 부록 E.

내 손으로 정리 · "새 값 10% 옛 값 90%"가 무대에서 뭘 뜻하는지 한 줄

다섯 문제: 돌리기 전에 맞히기

> **목표** 측정을 이해했다는 시험은 하나다. **돌리기 전에 결과를 맞히는가.** 아래 다섯 문제를 코드 안 보고 답할 수 있으면 "설명 관문"을 통과한 것이다. 답은 바로 아래 붙였다. 먼저 쓰고 넘기라.

왜 이 시험인가

19~23장을 읽었다고 이해한 건 아니에요. 읽은 건 "따라갈 수 있다"이고, 이해한 건 "안 보고 예측할 수 있다"예요. 둘 사이가 설명 관문이에요. 심사위원도 면접관도 이 다섯 문제와 똑같은 종류의 걸 물어요. "그럼 이런 관객이 오면 숫자가 어떻게 나오죠?" 그때 코드를 열면 늦어요.

다섯 문제는 각각 식의 다른 자리를 찔러요. 1번은 전체, 2번은 형태, 3번은 셋 규칙, 4번은 회피, 5번은 오염 알파. 순서대로 푸세요.

문제 1 · 관객이 모든 장면에서 작가와 똑같이 느꼈다

관객이 장면 다섯 개를 봤고, 다섯 장면 모두 작가가 심어 둔 원본 감정과 완전히 똑같은 감정을 입력했어요. 회피는 없었어요. 정렬도, 등급, 전이 패턴은?

> **풀이**
> 장면마다 코사인은 똑같은 화살표끼리니까 전부 1. 수준 = 1의 평균 = **1**.
>
> 움직임도 똑같아요. 관객 목록과 원본 목록이 같은 묶음들이니 빼면 같은 움직임이 나오고, 같은 걸 코사인하면 1. 형태 = **1**. 장면 다섯이라 셋 규칙은 안 걸려요.
>
> 회피 없음 → 1.

정렬도 = 1 × 1 × 1 = **1.0**. 등급: 고착 검사부터 하는데, 다섯 장면 감정이 서로 다르면(보통 그렇죠) 최근 셋 코사인이 0.85를 안 넘으니 FIXATED 아님. 0.50 이상이니 **HIGH**. 패턴: FIXATED 아니고, 형태 1이라 displacement 아니고, HIGH니까 **echo_follow**.

단, 함정 하나. 작가 원본 다섯 장면이 전부 비슷한 감정이었다면(예: 다섯 장면 다 슬픔), 관객이 그걸 그대로 따라 느낀 것만으로 최근 셋 코사인이 0.85를 넘어 **FIXATED**가 돼요. 정렬도는 1인데 등급은 FIXATED, 패턴은 fixation. "완벽히 따라갔는데 갇혔다"가 나오는 거예요. 이게 7월 28일 페르소나 리트머스에서 "자A는 병자"라고 한 자리예요. 고착 검사가 작가 쪽 궤적이 단조로운 걸 관객 탓으로 돌려요. 이 함정을 알면 문제 1은 만점이에요.

문제 2 · 장면마다 감정은 비슷한데, 움직인 방향이 반대

장면 셋. 작가는 장면 1에서 2로 갈 때 조금 더 무서워졌다가 3에서 돌아왔어요. 관객은 1에서 2로 갈 때 조금 덜 무서워졌다가 3에서 돌아왔어요. 장면마다 감정 자체는 거의 같아요. 두 칸짜리로 적으면:

	장면 1	장면 2	장면 3
작가	{ fear 0.5, joy 0.5 }	{ fear 0.6, joy 0.4 }	{ fear 0.5, joy 0.5 }
관객	{ fear 0.5, joy 0.5 }	{ fear 0.4, joy 0.6 }	{ fear 0.5, joy 0.5 }

수준, 형태, 정렬도, 패턴은? 그리고 "순서만 다르게 봤다면" 뭐가 달라집니까?

풀이
장면 점수: 1번 똑같으니 1. 2번은 { 0.6, 0.4 }와 { 0.4, 0.6 }의 코사인 = 0.48 ÷ 0.52 = 0.92. 3번 1. 수준 = (1 + 0.92 + 1) ÷ 3 = **0.97**. 높아요.

움직임: 작가는 { +0.1, −0.1 } 다음 { −0.1, +0.1 }. 관객은 { −0.1, +0.1 } 다음 { +0.1, −0.1 }. 한 줄로 펴면 [0.1, −0.1, −0.1, 0.1]과 [−0.1, 0.1, 0.1, −0.1]. 칸마다 부호가 정반대. 곱해서 더하면 −0.04, 길이 둘 다 0.2, 나누면 **−1**. 완전히 반대. 깎으면 형태 = **0**.

정렬도 = 0.97 × 0 × 1 = **0**. 수준이 아무리 높아도 곱이니까 0이에요. 이게 22장 "왜 곱인가"의 답이에요. 더했으면 0.49쯤 나와서 "그럭저럭 비슷"이 됐을 거예요.

패턴: FIXATED 아님. 수준 0.97 ≥ 0.5이고 형태 0 < 0.3이고 장면 셋 → 함수 둘째 갈림길에서 **displacement**. 등급은 정렬도 0이니 LOW인데, displacement 검사가 등급 검사보다 먼저라 LOW 쪽 갈림길까지 안 가요.

"순서만 다르게"는 아무것도 안 바꿔요. 관객이 3 → 1 → 2로 봤으면 원본도 3 → 1 → 2로 재배열해서 비교하니까요. 떨어지는 건 순서가 아니라 그 순서로 봤을 때의 움직임 방향이에요. 21장 끝에서 말한 그 자리예요.

문제 3 · 장면이 두 개뿐

관객이 장면 둘만 보고 나갔어요. 수준은 0.6이었어요. 형태는? 정렬도는? `debug.shape_active`는 뭐라고 찍힙니까? 왜 그렇게 설계했습니까?

풀이

형태 = **1.0**, `shape_active: false`. 정렬도 = 0.6 × 1 × 1 = **0.6**, 즉 수준 그대로.

왜냐. 장면 둘이면 움직임이 하나뿐이에요. 움직임 하나로는 "방향이 비슷한가"를 물을 수 없어요. 비교 대상이 둘은 있어야 하니까. 그래서 셋 미만이면 "형태 정보 없음"이고, 그걸 1로 놓으면 식이 정렬도 = 수준이 되어 갈림길 없이 한 식으로 다 덮여요(§9.3). 1 대신 0을 넣었으면 초반 정렬도가 항상 0이 됐을 거고, 0.5를 넣었으면 "형태가 반쯤 비슷하다"는 거짓말이 됐을 거예요. 1은 "곱해도 아무 영향 없음"이라 정직해요.

이건 22장 표에서 유일하게 "정한 값"이 아니라 "셀 수 있는 사실"이었던 자리예요. 누가 "왜 셋이냐"고 물으면 "움직임 둘을 비교하려면 점이 셋 필요하니까요"로 끝.

문제 4 · 관객도 회피했고 작가도 회피했다

어떤 장면에서 작가가 감정을 안 적고 비워 뒀어요(작가 쪽 회피). 관객도 그 장면에서 회피를 골랐어요. 회피 항은? 어긋남 종류는?

> **풀이**
>
> 회피 항 = **1.0**, 벌점 없음. 81번째 줄이 "관객 회피 그리고 작가 비회피"일 때만 0.7이니까, 둘 다 회피면 조건이 안 맞아요.
>
> 어긋남 종류: 127번째 줄이 "한쪽만 회피"(`!!a !== !!b`)일 때 회피 어긋남인데, 둘 다 참이라 같음 → 통과. 그다음 탓 · 대상 · 감정 검사로 내려가요. 둘 다 감정이 비어 있으면 코사인은 길이 0이라 0이 나오고(20장 25번째 줄), 0 < 0.5라 **감정 어긋남**이 찍힐 수 있어요. 이건 좀 이상하죠. "둘 다 아무 말 안 했는데 감정이 어긋났다"니까. 실제 코드가 그렇게 돼 있어요. 이 자리를 발견했으면 당신은 이미 코드를 "읽어서 검사"한 거예요. 고칠지 말지는 별개 결정이고(죽은 레버일 수도 있어요, 7월 2일 교훈), 지금은 "코드는 이렇게 돈다"를 아는 게 목표예요.

문제 5 · 오염 알파를 0.1에서 0.5로 올리면

23장의 "같은 쪽으로 어긋나는 관객, 신호 0.6"이 계속 온다고 할 때, α = 0.5면 몇 명째에 1단계(표류 0.35)가 켜집니까? 반감기는? 무대에서는 뭐가 달라집니까?

> **풀이**
>
> 식이 "새 값 50% + 옛 값 50%"가 돼요. 1회차: 0.6 × 0.5 + 0 = 0.30. 2회차: 0.6 × 0.5 + 0.30 × 0.5 = 0.30 + 0.15 = **0.45**. 두 명째에 0.35를 넘어요. α 0.1일 땐 아홉 명이었죠.

> 반감기: 0.5를 몇 번 곱해야 0.5가 되나 → **1번**. 옛 값의 힘이 한 회차 만에 절반으로 줄어요. α 0.1일 땐 6.6번.
>
> 무대에서: 한두 사람의 반응이 기억 전체의 단계를 뒤집어요. 어긋나는 사람 둘 오면 1단계 글로 바뀌고, 다음에 맞는 사람 하나 오면 도로 안정으로 돌아와요. 들썩거려요. "여러 사람이 같은 쪽으로 밀어야 기운다"는 설계 의도가 없어져요. 대신 "2 · 3단계 글이 한 번도 안 보인다"는 지금 문제는 풀려요. 그러니 이건 좋다 나쁘다가 아니라 **어느 쪽 연출을 원하느냐**예요. 그게 23장 첫 문단, "오염은 연출 제어"의 뜻이에요. 10월 튜닝이 이 손잡이를 돌릴 거고, 돌리기 전에 "돌리면 이렇게 된다"를 지금처럼 손으로 먼저 굴려 보는 게 순서예요.

여기까지 왔으면

다섯 문제를 답 안 보고 맞혔다면 이런 말을 할 수 있어요.

- 정렬도는 수준 × 형태 × 회피이고, 수준은 장면별 코사인 평균, 형태는 움직임 코사인, 회피는 0.7 또는 1.
- 두 항을 나눈 건 Butler, 곱한 건 보상 차단, 움직임을 보는 건 이유를 재지 않기 위해서.
- 0.7 · 0.85 · 0.50 · 0.5는 정한 값이고, 셋 규칙만 필연.
- 오염은 측정이 아니라 연출 제어, 새 값 10% 옛 값 90%, 아홉 명이 밀어야 기운다.
- 문서와 코드가 다른 자리를 두 군데 알고 있고, 둘이 다르면 코드가 진실이다.

이게 "내 엔진을 내가 설명할 수 있다"의 전부예요. 2년이 아니라 이 여섯 장이었어요. 남은 건 이걸 남 앞에서 5분 안에 말하는 연습이고, 그건 책으로 안 돼요. 22장 끝 문단을 소리 내서 읽는 걸로 시작하세요.

다음은 23장에서 뼈대만 본 3축과, 이 책이 아예 안 다룬 지층 침식(977줄)이에요. 둘 다 10월 튜닝 뒤에 손잡이 값이 자리 잡으면 그때 같은 방식으로 한 줄씩 보면 돼요. 지금 보면 숫자가 바뀔 걸 외우게 되니까요.

내 손으로 정리 · 다섯 문제 중 틀린 것과 왜 틀렸는지

부록 A

기호 사전

코드에서 만나는 기호를 **보이는 순서**가 아니라 **헷갈리는 짝**끼리 묶었습니다. 오른쪽 끝은 자세히 다룬 장.

기호	뜻	장
`//`	한 줄 메모. 컴퓨터는 무시	3
`/* … */`	여러 줄 메모	3
`/** … */`	함수 설명 메모(안에 @param 재료 설명, @returns 답 설명)	3
`--`	SQL 의 한 줄 메모	16
`<!-- … -->`	html 의 메모	2
`=`	담아라 (오른쪽을 왼쪽 상자에)	3
`=== !==`	같은가 · 다른가 (질문)	4
`== !=`	대충 같은가 · 다른가 (옛 방식). `x != null` = 비어 있지 않다	4
`+= -= *=`	그만큼 늘려 · 줄여 · 곱해 다시 담기	8
`++ --`	1 늘리기 · 1 줄이기 (반복 번호에서)	7
`< > <= >=`	작다 · 크다 · 작거나 같다 · 크거나 같다	4
`&& || !`	그리고 · 또는 · 아니다	4
`|| (값 자리에서)`	왼쪽이 비면 오른쪽 (기본값)	8
`??`	왼쪽이 정말 없을 때만 오른쪽	8
`?.`	있으면 점, 없으면 조용히 빔 (안전한 점)	8
`? :`	한 줄 갈림길 (질문 ? 맞으면 : 아니면)	4
`.`	"~의" (묶음에서 한 칸 꺼내기 · 함수 붙이기)	8
`...`	펼치기 (묶음 · 목록의 내용을 전부 쏟기)	10
`=>`	화살표 함수 (재료 => 답)	5
`( )`	재료 넣는 괄호. 이름 뒤에 붙으면 "돌려라"(호출)	5 · 9
`[ ]`	목록. `목록[0]` 은 첫 항목	3

`{ }` (쌍점 있음)	이름표 달린 묶음 (객체)	10
`{ }` (쌍점 없음)	코드 덩어리 (함수 몸통 · if 몸통 · 반복 몸통)	5 · 6
`{ }` (= 왼쪽)	묶음 풀기 (const { a, b } = 묶음)	10
`" " ' '`	글자 그대로	3
`` ` ` ${ } ``	글자 속에 값 끼우기	8
`;`	문장 끝 (없어도 대개 됨)	—
`,`	항목 나누기 (목록 · 묶음 · 재료)	3
`:`	묶음의 이름표와 값 사이 · css 의 속성과 값 사이 · TypeScript 의 종류 표시	10 · 2 · 16
`_이름`	안에서만 쓰는 것 (관례)	8
`대문자_이름`	안 바뀌는 설정값 (관례)	3
`0x…`	16진수 숫자. 색에 씀 (css 의 # 과 같음)	17
`/…/`	정규식 (글자 무늬 찾기). `/[가-힣]/.test(t)` = 한글이 있나	5
`#이름 (css)`	id 가 이름인 상자에게	2
`.이름 (css)`	class 에 이름이 있는 상자에게	2
`--이름 (css)`	색 · 값 이름표. `var(--이름)` 으로 꺼냄	2
`<태그> … </태그>`	html 상자 열고 닫기	2
`?v=260909a`	캐시버스터. 파일 고치면 올린다	12
`?memory=…&lang=ko`	주소창 값. `urlParams.get('lang')` 으로 읽음	6
`@@ -a,b +c,d @@`	비교표 덩어리 표시	15
`- +` (비교표)	지운 줄 · 넣은 줄	15
`./ ../`	같은 폴더 · 한 폴더 위	9

부록 B

코드 영어 단어장

이 프로젝트 코드에 실제로 자주 나오는 단어만 골랐습니다. 동사부터 외우면 함수 이름의 절반이 읽힙니다.

동사 (함수 이름 앞에 붙는 것)

단어	뜻	실제 이름
`get`	얻어 오다	getSupabaseClient, getState, getBucket
`set`	정하다 · 바꾸다	setState, setLanguage, setEnabled
`fetch / load`	서버 · 파일에서 가져오다	fetchMemories, loadMemoriesFromSupabase
`save / record / insert`	저장하다	savePlay, record-erosion, saveNote
`render / draw / show / hide`	그리다 · 보이다 · 숨기다	_renderShelf, showEndScreen, hideAllScreens
`init / setup / build`	준비하다 · 짓다	initApp, _setupAuthListener, buildDoor, buildBase
`detect / check / is / has`	감지 · 확인 (예/아니오)	detectCrisis, checkFixated, isEnabled, hasPast
`calculate / compute`	계산하다	calculateStep, computeFields
`pick / select / filter / sort`	고르다 · 거르다 · 정렬하다	pickTopMemory, filterVisibleMemories, sortMemories
`apply / update / merge`	적용 · 갱신 · 합치기	applyVisitorErosion, updateContamination
`enter / exit / start / stop / end`	들어가다 · 나가다 · 시작 · 멈춤 · 끝	enterArchive, exitOpeningToMenu, startFlow, stopSound, endConfession
`bind / handle / on`	사건을 붙이다 · 다루다	bindEvents, handleLogin, onAuthStateChange
`add / remove / push / clear`	더하다 · 빼다 · 끝에 넣다 · 비우다	classList.add, modal.remove, arr.push, clearInterval

`wait / resolve`	기다리다 · (약속이) 이뤄지다	waitForSupabaseClient, resolve()
`serialize / parse`	글자로 만들다 · 글자를 풀다	serializeLayer, JSON.parse
`rebuild / replay / reset / restart`	다시 짓다 · 다시 돌리다 · 되돌리다	rebuildFromPlays, replayFromPlays, resetSafetySession

상태와 자료

단어	뜻	실제 이름
`state / store`	상태 · 상태 창고	appStore.getState()
`current / prev / next / last`	현재 · 이전 · 다음 · 마지막	currentScene, prevState, next.cont_stage, lastEngineResult
`enabled / active / visible / hidden / ready`	켜짐 · 활성 · 보임 · 숨김 · 준비됨	_enabled, 'active', 'visible', 'hidden', 'ready'
`data / result / error / count / index / length`	자료 · 결과 · 오류 · 개수 · 번호 · 길이	{ data, error }, scenes.length, currentSceneIndex
`key / value / id / code / name / type`	이름표 · 값 · 번호 · 코드 · 이름 · 종류	FLAG_KEY, memory.id, memory.code, scene_type
`list / array / map / dist / vector`	목록 · 배열 · 대응표 · 분포 · 벡터	memoriesDataArray, EMOTION_ANCHOR_MAP, emotion_dist, userVector
`null / undefined / NaN / empty`	없음 · 정해지지 않음 · 숫자 아님 · 빔	currentMode: null, isNaN(master), _createEmptyResult
`true / false`	참 · 거짓	is_public: true
`default / fallback / option / flag`	기본값 · 대비책 · 선택 · 깃발(켜고 끄기)	default '{}', pickFallbackString, opt, FLAG_KEY
`threshold / scale / radius / weight / delta`	문턱값 · 배율 · 반경 · 무게 · 변화량	FIXATION_THRESHOLD, HEIGHT_SCALE, PHYS.radius, w, delta_conv

TEM 의 사물

단어	뜻	비고
`memory / scene / play / run`	기억 · 씬 · 회차 · 회차(같은 뜻)	표 이름
`user / session / auth / anon`	사용자 · 세션 · 인증 · 익명	auth.uid(), to anon
`client / server / request / response`	손님 쪽(브라우저) · 서버 · 요청 · 응답	supabaseClient, RequestBody
`event / listener / handler`	사건 · 듣는 것 · 다루는 함수	addEventListener
`element / node / document / window / canvas`	상자 · 마디 · 문서 · 창 · 그림판	DOM
`frame / tick / loop / raf`	프레임 · 한 박 · 반복 · requestAnimationFrame	17장
`terrain / layer / strata / erosion / uplift`	지형 · 층 · 지층 · 침식 · 융기	tem_variant_strata
`anchor / vector / emotion / valence / arousal`	닻(기준 감정) · 벡터 · 감정 · 쾌/불쾌 · 각성	VAD 의 V · A
`alignment / bucket / pattern / mismatch`	정렬도 · 구간 · 전이 패턴 · 어긋남	별이엔진 출력
`drift / fixation / contamination / stage`	표류 · 고착 · 오염 · 단계	ContaminationTracker
`ghost / variant / utterance / turn / seal`	유령 · 변주 · 발화 · 턴 · 봉인	대화 · 이본
`fog / pin / door / shelf / stain`	안개 · 핀 · 문 · 서가 · 얼룩	상영 연출

오류 문장

단어	뜻

`Uncaught`	아무도 안 잡은 (멈춤)
`Cannot read properties of null`	없는 것에 점을 찍음
`is not defined`	그런 이름 없음
`is not a function`	함수가 아닌데 ( ) 붙임
`already been declared`	같은 이름 두 번
`Unexpected token`	문법 깨짐 (괄호 · 따옴표)
`Failed to load resource`	파일 못 받음 (Network 보라)
`permission denied`	권한 없음 (403 · RLS)
`timeout`	기다리다 포기
`not found`	없음 (404)

부록 C

자주 보는 모양 카드

이 프로젝트에서 반복해서 나오는 모양 열 가지. 모양이 보이면 속을 안 읽어도 "무슨 종류의 일"인지 압니다.

카드 1 · 문지기

```
const el = document.getElementById('x');
if (!el) return;
```

없으면 나간다. 함수 첫머리. (4장)

카드 2 · 찾아서 종류표 바꾸기

```
const s = document.getElementById('introScreen');
if (s) { s.classList.remove('hidden'); s.classList.add('visible'); }
```

화면 전환의 거의 전부. css 가 종류표를 보고 옷을 갈아입힘. (2 · 13장)

카드 3 · 표 읽기

```
const { data, error } = await sb.from('scenes').select('*').eq('memory_id',
id).order('scene_order');
if (error) { console.error('…', error); return []; }
return data || [];
```

기다려서 · 묶음 풀고 · 오류면 빈 목록. (11 · 16장)

카드 4 · 저장하고 답 받기

```
const { data, error } = await
sb.from('plays').insert(row).select().single();
```

await 가 빠지면 저장 안 됨. (11장)

카드 5 · 반복하며 상자 만들기

```
list.forEach(item => {
  const el = document.createElement('div');
```

```
  el.className = '…';
  el.textContent = item.title;
  el.addEventListener('click', function () { … });
  parent.appendChild(el);
});
```

목록 → 카드들. 서가 · 아카이브 · 상담 목록. (7 · 13장)

카드 6 · 매 프레임 루프

```
function tick() {
  값 = 값 + 조금;
  그리기();
  if (아직) requestAnimationFrame(tick);
}
requestAnimationFrame(tick);
```

모든 애니메이션. (11 · 17장)

카드 7 · 사건 붙이기

```
대상.addEventListener('click', function (e) {
  if (조건 아니면) return;
  e.preventDefault();
  하는 일;
});
```

사건 → 문지기 → 하는 일. (13장)

카드 8 · 안전망

```
try { 해 보기; } catch (e) { console.error('[부품] …', e); }
try { 해 보기; } catch (_) {}     // 조용히 삼킴 - 원인이 숨는 자리
```

(14장)

카드 9 · 마당에 꽂기

```
window.getSupabaseClient = function () { return sb; };
window._temEnterSceneMode = enterSceneMode;   // 일반 script 에서 쓰게
```

방(module) → 마당(window). 메모에 이유. (12장)

카드 10 · 깃발 켜고 끄기

```
if (/[?&]variant_strata=1/.test(location.search)) on = true;
else if (/[?&]variant_strata=0/.test(location.search)) on = false;
try { localStorage.setItem(FLAG_KEY, on ? '1' : '0'); } catch (_) {}
```

주소창 `?x=1` 로 켜고, 브라우저 저장소에 기억. 테스터 모드 · 이본 지층 · 디버그 스위치가 전부 이 모양. (tem_variant_strata.js isEnabled)

부록 D

AI에게 시킬 문구 모음

그대로 복사해서 쓰세요. 18장 검사 순서 번호를 앞에 달았습니다.

#	문구
1	"고친 **파일 이름과 함수 이름**을 전부 나열해 줘. 함수마다 시작 줄과 끝 줄도."
2	"요약 말고 `git diff` **출력을 그대로** 붙여 줘. 덩어리(@@)가 몇 개인지 세고, 각 덩어리가 어느 함수 안인지 적어 줘."
2	"고친 함수를 `{` **부터** `}` **까지 통째로** 붙여 줘. 전과 후 둘 다."
3	"초록 줄을 **한 줄씩 한국어로** 풀어 줘. 계산 속은 빼고 '무엇을 하는가'만."
3	"이 숫자의 **단위**가 뭐야? (초 · 밀리초 · 픽셀 · 비율)"
4	"이 기능을 **진입점 세 곳**(index.html · play-test.html · demoFlow.js)에서 grep 한 결과를 붙여 줘. 빠진 문이 있으면 의도인지 말해 줘."
5	"고친 파일 이름을 전부 grep 해서 `?v=` **대조표**를 만들어 줘: 파일 · 나오는 html · 현재 ?v= · 오늘 날짜인지."
6	"화면을 열고 **콘솔과 네트워크 탭에 빨간 줄이 0개**인 캡처를 붙여 줘. 있으면 종류 · 내용 · 첫 at 줄을 적어 줘."
7	"콘솔에 값 심지 말고 **관객 순서 그대로**(서가 → 진입 → 걷기 → 대화 → 봉인) 끝까지 밟고, 단계마다 캡처를 붙여 줘. `npm run qa` 써도 돼."
—	"이 파일이 **몇 줄에서 몇 줄**로 바뀌었어? 다른 함수도 건드렸어?"
—	"이 열(칸)을 **읽는 코드**가 있어? grep 해서 `=` 오른쪽에 나오는 줄만 보여 줘." (죽은 레버 확인)
—	"이거 아무 데서도 안 부르는 것 같은데, **고치기 전에** 왜 남아 있는지 문서나 메모부터 찾아 줘." (죽은 코드 의도 확인)
—	"이 `catch`**가 삼키는 오류**가 뭐야? 콘솔에 남기게 바꾸면 무슨 위험이 있어?"
—	"이 호출 앞에 **await 있어?** 없으면 의도야?"

부록 E

미니 시험 정답

시험 1

① `js/ui/` 는 연출 층. 판단 엔진(`js/core/`)은 안 바뀜. 단 연출은 엔진의 답을 보여 주는 자리라, "엔진 답을 잘못 보여 줄" 가능성은 있음. ② 아니오. play-test.html 은 자기 안의 script 와 자기가 부른 js 만 돎. index.js 는 index.html 만 부름. 단 둘이 같이 쓰는 파일(math.js, ByeoriEngine.js, safety.js)은 양쪽에서 돎. ③ `supabase/migrations/`. 날짜시각_이름.sql 파일이 새로 생겨야 함.

시험 2

① css(옷). ② 둘 다 가진 상자. 점 사이에 공백이 없으면 "그리고"(둘 다), 공백이 있으면(`.a .b`) "a 안에 든 b". ③ 자바스크립트(신경)가 html(뼈대)의 상자를 찾는 동작.

시험 3

① `PHYS`. ② 묶음. 안에 `이름: 값`(쌍점)이 있음. ③ 9칸(inertia, capacity, minSlope, erode, deposit, evaporate, gravity, maxSteps, radius). 전부 숫자. ④ 안 바뀌는 설정값이라는 관례.

시험 4

① "지난 시간이 최대 대기 시간(10초) 이상이면" → 콘솔에 빨간 메모를 남기고 오류를 던져 멈춤(연결 포기). ② `start "" "주소"` (윈도우의 열기 명령). ③ 두 질문 다 아니오라서 맨 마지막 `xdg-open "주소"`.

시험 5

① `seedFromString`, 재료 1개(`s`), 몸통은 1번째 줄 `{`부터 6번째 줄 `}`. ② 예. 숫자(`h`가 5381 이라는 숫자로 시작해 계속 숫자 계산). ③ `'tem'`. ④ 이름(seed From String = 글자에서 씨앗을), 3번째 줄에서 재료를 `String()`으로 글자 취급, 답 `h`가 숫자. 세 군데가 다 말해 줌.

시험 6

① "주소창의 strata 값이 '1' 이고, memory 값이 있으면". ② 아니오. 바깥 몸통이 안 열리면 안의 정의도 생기지 않음. ③ 아니오. `return`으로 initApp 이 거기서 끝남.

시험 7

① 7 × 7 = 49번 (dz 7가지 × dx 7가지). 그중 원 밖은 `continue`로 건너뜀. ② `listeners` 목록을, 하나씩 `listener`라 부르며, 각각을 (state, prevState) 재료로 돌림. 하나가 실패해도 `catch`가 받아 기록만 하고 다음으로 가니 나머지는 돎. ③ "이름표 세 개(matchingSelection, modeSelection, sessionSetup)의 상자를 차례로 찾아, 있으면 숨긴다."

시험 8

① "next(다음 상태)의 평생 표류 합계를 driftSignal 만큼 늘려 담는다." ② `game`(회차 상태)의 지형 애니메이션 요청 번호. `requestAnimationFrame`이 돌려주는 번호가 담김(나중에 취소할 때 씀). ③ `(untitled)`. ④ 조용히 빔(`?.`가 두 번이라 어느 단계에서 없어도 멈추지 않음).

시험 9

① 7개. ② testerMode.js = js/app (같은 폴더, `./`), SeekerMatchEngine.js = js/core (한 칸 위로 가서 core, `../core/`). ③ 빌려 온 것. `js/app/confession.js`. ④ 호출(이름 뒤에 바로 괄호).

시험 10

① 목록. 2개. ② 5칸(text, entryLine, vad, x, y). `vad`가 안에 또 묶음(v, a, d). ③ `MONOLOGUES[1].vad.a` → 0.7. ④ 묶음 하나. 칸 2개(block, behavior). 함수에 "설정 묶음"을 재료로 넘기는 흔한 모양.

시험 11

① "1.5초 기다린다(그동안 브라우저는 다른 일을 할 수 있음)." ② 브라우저 안에서 저 함수를 돌려 memories 표에서 id 한 개를 읽어 오고, 그 답이 올 때까지 기다렸다가 memId 에 담음. ③ 0.2초 뒤에 `_waitForStrata`를 부름. 그 줄이 `_waitForStrata` 안에 있으면 "준비될 때까지 0.2초마다 자기를 다시 부르는" 반복(실제로 그렇게 되어 있음, index.js 147~158).

시험 12

① 방(module) 안에서 import 한 `QuiltDemoState`를 마당(window)에 꽂아, 일반 script 인 발자국 · 모의 시험 코드도 쓰게 함. ② 문제 없음(둘 다 260810a). 한쪽이 260714 였다면 그 문으로 들어온 관객은 옛 파일을 받아, 8월 10일 이후 고친 것(근접 문구 · 자막 확대)이 안 보임. ③ safety.js 맨 끝에 `if (typeof window !== 'undefined') { window.… = { detectCrisis, … } }` 모양으로 마당에 묶음을 꽂아 두었기 때문. 어떤 이름으로 꽂았는지는 파일 끝(약 269~283번째 줄)을 열면 보임. 15장의 safety.js 비교표 둘째 덩어리가 바로 그 자리.

시험 13

① 어두운 배경(modal 상자 자체)을 눌렀을 때만(`e.target === modal`). 창 안의 글을 누르면 target 이 안쪽 상자라 안 닫힘. ② "종류표 footer 인 첫 상자를 찾아, 있으면 visible 을 떼어 숨긴다." `if (footer)`가 없으면 그 상자가 없는 화면에서 `null.classList` 가 되어 TypeError 로 멈춤. ③ 대상 = 소리(`audio`), 사건 = 재생 위치가 바뀔 때(`timeupdate`). 재생 중 계속(초에 몇 번씩) 일어남. 끝나기 직전 소리를 줄이는 데 씀.

시험 14

① 인증 부품(`[Auth]`)이 "로그인 상태가 바뀌었다(SIGNED_IN), 이 사용자로"라고 알림. 정상 기록. ② `fetchScenes`의 `if (error)` 갈래(4장 ③의 두 번째 문지기). 데이터베이스가 scenes 표 읽기를 권한 거부(42501). 함수가 빈 목록을 돌려주니 화면에 씬이 하나도 안 뜸. ③ 후보: 경로 · 파일 이름 오타, 파일이 옮겨졌거나 지워짐. 아닌 원인: `?v=` 숫자. 서버는 물음표 뒤를 무시하므로 캐시버스터 값이 틀려도 404 는 안 남(304 나 옛 파일 문제는 나도).

시험 15

① `loadMemoriesFromSupabase` 안. 덩어리 1개. ② 전에는 합친 목록(merged)을 그대로 상태에 넣음. 지금은 "보이는 기억만 거르는" 함수를 필요할 때 꺼내와, 현재 사용자 번호를 구한 뒤, 걸러진 목록을 상태에 넣음(v4 로 기록된 씨앗은 기록자 본인에게만 보이게). ③ 알 수 있는 것: 44줄 늘고 0줄 줄어 "추가만" 했다. 알 수 없는 것: 어느 함수에, 무슨 일을 하는 코드인지. 비교표를 봐야 함.

시험 16

① `trajectory_bridges` 표에 `contact_utterance`(글) 열과 `contact_turn`(정수) 열을 더함. 이미 있으면 그 열은 건너뜀(오류 없음). ② "plays 표에 playData 묶음을 새 행으로 넣고, 넣은 그 행을 돌려받되 하나만." ③ 아니오. 정책이 `auth.uid() = recipient_id`(로그인한 사람 = 받는 사람)일 때만 읽기 허용이므로, 보낸 사람은 못 읽음(보낸 사람용 정책이 따로 없는 한).

시험 17

① 가로 0.15 · 높이 3.2 · 깊이 0.15 인 상자 모양에, 나무색 재질(frameMat)로, 오른쪽 0.7 · 위 1.6 · 앞뒤 0 에 놓음. 왼쪽 기둥과 다른 점은 x 가 -0.7 이 아니라 0.7. ② `--accent-memory: #c4a882`. 기억의 금색. ③ `requestAnimationFrame`. (짧게 `Raf` 나 `tick` 도 단서.) ④ 물체마다 `o.userData._exitDoor` 가 참인지.

시험 18 (예시)

① 다섯 질문 예: "어느 파일 어느 함수야?"(1 · 5장) / "git diff 그대로 붙여 줘, @@ 몇 개야?"(15장) / "초록 줄 한 줄씩 한국어로"(4 · 7장) / "오염 계산은 index.html 쪽 상영에서도 쓰이는데, 세 문 grep 결과는?"(12장) / "ContaminationTracker.js 의 ?v= 올렸어? index.html 과 play-test.html 둘 다?"(12장) / "콘솔 빨강 0 캡처"(14장) / "관객 순서로 끝까지 밟은 캡처"(18장). ② 빠진 문: index.html(오프닝 → 상영 진입 경로)과 demoFlow. "잘 돼요"가 play-test 직접 열기만 뜻한다면 오프닝을 거쳐 들어가는 경로는 안 밟은 것. 그 경로로 한 번 더. ③ 안 맞음. "계산만"인데 연출(`js/ui/`) 파일이 둘 바뀜. "왜 ui 파일을 건드렸어? 계산 결과를 보여 주는 자리라서야, 아니면 다른 것도 같이 고친 거야?" 를 묻고, 그 두 덩어리의 초록 줄을 따로 읽음.

시험 19

① S가 1.00이면 형태를 아직 안 잰 것 = 장면 셋 미만, 즉 첫째나 둘째 장면. V 0.70은 그 장면에서 관객은 회피했고 작가는 감정을 적어 둔 경우. ② 고장 아님. `plays` 표에는 정렬도와 어긋남 종류, 원시 감정만 남기고 등급 · 패턴은 안 남기는 설계(19장). 정렬도가 있으니 등급은 다시 계산 가능. ③ 성립 안 함. `ByeoriEngine.js` 21~23번째 줄 `constructor() { // 상태 없음 }`. 지금까지 궤적 · 점수 · 이전 등급은 전부 부르는 쪽이 재료로 넘김. 이상하면 엔진이 아니라 넘긴 재료(play-test 의 `game.userEmotionTrajectory`, `game.sceneScores` 등)를 봐야 함.

시험 20

① 곱해서 더함 0.6×0.8 + 0.8×0.6 = 0.96. 길이 둘 다 √(0.36+0.64) = 1. 코사인 **0.96**. ② **0**. 직각. 코사인은 "같은 칸에 같이 값이 있나"만 봄. 둘 다 고통이어도 칸이 다르면(공포 vs 슬픔) 90도. 17칸은 서로 독립이라 "고통끼리는 비슷"이라는 상위 묶음이 코사인엔 없음. ③ 그 두 칸만으로 잼. `sadness: 0.9`는 목록에 없으니 계산에서 통째로 빠짐(없는 칸 취급, 20장 코드 15~17번째 줄). 기억별 좌표계가 이렇게 작동 — 그 기억이 안 쓰는 감정은 안 잼.

시험 21

① 형태 날것 −0.2 → 깎아서 0. 회피 0.7. 정렬도 = 0.8 × 0 × 0.7 = **0**. 음수가 안 되는 건 `calculateShapeSimilarity` 14번째 줄 `Math.max(0, raw)`와 `calculateTrajectoryAlignment`의 `Math.max(0, …)` 두 겹. ② **HIGH**(끈적임: 직전 HIGH 이고 0.42 ≥ 0.40). 직전이 MID 였다면 끈적임 없음 → 표준 판정 0.42 < 0.50 → **MID**. ③ **attribution_mismatch**. 탓 검사(129~130)가 감정 검사(142~147)보다 위에 있어 먼저 걸림. 감정이 비슷해도 탓이 다르면 그게 답. ④ **displacement**. 155번째 줄(둘째 갈림길): 수준 0.6 ≥ 0.5, 형태 0.2 < 0.3, 장면 셋. HIGH 검사(157)보다 위라 echo_follow 가 안 됨.

시험 22

① "논문 값이 아니라 설계 값입니다. V3 의 태도 항이 거의 항상 0.7 로 고정돼 상수처럼 굴던 걸, 회피가 일어났을 때만 곱하는 보정으로 바꾸면서 이어받은 값이고, 파일럿 분포를 보고 조정하는 대상입니다." ② 0.5×0.97 + 0.5×0 = **0.485**. 움직임이 정반대인 관객이 "반쯤 비슷"으로 나옴. 한 항이 다른 항을 메움 — V3 의 ③ 문제가 그대로 돌아옴. ③ "식의 '원본'은 진실이 아니라 작가의 첫 서술 궤적이고, 정렬도는 그 첫 서술과 같은 결로 겪었는지를 잽니다."

시험 23

① 0.1×0.8 + 0.9×0.30 = 0.08 + 0.27 = **0.35**. 0.35 ≥ 0.35 → **biased_inclination**(편향 기울기). ② **hypercompletion**(과완결). 245~248번째 줄: 고착 0.70 ≥ 0.65 이고 0.70 > 표류 0.35. 과완결 검사가 먼저라 이김. ③ "엔진의 정렬도 계산을 바꿨다"가 위반. 오염은 엔진 출력의 소비자이고 엔진을 안 건드림 — `ContaminationTracker.js` 13번째 줄 "별이엔진 V4는 변경하지 않는다. 이 모듈은 V4 출력의 소비자이다". 오염이 느리면 α(27번째 줄)나 단계 문턱(61~64번째 줄)을 만져야지 `ByeoriEngine.js`를 만지면 안 됨. 그리고 바꿨으면 어느 손잡이를 얼마에서 얼마로 바꿨는지 기록(22장 끝 함정 상자).

— 끝 —